

Human review packet: PZMH20PerformativePrediction

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: PZMH20PerformativePrediction

Paper id: PZMH20PerformativePrediction

Source version: PMLR 119 official paper text, byte-pinned in cited publication; arXiv v4 retained for documented alternate-proof provenance

Source record: audit/paper_statement_map.json

Rows: 11

Generated: 2026-09-27

Source URL: [official source](#)

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper PZMH20PerformativePrediction --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

Contents

- [Governing Lean declarations \(176; supporting context\)](#)
- [Semantic library prerequisites \(13; not paper claims\)](#)
 - [AppliedModelingLib.IsJointlySmoothGradient](#)
 - [AppliedModelingLib.MeasurePerformativeModel](#)
 - [AppliedModelingLib.IsMeasurePointwiseGradientStronglyConvex](#)
 - [AppliedModelingLib.IsMeasureWassersteinSensitive](#)
 - [AppliedModelingLib.MeasurePerformativeModelOn](#)
 - [AppliedModelingLib.MeasurePerformativeModelOn.IsDataLossLipschitz](#)
 - [AppliedModelingLib.MeasurePerformativeModelOn.decoupledPerformativeRisk](#)
 - [AppliedModelingLib.MeasurePerformativeModelOn.performativeRisk](#)
 - [AppliedModelingLib.MeasurePerformativeModelOn.IsPerformativelyOptimal](#)
 - [AppliedModelingLib.MeasurePerformativeModelOn.IsPerformativelyStable](#)
 - [AppliedModelingLib.MeasurePerformativeModelOn.IsPointwiseGradientStronglyConvex](#)
 - [AppliedModelingLib.MeasurePerformativeModelOn.IsWassersteinSensitive](#)
 - [AppliedModelingLib.measureDecoupledPerformativeRisk](#)
- [Paper-specific semantic prerequisites \(6; not paper claims\)](#)
 - [PZMH20PerformativePrediction.IsMeasurePerformativelyOptimal](#)
 - [PZMH20PerformativePrediction.IsMeasureRRMOn](#)
 - [PZMH20PerformativePrediction.IsMeasureStable](#)
 - [PZMH20PerformativePrediction.IsMeasureWassersteinSensitive](#)
 - [PZMH20PerformativePrediction.Theorem310CorrectedRGDAllRoundTrajectoryData](#)
 - [PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData](#)
- [Main-text source claims \(11\)](#)
 - [Theorem 3.5](#)
 - [Theorem 3.8](#)

- Proposition 4.1
- Proposition 4.2
- Theorem 4.3
- Corollary 5.1
- Proposition 3.6
- Proposition 3.6
- Proposition 3.6
- Theorem 3.10
- Theorem 3.10

Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

AppliedModelingLib.FiniteCoupling

Declaration location: reusable library

Lean declaration body

```
field law:
  {α : Type u_1} → {β : Type u_2} → {μ : PMF α} → {ν : PMF β} → AppliedModelingLib.FiniteCoupling μ ν → PMF (α × β)

field fst_marginal:
  ∀ {α : Type u_1} {β : Type u_2} {μ : PMF α} {ν : PMF β} (self : AppliedModelingLib.FiniteCoupling μ ν),
    PMF.map Prod.fst self.law = μ

field snd_marginal:
  ∀ {α : Type u_1} {β : Type u_2} {μ : PMF α} {ν : PMF β} (self : AppliedModelingLib.FiniteCoupling μ ν),
    PMF.map Prod.snd self.law = ν
```

AppliedModelingLib.FinitePerformativeModel

Declaration location: reusable library

Lean declaration body

```
field dataLaw:
  {Parameter : Type u_1} →
  {Data : Type u_2} →
  [inst : Fintype Data] →
  [inst_1 : DecidableEq Data] → AppliedModelingLib.FinitePerformativeModel Parameter Data → Parameter → PMF Data

field loss:
  {Parameter : Type u_1} →
  {Data : Type u_2} →
  [inst : Fintype Data] →
  [inst_1 : DecidableEq Data] → AppliedModelingLib.FinitePerformativeModel Parameter Data → Data → Parameter → ℝ
```

AppliedModelingLib.HeterogeneousBatchTrace

Declaration location: reusable library

Lean declaration body

```
type:
  (ℕ → Type u_1) → Type u_2 → ℕ → Type (max u_1 u_2)

value:
  fun (BatchIndex : ℕ → Type u_1) (Data : Type u_2) (iteration : ℕ) => (round : Fin iteration) → BatchIndex ↑ round → Data
```

AppliedModelingLib.HeterogeneousBatchTraceCoordinate

Declaration location: reusable library

Lean declaration body

```
type:
  (ℕ → Type u_1) → Type u_2 → ℕ → Type (max u_1 u_2)

value:
  fun (BatchIndex : ℕ → Type u_1) (Data : Type u_2) (a : ℕ) =>
    match a with
    | 0 => PUnit.{(max u_1 u_2) + 1}
    | round.succ => BatchIndex round → Data
```

AppliedModelingLib.IsDataGradientLipschitz

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : NormedAddCommGroup Parameter] →
[InnerProductSpace ℝ Parameter] → [MetricSpace Data] → (Data → Parameter → Parameter) → NNReal → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [NormedAddCommGroup Parameter] [InnerProductSpace ℝ Parameter]
[MetricSpace Data] (gradient : Data → Parameter → Parameter) (smoothness : NNReal) =>
  ∀ (first second : Data) (parameter : Parameter),
    ||gradient first parameter - gradient second parameter|| ≤ ↑smoothness * dist first second
```

AppliedModelingLib.IsVariationalEuclideanProjectionOn

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
[inst : NormedAddCommGroup Parameter] →
[InnerProductSpace ℝ Parameter] → Set Parameter → (Parameter → Parameter) → Prop

value:
fun {Parameter : Type u_1} [NormedAddCommGroup Parameter] [InnerProductSpace ℝ Parameter] (domain : Set Parameter)
(project : Parameter → Parameter) =>
  (∀ (input : Parameter), project input ∈ domain) ∧
  ∀ (input candidate : Parameter),
    candidate ∈ domain → inner ℝ (input - project input) (candidate - project input) ≤ 0
```

AppliedModelingLib.Math.dyadicEffectiveCountDepth

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → ℝ → ℕ

value:
fun (dimension : ℕ) (effectiveCount : ℝ) => [Real.logb (↑(2 ^ dimension)) (effectiveCount / ↑(2 ^ dimension))]]+
```

AppliedModelingLib.Math.dyadicGeometricEffectiveDepth

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → ℝ → ℝ → ℕ

value:
fun (dimension : ℕ) (effectiveCount confidence : ℝ) =>
  if 16 * ↑(2 ^ dimension) ≤ effectiveCount ∧ 8 * confidence ≤ effectiveCount then
    AppliedModelingLib.Math.dyadicEffectiveCountDepth dimension (effectiveCount / 16)
  else 0
```

Direct retained dependencies

- [AppliedModelingLib.Math.dyadicEffectiveCountDepth](#)

AppliedModelingLib.Math.expConfidenceForHalfBudget

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → ℝ

value:
fun (budget : ℝ) => max 0 (Real.log (2 / budget))
```

AppliedModelingLib.Math.positiveNatCeil

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{N}$ 

value:
fun (threshold :  $\mathbb{R}$ ) => [max 1 threshold]₊
```

AppliedModelingLib.MeasurePerformativeSamplingKernel

Declaration location: reusable library

Lean declaration body

```
field kernel:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Parameter] →
[inst_1 : MeasurableSpace Data] →
{model : AppliedModelingLib.MeasurePerformativeModel Parameter Data} →
AppliedModelingLib.MeasurePerformativeSamplingKernel model → ProbabilityTheory.Kernel Parameter Data

field isMarkov:
 $\forall$  {Parameter : Type u_1} {Data : Type u_2} [inst : MeasurableSpace Parameter] [inst_1 : MeasurableSpace Data]
{model : AppliedModelingLib.MeasurePerformativeModel Parameter Data}
(self : AppliedModelingLib.MeasurePerformativeSamplingKernel model), ProbabilityTheory.IsMarkovKernel self.kernel

field apply_eq_dataLaw:
 $\forall$  {Parameter : Type u_1} {Data : Type u_2} [inst : MeasurableSpace Parameter] [inst_1 : MeasurableSpace Data]
{model : AppliedModelingLib.MeasurePerformativeModel Parameter Data}
(self : AppliedModelingLib.MeasurePerformativeSamplingKernel model) (parameter : Parameter),
self.kernel parameter =  $\uparrow$ (model.dataLaw parameter)
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)

AppliedModelingLib.Probability.HasExponentialRadialMoment

Declaration location: reusable library

Lean declaration body

```
type:
{E : Type u_1} → [inst : MeasurableSpace E] → [NormedAddCommGroup E] → MeasureTheory.ProbabilityMeasure E →  $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{Prop}$ 

value:
fun {E : Type u_1} [MeasurableSpace E] [NormedAddCommGroup E] (law : MeasureTheory.ProbabilityMeasure E)
(alpha gamma :  $\mathbb{R}$ ) =>
MeasureTheory.Integrable (fun (x : E) => Real.exp (gamma *  $\|x\|$ .rpow alpha))  $\uparrow$ law
```

AppliedModelingLib.Probability.HasBoundedExponentialRadialMoment

Declaration location: reusable library

Lean declaration body

```
type:
{E : Type u_1} →
[inst : MeasurableSpace E] → [NormedAddCommGroup E] → MeasureTheory.ProbabilityMeasure E →  $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{Prop}$ 

value:
fun {E : Type u_1} [MeasurableSpace E] [NormedAddCommGroup E] (law : MeasureTheory.ProbabilityMeasure E)
(alpha gamma momentBound :  $\mathbb{R}$ ) =>
AppliedModelingLib.Probability.HasExponentialRadialMoment law alpha gamma  $\wedge$ 
 $\int (x : E), \text{Real.exp} (gamma * \|x\| . \text{rpow } alpha) \partial \uparrow law \leq \text{momentBound}$ 
```

Direct retained dependencies

- [AppliedModelingLib.Probability.HasExponentialRadialMoment](#)

AppliedModelingLib.Probability.dyadicCellIndex

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \text{Type}$ 

value:
fun (level :  $\mathbb{N}$ ) => Fin (2 ^ level)
```

AppliedModelingLib.Probability.dyadicCubeIndex

Declaration location: reusable library

Lean declaration body

type:
 $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Type}$

value:
fun (dimension level : $\mathbb{N}$) => Fin dimension $\rightarrow$ AppliedModelingLib.Probability.dyadicCellIndex level

Direct retained dependencies

- [AppliedModelingLib.Probability.dyadicCellIndex](#)

AppliedModelingLib.Probability.euclideanDyadicCubeRadius

Declaration location: reusable library

Lean declaration body

type:
 $\mathbb{N} \rightarrow \mathbb{R}$

value:
fun (scale : $\mathbb{N}$) => $\uparrow(2^{\text{scale}})$

AppliedModelingLib.Probability.euclideanDyadicCube

Declaration location: reusable library

Lean declaration body

type:
(dimension : $\mathbb{N}$) $\rightarrow$ $\mathbb{N} \rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin dimension) $\rightarrow$ Prop

value:
fun (dimension scale : $\mathbb{N}$) (a : EuclideanSpace $\mathbb{R}$ (Fin dimension)) =>
{x : EuclideanSpace $\mathbb{R}$ (Fin dimension) |
 $\forall$ (coordinate : Fin dimension),
 x.ofLp coordinate $\in$
 Set.loc (-AppliedModelingLib.Probability.euclideanDyadicCubeRadius scale)
 (AppliedModelingLib.Probability.euclideanDyadicCubeRadius scale)}
a

Direct retained dependencies

- [AppliedModelingLib.Probability.euclideanDyadicCubeRadius](#)

AppliedModelingLib.Probability.euclideanDyadicCubeShell

Declaration location: reusable library

Lean declaration body

type:
(dimension : $\mathbb{N}$) $\rightarrow$ $\mathbb{N} \rightarrow$ EuclideanSpace $\mathbb{R}$ (Fin dimension) $\rightarrow$ Prop

value:
fun (dimension shell : $\mathbb{N}$) (a : EuclideanSpace $\mathbb{R}$ (Fin dimension)) =>
(match shell with
| 0 => AppliedModelingLib.Probability.euclideanDyadicCube dimension 0
| shell.succ =>
 AppliedModelingLib.Probability.euclideanDyadicCube dimension (shell + 1) \
 AppliedModelingLib.Probability.euclideanDyadicCube dimension shell)
a

Direct retained dependencies

- [AppliedModelingLib.Probability.euclideanDyadicCube](#)

AppliedModelingLib.Probability.fifthOrderExponentialRadialTailConstant

Declaration location: reusable library

Lean declaration body

```
type:
{E : Type u_1} → [inst : MeasurableSpace E] → [NormedAddCommGroup E] → MeasureTheory.ProbabilityMeasure E → ℝ → ℝ → ℝ

value:
fun {E : Type u_1} [MeasurableSpace E] [NormedAddCommGroup E] (law : MeasureTheory.ProbabilityMeasure E)
  (alpha gamma : ℝ) =>
  ↑(Nat.factorial 5) * gamma-1 ^ 5 * ∫ (x : E), Real.exp (gamma * ||x||.rpow alpha) ∂ ↑law
```

AppliedModelingLib.Probability.finSnocMeasurableEquiv

Declaration location: reusable library

Lean declaration body

```
type:
{beta : Type u_2} → [inst : MeasurableSpace beta] → (n : ℕ) → (Fin n → beta) × beta ≈m (Fin (n + 1) → beta)

value:
fun {beta : Type u_2} [MeasurableSpace beta] (n : ℕ) =>
  MeasurableEquiv.prodComm.trans (MeasurableEquiv.piFinSuccAbove (fun (x : Fin (n + 1)) => beta) (Fin.last n)).symm
```

AppliedModelingLib.Probability.finiteIIDCellFrequency

Declaration location: reusable library

Lean declaration body

```
type:
{E : Type u_1} → [MeasurableSpace E] → (count : ℕ) → Set E → (Fin count → E) → ℝ

value:
fun {E : Type u_1} [MeasurableSpace E] (count : ℕ) (cell : Set E) (sample : Fin count → E) =>
  (∑ index : Fin count, cell.indicator (fun (x : E) => 1) (sample index)) / ↑count
```

AppliedModelingLib.Probability.finiteIIDEuclideanDyadicCubeShellMassRelativeTail

Declaration location: reusable library

Lean declaration body

```
type:
(dimension : ℕ) →
  MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension)) →
  (count : ℕ) → ℕ → ℝ → (Fin count → EuclideanSpace ℝ (Fin dimension)) → Prop

value:
fun (dimension : ℕ) (law : MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension))) (count shell : ℕ)
  (z : ℝ) (a : Fin count → EuclideanSpace ℝ (Fin dimension)) =>
  {sample : Fin count → EuclideanSpace ℝ (Fin dimension) |
    (↑law).real (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell) * z ≤
    |AppliedModelingLib.Probability.finiteIIDCellFrequency count
    (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell) sample -
    (↑law).real (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell)|}
a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.euclideanDyadicCubeShell](#)
- [AppliedModelingLib.Probability.finiteIIDCellFrequency](#)

AppliedModelingLib.Probability.finiteIIDEuclideanDyadicCubeShellMassRelativeTailTotalized

Declaration location: reusable library

Lean declaration body

```
type:
(dimension : ℕ) →
  MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension)) →
  (count : ℕ) → ℕ → ℝ → (Fin count → EuclideanSpace ℝ (Fin dimension)) → Prop

value:
fun (dimension : ℕ) (law : MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension))) (count shell : ℕ)
  (z : ℝ) (a : Fin count → EuclideanSpace ℝ (Fin dimension)) =>
  (if law.toFiniteMeasure.restrict (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell) = 0 then
    {sample : Fin count → EuclideanSpace ℝ (Fin dimension) |
      ∃ (index : Fin count), sample index ∈ AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell}
  else AppliedModelingLib.Probability.finiteIIDEuclideanDyadicCubeShellMassRelativeTail dimension law count shell z)
a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.euclideanDyadicCubeShell](#)
- [AppliedModelingLib.Probability.finiteIIDEuclideanDyadicCubeShellMassRelativeTail](#)

AppliedModelingLib.Probability.finiteIIDSampleLaw

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] → MeasureTheory.Measure X → (count : ℕ) → MeasureTheory.Measure (Fin count → X)

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure X) (count : ℕ) =>
MeasureTheory.Measure.pi fun (x : Fin count) => law
```

AppliedModelingLib.Probability.finiteIIDSampleMap

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} → {Y : Type u_2} → {count : ℕ} → (X → Y) → (Fin count → X) → Fin count → Y

value:
fun {X : Type u_1} {Y : Type u_2} {count : ℕ} (f : X → Y) (a : Fin count → X) (a_1 : Fin count) => f (a a_1)
```

AppliedModelingLib.Probability.finiteIIDSampleOnFinset

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} → {count : ℕ} → (inside : Finset (Fin count)) → (Fin count → X) → 1 inside → X

value:
fun {X : Type u_1} {count : ℕ} (inside : Finset (Fin count)) (a : Fin count → X) (a_1 : 1 inside) => a ↑a_1
```

AppliedModelingLib.Probability.finiteIIDSampleOnFinsetToFin

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} → {count : ℕ} → (inside : Finset (Fin count)) → (Fin count → X) → Fin inside.card → X

value:
fun {X : Type u_1} {count : ℕ} (inside : Finset (Fin count)) (a : Fin count → X) (a_1 : Fin inside.card) =>
AppliedModelingLib.Probability.finiteIIDSampleOnFinset inside a
(((finCongr (AppliedModelingLib.Probability.finiteIIDSampleOnFinsetToFin._proof_1 inside)).trans
(Fintype.equivFin 1 inside).symm)
a_1)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.finiteIIDSampleOnFinset](#)

AppliedModelingLib.Probability.finiteKernelProduct

Declaration location: reusable library

Lean declaration body

```
type:
{alpha : Type u_1} →
{beta : Type u_2} →
[inst : MeasurableSpace alpha] →
[inst_1 : MeasurableSpace beta] →
(n : ℕ) → (Fin n → ProbabilityTheory.Kernel alpha beta) → ProbabilityTheory.Kernel alpha (Fin n → beta)

value:
fun {alpha : Type u_1} {beta : Type u_2} [MeasurableSpace alpha] [MeasurableSpace beta] (n : ℕ)
(a : Fin n → ProbabilityTheory.Kernel alpha beta) =>
Nat.brecOn (motive := fun (x : ℕ) =>
(Fin x → ProbabilityTheory.Kernel alpha beta) → ProbabilityTheory.Kernel alpha (Fin x → beta)) n
AppliedModelingLib.Probability.finiteKernelProduct._f a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.finSnocMeasurableEquiv](#)

AppliedModelingLib.Probability.finiteIIDBatchKernel

Declaration location: reusable library

Lean declaration body

```
type:
{alpha : Type u_1} →
{beta : Type u_2} →
[inst : MeasurableSpace alpha] →
[inst_1 : MeasurableSpace beta] →
(n : ℕ) → ProbabilityTheory.Kernel alpha beta → ProbabilityTheory.Kernel alpha (Fin n → beta)

value:
fun {alpha : Type u_1} {beta : Type u_2} [MeasurableSpace alpha] [MeasurableSpace beta] (n : ℕ)
(kappa : ProbabilityTheory.Kernel alpha beta) =>
AppliedModelingLib.Probability.finiteKernelProduct n fun (x : Fin n) => kappa
```

Direct retained dependencies

- [AppliedModelingLib.Probability.finiteKernelProduct](#)

AppliedModelingLib.Probability.halfOpenUnitInterval

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → Prop

value:
fun (a : ℝ) => Set.loc (-1) 1 a
```

AppliedModelingLib.Probability.HalfOpenUnitCube

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → Type

value:
fun (dimension : ℕ) =>
{ x : EuclideanSpace ℝ (Fin dimension) //
  ∀ (i : Fin dimension), x.ofLp i ∈ AppliedModelingLib.Probability.halfOpenUnitInterval }
```

Direct retained dependencies

- [AppliedModelingLib.Probability.halfOpenUnitInterval](#)

AppliedModelingLib.Probability.euclideanDyadicCubeShellScale

Declaration location: reusable library

Lean declaration body

```
type:
  (dimension shell : ℕ) →
  ↑ (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell) →
  AppliedModelingLib.Probability.HalfOpenUnitCube dimension

value:
  fun (dimension shell : ℕ) (a : ↑ (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell)) =>
  ((AppliedModelingLib.Probability.euclideanDyadicCubeRadius shell)-1 • ↑ a,
  AppliedModelingLib.Probability.euclideanDyadicCubeShellScale._proof_1 dimension shell a)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.HalfOpenUnitCube](#)
- [AppliedModelingLib.Probability.euclideanDyadicCubeRadius](#)
- [AppliedModelingLib.Probability.euclideanDyadicCubeShell](#)
- [AppliedModelingLib.Probability.halfOpenUnitInterval](#)

AppliedModelingLib.Probability.euclideanDyadicCubeShellScaleExtension

Declaration location: reusable library

Lean declaration body

```
type:
  (dimension : ℕ) → ℕ → EuclideanSpace ℝ (Fin dimension) → AppliedModelingLib.Probability.HalfOpenUnitCube dimension

value:
  fun (dimension shell : ℕ) (a : EuclideanSpace ℝ (Fin dimension)) =>
  Classical.choose
  (AppliedModelingLib.Probability.exists_measurable_euclideanDyadicCubeShellScaleExtension dimension shell) a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.HalfOpenUnitCube](#)
- [AppliedModelingLib.Probability.euclideanDyadicCubeShell](#)
- [AppliedModelingLib.Probability.euclideanDyadicCubeShellScale](#)
- [AppliedModelingLib.Probability.halfOpenUnitInterval](#)

AppliedModelingLib.Probability.euclideanDyadicCubeShellCompactLaw

Declaration location: reusable library

Lean declaration body

```
type:
  (dimension : ℕ) →
  ℕ →
  MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension)) →
  MeasureTheory.ProbabilityMeasure (AppliedModelingLib.Probability.HalfOpenUnitCube dimension)

value:
  fun (dimension shell : ℕ) (law : MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension))) =>
  ((law.toFiniteMeasure.restrict (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell)).map
  (AppliedModelingLib.Probability.euclideanDyadicCubeShellScaleExtension dimension shell)).normalize
```

Direct retained dependencies

- [AppliedModelingLib.Probability.HalfOpenUnitCube](#)
- [AppliedModelingLib.Probability.euclideanDyadicCubeShell](#)
- [AppliedModelingLib.Probability.euclideanDyadicCubeShellScaleExtension](#)
- [AppliedModelingLib.Probability.halfOpenUnitInterval](#)

AppliedModelingLib.Probability.normalizedUnitCoordinate

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (x :  $\mathbb{R}$ ) => (x + 1) / 2
```

AppliedModelingLib.Probability.dyadicCoordinateNumber

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{N}$ 

value:
fun (level :  $\mathbb{N}$ ) (x :  $\mathbb{R}$ ) =>  $\lceil \uparrow (2^{\text{level}}) * \text{AppliedModelingLib.Probability.normalizedUnitCoordinate } x \rceil - 1$ 
```

Direct retained dependencies

- [AppliedModelingLib.Probability.normalizedUnitCoordinate](#)

AppliedModelingLib.Probability.dyadicCoordinateIndex

Declaration location: reusable library

Lean declaration body

```
type:
(level :  $\mathbb{N}$ ) →
(x :  $\mathbb{R}$ ) →
x ∈ AppliedModelingLib.Probability.halfOpenUnitInterval → AppliedModelingLib.Probability.dyadicCellIndex level

value:
fun (level :  $\mathbb{N}$ ) (x :  $\mathbb{R}$ ) (hx : x ∈ AppliedModelingLib.Probability.halfOpenUnitInterval) =>
(AppliedModelingLib.Probability.dyadicCoordinateNumber level x,
AppliedModelingLib.Probability.dyadicCoordinateNumber_lt level x hx)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.dyadicCellIndex](#)
- [AppliedModelingLib.Probability.dyadicCoordinateNumber](#)
- [AppliedModelingLib.Probability.halfOpenUnitInterval](#)

AppliedModelingLib.Probability.dyadicCubeLocate

Declaration location: reusable library

Lean declaration body

```
type:
(dimension level :  $\mathbb{N}$ ) →
AppliedModelingLib.Probability.HalfOpenUnitCube dimension →
Fin dimension → AppliedModelingLib.Probability.dyadicCellIndex level

value:
fun (dimension level :  $\mathbb{N}$ ) (x : AppliedModelingLib.Probability.HalfOpenUnitCube dimension) (a : Fin dimension) =>
AppliedModelingLib.Probability.dyadicCoordinateIndex level (( $\uparrow$  x).ofLp a)
(AppliedModelingLib.Probability.dyadicCubeLocate._proof_1 dimension x a)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.HalfOpenUnitCube](#)
- [AppliedModelingLib.Probability.dyadicCellIndex](#)
- [AppliedModelingLib.Probability.dyadicCoordinateIndex](#)
- [AppliedModelingLib.Probability.halfOpenUnitInterval](#)

AppliedModelingLib.Probability.dyadicPartitionL1Deviation

Declaration location: reusable library

Lean declaration body

```
type:
  (dimension : ℕ) →
    MeasureTheory.ProbabilityMeasure (AppliedModelingLib.Probability.HalfOpenUnitCube dimension) →
      (count : ℕ) → ℕ → (Fin count → AppliedModelingLib.Probability.HalfOpenUnitCube dimension) → ℝ

value:
  fun (dimension : ℕ) (law : MeasureTheory.ProbabilityMeasure (AppliedModelingLib.Probability.HalfOpenUnitCube dimension))
    (count level : ℕ) (sample : Fin count → AppliedModelingLib.Probability.HalfOpenUnitCube dimension) =>
    ∑ index : AppliedModelingLib.Probability.dyadicCubeIndex dimension level,
      |AppliedModelingLib.Probability.finiteIIDCellFrequency count
        {x : AppliedModelingLib.Probability.HalfOpenUnitCube dimension |
          AppliedModelingLib.Probability.dyadicCubeLocate dimension level x = index}
        sample -
        (↑law).real
        {x : AppliedModelingLib.Probability.HalfOpenUnitCube dimension |
          AppliedModelingLib.Probability.dyadicCubeLocate dimension level x = index}]
```

Direct retained dependencies

- [AppliedModelingLib.Probability.HalfOpenUnitCube](#)
- [AppliedModelingLib.Probability.dyadicCellIndex](#)
- [AppliedModelingLib.Probability.dyadicCubeIndex](#)
- [AppliedModelingLib.Probability.dyadicCubeLocate](#)
- [AppliedModelingLib.Probability.finiteIIDCellFrequency](#)
- [AppliedModelingLib.Probability.halfOpenUnitInterval](#)

AppliedModelingLib.Probability.pOneFournierGuillinHeadGateRadius

Declaration location: reusable library

Lean declaration body

```
type:
  ℝ → ℕ → ℝ

value:
  fun (eta : ℝ) (gateScale : ℕ) => (2 ^ gateScale).rpow (-(2 * (1 + eta)))
```

AppliedModelingLib.Probability.pOneFournierGuillinCappedScaledDeviation

Declaration location: reusable library

Lean declaration body

```
type:
  ℝ → ℕ → ℝ → ℝ

value:
  fun (eta : ℝ) (gateScale : ℕ) (scaledDeviation : ℝ) =>
    min scaledDeviation (AppliedModelingLib.Probability.pOneFournierGuillinHeadGateRadius eta gateScale)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinHeadGateRadius](#)

AppliedModelingLib.Probability.pOneFournierGuillinCappedDeviation

Declaration location: reusable library

Lean declaration body

```
type:
  ℝ → ℕ → ℝ → ℝ

value:
  fun (eta : ℝ) (gateScale : ℕ) (scaledDeviation : ℝ) =>
    AppliedModelingLib.Probability.pOneFournierGuillinCappedScaledDeviation eta gateScale scaledDeviation /
    Real.rpow 2 (-(1 + eta))
```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinCappedScaledDeviation](#)

AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (eta :  $\mathbb{R}$ ) => 1 - Real.rpow 2 (-eta)
```

AppliedModelingLib.Probability.pOneFournierGuillinShellWeight

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{N} \rightarrow \mathbb{R}$ 

value:
fun (eta deviation :  $\mathbb{R}$ ) (shell :  $\mathbb{N}$ ) =>
  AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta * deviation *
  Real.rpow 2 (-(1 + eta) * ↑shell)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient](#)

AppliedModelingLib.Probability.pOneFournierGuillinShellToleranceWithEta

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{N} \rightarrow \mathbb{R}$ 

value:
fun (eta deviation shellMass :  $\mathbb{R}$ ) (shell :  $\mathbb{N}$ ) =>
  AppliedModelingLib.Probability.pOneFournierGuillinShellWeight eta deviation shell / shellMass
```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinShellWeight](#)

AppliedModelingLib.Probability.IsPOneFournierGuillinUniformAllShellMassGateScale

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{N} \rightarrow \text{Prop}$ 

value:
fun (dimension :  $\mathbb{N}$ ) (eta alpha gamma momentBound :  $\mathbb{R}$ ) (gateScale :  $\mathbb{N}$ ) =>
  ∀ (count :  $\mathbb{N}$ ),
  0 < count →
  ∀ (law : MeasureTheory.ProbabilityMeasure (EuclideanSpace  $\mathbb{R}$  (Fin dimension))),
  AppliedModelingLib.Probability.HasBoundedExponentialRadialMoment law alpha gamma momentBound →
  ∀ {deviation scaledDeviation :  $\mathbb{R}$ },
  scaledDeviation = deviation * Real.rpow 2 (-(1 + eta)) →
  0 < scaledDeviation →
  scaledDeviation ≤ 1 →
  scaledDeviation ≤ AppliedModelingLib.Probability.pOneFournierGuillinHeadGateRadius eta gateScale →
  1 ≤
    ↑count * AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta ^ 2 *
    scaledDeviation ^ 2 /
    (18 *
      AppliedModelingLib.Probability.fifthOrderExponentialRadialTailConstant law alpha gamma) →
  1 ≤
    ↑count * AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta *
    Real.rpow 2 (-(1 + eta)) *
    scaledDeviation ^ 2 *
    Real.log 9 /
    8 →
  1 ≤
    ↑count * AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta *
    scaledDeviation ^ 2 *
```

```

      gamma /
      4 *
      (Real.rpow 2 (alpha - 1 - eta) - 1) →
      (AppliedModelingLib.Probability.finiteIIDSampleLaw (↑ law) count).real
      (↑ (shell : ℕ),
      AppliedModelingLib.Probability.finiteIIDEuclideanDyadicCubeShellMassRelativeTailTotalized
      dimension law count shell
      (AppliedModelingLib.Probability.pOneFournierGuillinShellToleranceWithEta eta deviation
      ((↑ law).real
      (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell))
      shell)) ≤
2 *
Real.exp
  (-(↑ count *
    AppliedModelingLib.Probability.pOneFournierGuillinShellWeight eta deviation
    0) ^
    2 /
    (2 * ↑ count * (1 / 4))) +
(4 *
Real.exp
  (-(↑ count *
    AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta ^
    2 *
    scaledDeviation ^ 2 /
    (18 *
    AppliedModelingLib.Probability.fifthOrderExponentialRadialTailConstant law
    alpha gamma))) +
Real.exp
  (-(↑ count * AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta *
    Real.rpow 2 (-(1 + eta)) *
    scaledDeviation ^ 2 *
    Real.log 9 /
    4)) +
Real.exp
  (-(↑ count * AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta *
    scaledDeviation ^ 2 *
    gamma /
    4)) /
  (1 - Real.exp (-1)))

```

Direct retained dependencies

- [AppliedModelingLib.Probability.HasBoundedExponentialRadialMoment](#)
- [AppliedModelingLib.Probability.euclideanDyadicCubeShell](#)
- [AppliedModelingLib.Probability.fifthOrderExponentialRadialTailConstant](#)
- [AppliedModelingLib.Probability.finiteIIDEuclideanDyadicCubeShellMassRelativeTailTotalized](#)
- [AppliedModelingLib.Probability.finiteIIDSampleLaw](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinHeadGateRadius](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinShellToleranceWithEta](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinShellWeight](#)

AppliedModelingLib.Probability.pOneFournierGuillinUniformAllShellMassEffectiveCountRequirement

Declaration location: reusable library

Lean declaration body

```

type:
ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ
value:
fun (eta deviation scaledDeviation gamma tailBound tolerance : ℝ) =>
  max
    (Real.log (8 / tolerance) / (2 * AppliedModelingLib.Probability.pOneFournierGuillinShellWeight eta deviation 0 ^ 2))
    (max
      (Real.log (16 / tolerance) /
      (AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta ^ 2 * scaledDeviation ^ 2 /
      (18 * tailBound)))
      (max
        (Real.log (4 / tolerance) /
        (AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta * Real.rpow 2 (-(1 + eta)) *
        scaledDeviation ^ 2 *
        Real.log 9 /
        4))
        (Real.log (4 / (tolerance * (1 - Real.exp (-1)))) /
        (AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta * scaledDeviation ^ 2 * gamma / 4)))
    )

```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinShellWeight](#)

AppliedModelingLib.Probability.pOneFournierGuillinUniformAllShellMassGateScale

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{N}$ 

value:
fun (dimension :  $\mathbb{N}$ ) (eta alpha gamma momentBound :  $\mathbb{R}$ ) =>
  Classical.epsilon
  (AppliedModelingLib.Probability.IsPOneFournierGuillinUniformAllShellMassGateScale dimension eta alpha gamma
   momentBound)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.IsPOneFournierGuillinUniformAllShellMassGateScale](#)

AppliedModelingLib.Probability.pOneFournierGuillinUniformCappedDeviation

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (dimension :  $\mathbb{N}$ ) (eta alpha gamma momentBound scaledDeviation :  $\mathbb{R}$ ) =>
  AppliedModelingLib.Probability.pOneFournierGuillinCappedDeviation eta
  (AppliedModelingLib.Probability.pOneFournierGuillinUniformAllShellMassGateScale dimension eta alpha gamma
   momentBound)
  scaledDeviation
```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinCappedDeviation](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinUniformAllShellMassGateScale](#)

AppliedModelingLib.Probability.pOneFournierGuillinUniformCappedScaledDeviation

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (dimension :  $\mathbb{N}$ ) (eta alpha gamma momentBound scaledDeviation :  $\mathbb{R}$ ) =>
  AppliedModelingLib.Probability.pOneFournierGuillinCappedScaledDeviation eta
  (AppliedModelingLib.Probability.pOneFournierGuillinUniformAllShellMassGateScale dimension eta alpha gamma
   momentBound)
  scaledDeviation
```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinCappedScaledDeviation](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinUniformAllShellMassGateScale](#)

AppliedModelingLib.Probability.pOneFournierGuillinUniformSquareRateEffectiveCountRequirement

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (eta alpha gamma tailBound scaledDeviation :  $\mathbb{R}$ ) =>
  max
  (1 /
   (AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta ^ 2 * scaledDeviation ^ 2 /
    (18 * tailBound)))
```

```
(max
  (1 /
    (AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta * Real.rpow 2 (-(1 + eta)) *
      scaledDeviation ^ 2 *
      Real.log 9 /
      8))
  (1 /
    (AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient eta * scaledDeviation ^ 2 * gamma / 4 *
      (Real.rpow 2 (alpha - 1 - eta) - 1))))
```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinShellCoefficient](#)

AppliedModelingLib.Probability.selectedShellDyadicGeometricConfidenceThreshold

Declaration location: reusable library

Lean declaration body

```
type:
  ℕ → ℕ → ℕ → ℝ → ℝ → ℝ

value:
  fun (dimension count level : ℕ) (shellMass confidence : ℝ) =>
    √(4 * (2 * ↑(2 ^ (dimension * level)) + confidence) / (↑count * shellMass))
```

AppliedModelingLib.Probability.selectedShellGeometricCentralSchedule_dimensionOne

Declaration location: reusable library

Lean declaration body

```
type:
  ℕ → ℝ → ENNReal

value:
  fun (count : ℕ) (confidence : ℝ) =>
    ENNReal.ofReal (128 / ↑count + (2 * (1 / (1 - 1 / √2)) + 2 * √(confidence + Real.log 2)) / √↑count) +
    ENNReal.ofReal (2 * √1 * ((16 * ↑(2 ^ 1) + 8 * (confidence + Real.log 2)) / ↑count))
```

AppliedModelingLib.Probability.selectedShellGeometricCentralSchedule_dimensionTwo

Declaration location: reusable library

Lean declaration body

```
type:
  ℕ → ℝ → ENNReal

value:
  fun (count : ℕ) (confidence : ℝ) =>
    ENNReal.ofReal
      ((32 * √2 + 4 * ↑(AppliedModelingLib.Math.dyadicEffectiveCountDepth 2 (↑count / 16)) +
        2 * √2 * √(confidence + Real.log 2)) /
        √↑count) +
    ENNReal.ofReal (2 * √2 * ((16 * ↑(2 ^ 2) + 8 * (confidence + Real.log 2)) / ↑count))
```

Direct retained dependencies

- [AppliedModelingLib.Math.dyadicEffectiveCountDepth](#)

AppliedModelingLib.Probability.selectedShellGeometricConfidence

Declaration location: reusable library

Lean declaration body

```
type:
  ℝ → ℕ → ℝ

value:
  fun (confidence : ℝ) (shell : ℕ) => confidence + (↑shell + 1) * Real.log 2
```

AppliedModelingLib.Probability.selectedShellGeometricDimensionOneCentralCountCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (confidence :  $\mathbb{R}$ ) => 2 *  $\sqrt{1}$  * (16 *  $\uparrow(2^1)$  + 8 * (confidence + Real.log 2))
```

AppliedModelingLib.Probability.selectedShellGeometricDimensionOneCentralRootCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (confidence :  $\mathbb{R}$ ) => 2 * (1 / (1 - 1 /  $\sqrt{2}$ )) + 2 *  $\sqrt{(\text{confidence} + \text{Real.log } 2)}$ 
```

AppliedModelingLib.Probability.selectedShellGeometricDimensionOneSuccessorCountCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R}$ 

value:
fun (cutoff :  $\mathbb{N}$ ) => 128 * (2 * (2 ^ cutoff - 1))
```

AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoCentralCountCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (confidence :  $\mathbb{R}$ ) => 2 *  $\sqrt{2}$  * (16 *  $\uparrow(2^2)$  + 8 * (confidence + Real.log 2))
```

AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoCentralDepthQuarterCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R}$ 

value:
16 / Real.log 4
```

AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoCentralRootCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (confidence :  $\mathbb{R}$ ) => 32 *  $\sqrt{2}$  + 2 *  $\sqrt{2}$  *  $\sqrt{(\text{confidence} + \text{Real.log } 2)}$ 
```

AppliedModelingLib.Probability.selectedShellGeometricEffectiveDepth

Declaration location: reusable library

Lean declaration body

```
type:
(dimension : ℕ) → ℕ → MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension)) → ℝ → ℕ → ℕ

value:
fun (dimension count : ℕ) (law : MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension))) (confidence : ℝ)
  (shell : ℕ) =>
  AppliedModelingLib.Math.dyadicGeometricEffectiveDepth dimension
    (↑count * (↑law).real (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell))
    (AppliedModelingLib.Probability.selectedShellGeometricConfidence confidence shell)
```

Direct retained dependencies

- [AppliedModelingLib.Math.dyadicGeometricEffectiveDepth](#)
- [AppliedModelingLib.Probability.euclideanDyadicCubeShell](#)
- [AppliedModelingLib.Probability.selectedShellGeometricConfidence](#)

AppliedModelingLib.Probability.selectedShellGeometricEffectiveHeadDepth

Declaration location: reusable library

Lean declaration body

```
type:
(dimension : ℕ) → ℕ → ℕ → MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension)) → ℝ → ℕ → ℕ

value:
fun (dimension cutoff count : ℕ) (law : MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension)))
  (confidence : ℝ) (shell : ℕ) =>
  if shell ≤ cutoff then
    AppliedModelingLib.Probability.selectedShellGeometricEffectiveDepth dimension count law confidence shell
  else 0
```

Direct retained dependencies

- [AppliedModelingLib.Probability.selectedShellGeometricEffectiveDepth](#)

AppliedModelingLib.Probability.selectedShellGeometricHeadSchedule_dimensionGreaterTwo

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → ℕ → ℕ → ℝ → ℝ → ENNReal

value:
fun (dimension cutoff count : ℕ) (confidence tailConstant : ℝ) =>
  ENNReal.ofReal
    ((2 * √↑dimension + √↑dimension * √2 / (4 * (√↑(2 ^ dimension) / 2 - 1))) *
      ((16 * ↑(2 ^ dimension) ^ 2) ^ (↑dimension)-1 / ↑count ^ (↑dimension)-1) +
      2 * √↑dimension * √(confidence + Real.log 2) / √↑count) +
  ENNReal.ofReal
    ((2 * √↑dimension * ((16 * ↑(2 ^ dimension) + 8 * (confidence + Real.log 2)) / ↑count)) +
      ((2 * √↑dimension + √↑dimension * √2 / (4 * (√↑(2 ^ dimension) / 2 - 1))) *
        (16 * ↑(2 ^ dimension) ^ 2) ^ (↑dimension)-1 /
        ↑count ^ (↑dimension)-1 +
        2 * √↑dimension * √(confidence + (↑cutoff + 1) * Real.log 2) / √↑count) *
        (4 * √tailConstant)) +
  ENNReal.ofReal
    (2 * √↑dimension * √((16 * ↑(2 ^ dimension) + 8 * (confidence + (↑cutoff + 1) * Real.log 2)) / ↑count) *
      (4 * √tailConstant)))
```

AppliedModelingLib.Probability.selectedShellGeometricHeadSchedule_dimensionOne

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → ℕ → ℝ → ℝ → ENNReal

value:
fun (cutoff count : ℕ) (confidence tailConstant : ℝ) =>
  AppliedModelingLib.Probability.selectedShellGeometricCentralSchedule_dimensionOne count confidence +
  ENNReal.ofReal
    ((128 / ↑count * (2 * (2 ^ cutoff - 1)) +
      (2 * (1 / (1 - 1 / √2)) + 2 * √(confidence + (↑cutoff + 1) * Real.log 2)) / √↑count * (4 * √tailConstant)) +
  ENNReal.ofReal
    (2 * √((16 * ↑(2 ^ 1) + 8 * (confidence + (↑cutoff + 1) * Real.log 2)) / ↑count) * (4 * √tailConstant)))
```

Direct retained dependencies

- [AppliedModelingLib.Probability.selectedShellGeometricCentralSchedule_dimensionOne](#)

AppliedModelingLib.Probability.selectedShellGeometricHeadSchedule_dimensionTwo

Declaration location: reusable library

Lean declaration body

```
type:
  ℕ → ℕ → ℝ → ℝ → ENNReal

value:
  fun (cutoff count : ℕ) (confidence tailConstant : ℝ) =>
    AppliedModelingLib.Probability.selectedShellGeometricCentralSchedule_dimensionTwo count confidence +
      (ENNReal.ofReal
        ((32 * √2 + 4 * ↑(AppliedModelingLib.Math.dyadicEffectiveCountDepth 2 (↑count / 16)) +
          2 * √2 * √(confidence + (↑cutoff + 1) * Real.log 2)) /
          √↑count *
          (4 * √tailConstant)) +
        ENNReal.ofReal
          (2 * √2 * √((16 * ↑(2 ^ 2) + 8 * (confidence + (↑cutoff + 1) * Real.log 2)) / ↑count) * (4 * √tailConstant)))
```

Direct retained dependencies

- [AppliedModelingLib.Math.dyadicEffectiveCountDepth](#)
- [AppliedModelingLib.Probability.selectedShellGeometricCentralSchedule_dimensionTwo](#)

AppliedModelingLib.Probability.selectedShellGeometricHeadSchedule_byDimension

Declaration location: reusable library

Lean declaration body

```
type:
  ℕ → ℕ → ℕ → ℝ → ℝ → ENNReal

value:
  fun (dimension cutoff count : ℕ) (confidence tailConstant : ℝ) =>
    if hdimensionOne : dimension = 1 then
      AppliedModelingLib.Probability.selectedShellGeometricHeadSchedule_dimensionOne cutoff count confidence tailConstant
    else
      if hdimensionTwo : dimension = 2 then
        AppliedModelingLib.Probability.selectedShellGeometricHeadSchedule_dimensionTwo cutoff count confidence
        tailConstant
      else
        AppliedModelingLib.Probability.selectedShellGeometricHeadSchedule_dimensionGreaterTwo dimension cutoff count
        confidence tailConstant
```

Direct retained dependencies

- [AppliedModelingLib.Probability.selectedShellGeometricHeadSchedule_dimensionGreaterTwo](#)
- [AppliedModelingLib.Probability.selectedShellGeometricHeadSchedule_dimensionOne](#)
- [AppliedModelingLib.Probability.selectedShellGeometricHeadSchedule_dimensionTwo](#)

AppliedModelingLib.Probability.selectedShellGeometricSupercriticalRateCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
  ℕ → ℝ

value:
  fun (dimension : ℕ) =>
    (2 * √↑dimension + √↑dimension * √2 / (4 * (√↑(2 ^ dimension) / 2 - 1))) *
    (16 * ↑(2 ^ dimension) ^ 2) ^ (↑dimension)-1
```

AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (tailConstant :  $\mathbb{R}$ ) => 4 *  $\sqrt{\text{tailConstant}}$ 
```

AppliedModelingLib.Probability.selectedShellGeometricDimensionOneSuccessorRootCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (cutoff :  $\mathbb{N}$ ) (confidence tailConstant :  $\mathbb{R}$ ) =>
  (2 * (1 / (1 - 1 /  $\sqrt{2}$ )) + 2 *  $\sqrt{(\text{confidence} + (\uparrow \text{cutoff} + 1) * \text{Real.log } 2)}) *
    \text{AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier tailConstant}$ 
```

Direct retained dependencies

- [AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier](#)

AppliedModelingLib.Probability.selectedShellGeometricDimensionOneSuccessorSqrtCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (cutoff :  $\mathbb{N}$ ) (confidence tailConstant :  $\mathbb{R}$ ) =>
  2 *  $\sqrt{(16 * \uparrow(2^1) + 8 * (\text{confidence} + (\uparrow \text{cutoff} + 1) * \text{Real.log } 2))} *
    \text{AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier tailConstant}$ 
```

Direct retained dependencies

- [AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier](#)

AppliedModelingLib.Probability.selectedShellGeometricDimensionOneEffectiveCountRequirement

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (cutoff :  $\mathbb{N}$ ) (confidence tailConstant radius :  $\mathbb{R}$ ) =>
  max (6 * 128 / radius)
    (max
      (6 ^ 2 * AppliedModelingLib.Probability.selectedShellGeometricDimensionOneCentralRootCoefficient confidence ^ 2 /
        radius ^ 2)
      (max
        (6 * AppliedModelingLib.Probability.selectedShellGeometricDimensionOneCentralCountCoefficient confidence /
          radius)
        (max
          (6 * AppliedModelingLib.Probability.selectedShellGeometricDimensionOneSuccessorCountCoefficient cutoff /
            radius)
          (max
            (6 ^ 2 *
              AppliedModelingLib.Probability.selectedShellGeometricDimensionOneSuccessorRootCoefficient cutoff
              confidence tailConstant ^
                2 /
              radius ^ 2)
            (6 ^ 2 *
              AppliedModelingLib.Probability.selectedShellGeometricDimensionOneSuccessorSqrtCoefficient cutoff
              confidence tailConstant ^
                2 /
              radius ^ 2))))))
```

Direct retained dependencies

- [AppliedModelingLib.Probability.selectedShellGeometricDimensionOneCentralCountCoefficient](#)
- [AppliedModelingLib.Probability.selectedShellGeometricDimensionOneCentralRootCoefficient](#)
- [AppliedModelingLib.Probability.selectedShellGeometricDimensionOneSuccessorCountCoefficient](#)
- [AppliedModelingLib.Probability.selectedShellGeometricDimensionOneSuccessorRootCoefficient](#)
- [AppliedModelingLib.Probability.selectedShellGeometricDimensionOneSuccessorSqrtCoefficient](#)

AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoSuccessorDepthQuarterCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → ℝ

value:
fun (tailConstant : ℝ) =>
  16 / Real.log 4 * AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier tailConstant
```

Direct retained dependencies

- [AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier](#)

AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoSuccessorRootCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → ℝ → ℝ → ℝ

value:
fun (cutoff : ℕ) (confidence tailConstant : ℝ) =>
  (32 * √2 + 2 * √2 * √(confidence + (↑ cutoff + 1) * Real.log 2)) *
    AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier tailConstant
```

Direct retained dependencies

- [AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier](#)

AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoSuccessorSqrtCoefficient

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → ℝ → ℝ → ℝ

value:
fun (cutoff : ℕ) (confidence tailConstant : ℝ) =>
  2 * √2 * √(16 * ↑(2 ^ 2) + 8 * (confidence + (↑ cutoff + 1) * Real.log 2)) *
    AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier tailConstant
```

Direct retained dependencies

- [AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier](#)

AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoEffectiveCountRequirement

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (cutoff :  $\mathbb{N}$ ) (confidence tailConstant radius :  $\mathbb{R}$ ) =>
  max 64
    (max
      (6 ^ 2 * AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoCentralRootCoefficient confidence ^ 2 /
        radius ^ 2)
      (max
        (6 ^ 4 * AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoCentralDepthQuarterCoefficient ^ 4 /
          radius ^ 4)
        (max
          (6 * AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoCentralCountCoefficient confidence /
            radius)
          (max
            (6 ^ 2 *
              AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoSuccessorRootCoefficient cutoff
              confidence tailConstant ^
                2 /
                radius ^ 2)
            (max
              (6 ^ 4 *
                AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoSuccessorDepthQuarterCoefficient
                tailConstant ^
                  4 /
                  radius ^ 4)
              (6 ^ 2 *
                AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoSuccessorSqrtCoefficient cutoff
                confidence tailConstant ^
                  2 /
                  radius ^ 2))))))
```

Direct retained dependencies

- [AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoCentralCountCoefficient](#)
- [AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoCentralDepthQuarterCoefficient](#)
- [AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoCentralRootCoefficient](#)
- [AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoSuccessorDepthQuarterCoefficient](#)
- [AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoSuccessorRootCoefficient](#)
- [AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoSuccessorSqrtCoefficient](#)

AppliedModelingLib.Probability.selectedShellGeometricSupercriticalEffectiveCountRequirement

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (dimension cutoff :  $\mathbb{N}$ ) (confidence tailConstant radius :  $\mathbb{R}$ ) =>
  max
    (AppliedModelingLib.Probability.selectedShellGeometricSupercriticalRateCoefficient dimension ^ dimension /
      (radius / 6) ^ dimension)
    (max (6 ^ 2 * (2 *  $\sqrt{\uparrow \text{dimension}} * \sqrt{(\text{confidence} + \text{Real.log } 2)}) ^ 2 / \text{radius} ^ 2)$ 
      (max (6 * (2 *  $\sqrt{\uparrow \text{dimension}} * (16 * \uparrow (2 ^ \text{dimension}) + 8 * (\text{confidence} + \text{Real.log } 2))) / \text{radius}$ 
        (max
          ((AppliedModelingLib.Probability.selectedShellGeometricSupercriticalRateCoefficient dimension *
            AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier tailConstant) ^
            dimension /
            (radius / 6) ^ dimension)
          (max
            (6 ^ 2 * (2 *  $\sqrt{\uparrow \text{dimension}} * \sqrt{(\text{confidence} + (\uparrow \text{cutoff} + 1) * \text{Real.log } 2)} * (4 * \sqrt{\text{tailConstant}})) ^ 2 /$ 
              radius ^ 2)
            (6 ^ 2 *
              (2 *  $\sqrt{\uparrow \text{dimension}} * \sqrt{(16 * \uparrow (2 ^ \text{dimension}) + 8 * (\text{confidence} + (\uparrow \text{cutoff} + 1) * \text{Real.log } 2)) *$ 
                (4 *  $\sqrt{\text{tailConstant}})) ^$ 
                2 /
                radius ^ 2))))))
```

Direct retained dependencies

- [AppliedModelingLib.Probability.selectedShellGeometricSupercriticalRateCoefficient](#)
- [AppliedModelingLib.Probability.selectedShellGeometricTailMultiplier](#)

AppliedModelingLib.Probability.pOneFournierGuillinSupercriticalAllShellEffectiveCountRequirement

Declaration location: reusable library

Lean declaration body

```
type:
  ℕ → ℕ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ

value:
  fun (dimension cutoff : ℕ) (eta alpha gamma tailBound deviation scaledDeviation confidence radius tolerance : ℝ) =>
    max
      (AppliedModelingLib.Probability.selectedShellGeometricSupercriticalEffectiveCountRequirement dimension cutoff
        confidence tailBound radius)
      (max
        (AppliedModelingLib.Probability.pOneFournierGuillinUniformAllShellMassEffectiveCountRequirement eta deviation
          scaledDeviation gamma tailBound tolerance)
        (AppliedModelingLib.Probability.pOneFournierGuillinUniformSquareRateEffectiveCountRequirement eta alpha gamma
          tailBound scaledDeviation))
```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinUniformAllShellMassEffectiveCountRequirement](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinUniformSquareRateEffectiveCountRequirement](#)
- [AppliedModelingLib.Probability.selectedShellGeometricSupercriticalEffectiveCountRequirement](#)

AppliedModelingLib.Probability.pOneFournierGuillinSupercriticalAllShellEffectiveCount

Declaration location: reusable library

Lean declaration body

```
type:
  ℕ → ℕ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℕ

value:
  fun (dimension cutoff : ℕ) (eta alpha gamma tailBound deviation scaledDeviation confidence radius tolerance : ℝ) =>
    AppliedModelingLib.Math.positiveNatCeil
      (AppliedModelingLib.Probability.pOneFournierGuillinSupercriticalAllShellEffectiveCountRequirement dimension cutoff
        eta alpha gamma tailBound deviation scaledDeviation confidence radius tolerance)
```

Direct retained dependencies

- [AppliedModelingLib.Math.positiveNatCeil](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinSupercriticalAllShellEffectiveCountRequirement](#)

AppliedModelingLib.Probability.pOneFournierGuillinSupercriticalCappedAllShellConcreteCountSchedule

Declaration location: reusable library

Lean declaration body

```
type:
  ℕ → ℕ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → (ℕ → ℝ) → (ℕ → ℝ) → ℕ → ℕ

value:
  fun (dimension cutoff : ℕ) (eta alpha gamma momentBound tailBound scaledDeviation : ℝ)
    (failureBudget radiusSchedule : ℕ → ℝ) (a : ℕ) =>
    AppliedModelingLib.Probability.pOneFournierGuillinSupercriticalAllShellEffectiveCount dimension cutoff eta alpha gamma
      tailBound
      (AppliedModelingLib.Probability.pOneFournierGuillinUniformCappedDeviation dimension eta alpha gamma momentBound
        scaledDeviation)
      (AppliedModelingLib.Probability.pOneFournierGuillinUniformCappedScaledDeviation dimension eta alpha gamma
        momentBound scaledDeviation)
      (AppliedModelingLib.Math.expConfidenceForHalfBudget (failureBudget a)) (radiusSchedule a) (failureBudget a / 2)
```

Direct retained dependencies

- [AppliedModelingLib.Math.expConfidenceForHalfBudget](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinSupercriticalAllShellEffectiveCount](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinUniformCappedDeviation](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinUniformCappedScaledDeviation](#)

AppliedModelingLib.Probability.selectedShellGeometricHeadEffectiveCountRequirement_byDimension

Declaration location: reusable library

Lean declaration body

```
type:
  ℕ → ℕ → ℝ → ℝ → ℝ → ℝ

value:
  fun (dimension cutoff : ℕ) (confidence tailConstant radius : ℝ) =>
    if hdimensionOne : dimension = 1 then
      AppliedModelingLib.Probability.selectedShellGeometricDimensionOneEffectiveCountRequirement cutoff confidence
      tailConstant radius
    else
      if hdimensionTwo : dimension = 2 then
        AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoEffectiveCountRequirement cutoff confidence
        tailConstant radius
      else
        AppliedModelingLib.Probability.selectedShellGeometricSupercriticalEffectiveCountRequirement dimension cutoff
        confidence tailConstant radius
```

Direct retained dependencies

- [AppliedModelingLib.Probability.selectedShellGeometricDimensionOneEffectiveCountRequirement](#)
- [AppliedModelingLib.Probability.selectedShellGeometricDimensionTwoEffectiveCountRequirement](#)
- [AppliedModelingLib.Probability.selectedShellGeometricSupercriticalEffectiveCountRequirement](#)

AppliedModelingLib.Probability.pOneFournierGuillinAllDimensionalAllShellEffectiveCountRequirement

Declaration location: reusable library

Lean declaration body

```
type:
  ℕ → ℕ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ

value:
  fun (dimension cutoff : ℕ) (eta alpha gamma tailBound deviation scaledDeviation confidence radius tolerance : ℝ) =>
    max
      (AppliedModelingLib.Probability.selectedShellGeometricHeadEffectiveCountRequirement_byDimension dimension cutoff
       confidence tailBound radius)
    (max
      (AppliedModelingLib.Probability.pOneFournierGuillinUniformAllShellMassEffectiveCountRequirement eta deviation
       scaledDeviation gamma tailBound tolerance)
      (AppliedModelingLib.Probability.pOneFournierGuillinUniformSquareRateEffectiveCountRequirement eta alpha gamma
       tailBound scaledDeviation))
```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinUniformAllShellMassEffectiveCountRequirement](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinUniformSquareRateEffectiveCountRequirement](#)
- [AppliedModelingLib.Probability.selectedShellGeometricHeadEffectiveCountRequirement_byDimension](#)

AppliedModelingLib.Probability.pOneFournierGuillinAllDimensionalAllShellEffectiveCount

Declaration location: reusable library

Lean declaration body

```
type:
  ℕ → ℕ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → ℕ

value:
  fun (dimension cutoff : ℕ) (eta alpha gamma tailBound deviation scaledDeviation confidence radius tolerance : ℝ) =>
    AppliedModelingLib.Math.positiveNatCeil
    (AppliedModelingLib.Probability.pOneFournierGuillinAllDimensionalAllShellEffectiveCountRequirement dimension cutoff
     eta alpha gamma tailBound deviation scaledDeviation confidence radius tolerance)
```

Direct retained dependencies

- [AppliedModelingLib.Math.positiveNatCeil](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinAllDimensionalAllShellEffectiveCountRequirement](#)

AppliedModelingLib.Probability.pOneFournierGuillinAllDimensionalAllShellConcreteCountSchedule

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → ℕ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ → (ℕ → ℝ) → (ℕ → ℝ) → ℕ → ℕ

value:
fun (dimension cutoff : ℕ) (eta alpha gamma tailBound deviation scaledDeviation : ℝ)
  (failureBudget radiusSchedule : ℕ → ℝ) (a : ℕ) =>
  AppliedModelingLib.Probability.pOneFournierGuillinAllDimensionalAllShellEffectiveCount dimension cutoff eta alpha
  gamma tailBound deviation scaledDeviation (AppliedModelingLib.Math.expConfidenceForHalfBudget (failureBudget a))
  (radiusSchedule a) (failureBudget a / 2)
```

Direct retained dependencies

- [AppliedModelingLib.Math.expConfidenceForHalfBudget](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinAllDimensionalAllShellEffectiveCount](#)

AppliedModelingLib.ProbabilityCoupling

Declaration location: reusable library

Lean declaration body

```
field joint:
{α : Type u_1} →
{β : Type u_2} →
[inst : MeasurableSpace α] →
[inst_1 : MeasurableSpace β] →
{μ : MeasureTheory.ProbabilityMeasure α} →
{ν : MeasureTheory.ProbabilityMeasure β} →
AppliedModelingLib.ProbabilityCoupling μ ν → MeasureTheory.ProbabilityMeasure (α × β)

field map_fst:
∀ {α : Type u_1} {β : Type u_2} [inst : MeasurableSpace α] [inst_1 : MeasurableSpace β]
{μ : MeasureTheory.ProbabilityMeasure α} {ν : MeasureTheory.ProbabilityMeasure β}
(self : AppliedModelingLib.ProbabilityCoupling μ ν), self.joint.map (Measurable.aemeasurable measurable_fst) = μ

field map_snd:
∀ {α : Type u_1} {β : Type u_2} [inst : MeasurableSpace α] [inst_1 : MeasurableSpace β]
{μ : MeasureTheory.ProbabilityMeasure α} {ν : MeasureTheory.ProbabilityMeasure β}
(self : AppliedModelingLib.ProbabilityCoupling μ ν), self.joint.map (Measurable.aemeasurable measurable_snd) = ν
```

AppliedModelingLib.ProbabilityCoupling.cost

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} →
{β : Type u_2} →
[inst : MeasurableSpace α] →
[inst_1 : MeasurableSpace β] →
{μ : MeasureTheory.ProbabilityMeasure α} →
{ν : MeasureTheory.ProbabilityMeasure β} →
AppliedModelingLib.ProbabilityCoupling μ ν → (α × β → ENNReal) → ENNReal

value:
fun {α : Type u_1} {β : Type u_2} [MeasurableSpace α] [MeasurableSpace β] {μ : MeasureTheory.ProbabilityMeasure α}
{ν : MeasureTheory.ProbabilityMeasure β} (π : AppliedModelingLib.ProbabilityCoupling μ ν) (c : α × β → ENNReal) =>
  ⌊ (z : α × β), c z ∂ ↑ π.joint
```

Direct retained dependencies

- [AppliedModelingLib.ProbabilityCoupling](#)

AppliedModelingLib.ProbabilityCoupling.expectedDistanceCosts

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_3} →
[inst : MeasurableSpace α] →
[PseudoMetricSpace α] → MeasureTheory.ProbabilityMeasure α → MeasureTheory.ProbabilityMeasure α → ℝ → Prop

value:
fun {α : Type u_3} [MeasurableSpace α] [PseudoMetricSpace α] (μ ν : MeasureTheory.ProbabilityMeasure α) (a : ℝ) =>
  {r : ℝ |
    ∃ (π : AppliedModelingLib.ProbabilityCoupling μ ν) (⌊ :
      MeasureTheory.Integrable (fun (z : α × α) => dist z.1 z.2) ↑ π.joint), r = ∫ (z : α × α), dist z.1 z.2 ∂ ↑ π.joint}
  a
```

Direct retained dependencies

- [AppliedModelingLib.ProbabilityCoupling](#)

AppliedModelingLib.ProbabilityCoupling.transportCost

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} →
{β : Type u_2} →
  [inst : MeasurableSpace α] →
  [inst_1 : MeasurableSpace β] →
  (α × β → ENNReal) → MeasureTheory.ProbabilityMeasure α → MeasureTheory.ProbabilityMeasure β → ENNReal

value:
fun {α : Type u_1} {β : Type u_2} [MeasurableSpace α] [MeasurableSpace β] (c : α × β → ENNReal)
  (μ : MeasureTheory.ProbabilityMeasure α) (ν : MeasureTheory.ProbabilityMeasure β) =>
  sInf (Set.range fun (π : AppliedModelingLib.ProbabilityCoupling μ ν) => π.cost c)
```

Direct retained dependencies

- [AppliedModelingLib.ProbabilityCoupling](#)
- [AppliedModelingLib.ProbabilityCoupling.cost](#)

AppliedModelingLib.ProbabilityCoupling.wassersteinOne

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_3} →
  [inst : MeasurableSpace α] →
  [PseudoMetricSpace α] → MeasureTheory.ProbabilityMeasure α → MeasureTheory.ProbabilityMeasure α → ENNReal

value:
fun {α : Type u_3} [MeasurableSpace α] [PseudoMetricSpace α] (μ ν : MeasureTheory.ProbabilityMeasure α) =>
  AppliedModelingLib.ProbabilityCoupling.transportCost (fun (z : α × α) => edist z.1 z.2) μ ν
```

Direct retained dependencies

- [AppliedModelingLib.ProbabilityCoupling.transportCost](#)

AppliedModelingLib.ProbabilityCoupling.wassersteinOneReal

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_3} →
  [inst : MeasurableSpace α] →
  [inst_1 : PseudoMetricSpace α] →
  (μ ν : MeasureTheory.ProbabilityMeasure α) →
  (AppliedModelingLib.ProbabilityCoupling.expectedDistanceCosts μ ν).Nonempty → ℝ

value:
fun {α : Type u_3} [MeasurableSpace α] [PseudoMetricSpace α] (μ ν : MeasureTheory.ProbabilityMeasure α)
  (x : (AppliedModelingLib.ProbabilityCoupling.expectedDistanceCosts μ ν).Nonempty) =>
  sInf (AppliedModelingLib.ProbabilityCoupling.expectedDistanceCosts μ ν)
```

Direct retained dependencies

- [AppliedModelingLib.ProbabilityCoupling.expectedDistanceCosts](#)

AppliedModelingLib.IsMeasureWassersteinSensitiveOn

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[MetricSpace Parameter] →
[inst : MeasurableSpace Data] →
[MetricSpace Data] → AppliedModelingLib.MeasurePerformativeModel Parameter Data → Set Parameter → ℝ → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MetricSpace Parameter] [MeasurableSpace Data] [MetricSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (domain : Set Parameter) (sensitivity : ℝ) =>
  ∀ first ∈ domain,
  ∀ second ∈ domain,
  ∃ (hfinite :
    (AppliedModelingLib.ProbabilityCoupling.expectedDistanceCosts (model.dataLaw first)
      (model.dataLaw second)).Nonempty),
    AppliedModelingLib.ProbabilityCoupling.wassersteinOneReal (model.dataLaw first) (model.dataLaw second) hfinite ≤
      sensitivity * dist first second
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.ProbabilityCoupling.expectedDistanceCosts](#)
- [AppliedModelingLib.ProbabilityCoupling.wassersteinOneReal](#)

AppliedModelingLib.empiricalFintypeIndicesInSet

Declaration location: reusable library

Lean declaration body

```
type:
{Index : Type u_1} → {Sample : Type u_2} → [Fintype Index] → (Index → Sample) → Set Sample → Finset Index

value:
fun {Index : Type u_1} {Sample : Type u_2} [Fintype Index] (sample : Index → Sample) (s : Set Sample) =>
  {index : Index | sample index ∈ s}
```

AppliedModelingLib.empiricalSampleIndicesInSet

Declaration location: reusable library

Lean declaration body

```
type:
{Sample : Type u_1} → {count : ℕ} → (Fin count → Sample) → Set Sample → Finset (Fin count)

value:
fun {Sample : Type u_1} {count : ℕ} (sample : Fin count → Sample) (s : Set Sample) =>
  AppliedModelingLib.empiricalFintypeIndicesInSet sample s
```

Direct retained dependencies

- [AppliedModelingLib.empiricalFintypeIndicesInSet](#)

AppliedModelingLib.Probability.finiteIIDRealizedCompactShellDyadicPartitionTail

Declaration location: reusable library

Lean declaration body

```
type:
(dimension : ℕ) →
  ℕ →
    MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension)) →
      (count : ℕ) → ℕ → ℝ → (Fin count → EuclideanSpace ℝ (Fin dimension)) → Prop

value:
fun (dimension shell : ℕ) (law : MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension))) (count level : ℕ)
(error : ℝ) (a : Fin count → EuclideanSpace ℝ (Fin dimension)) =>
  {sample : Fin count → EuclideanSpace ℝ (Fin dimension) |
    ∃ ( _ :
      (AppliedModelingLib.empiricalSampleIndicesInSet sample
        (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell)).Nonempty),
      error ≤
        AppliedModelingLib.Probability.dyadicPartitionL1Deviation dimension
          (AppliedModelingLib.Probability.euclideanDyadicCubeShellCompactLaw dimension shell law)
            (AppliedModelingLib.empiricalSampleIndicesInSet sample
              (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell)).card
```

```

level
(AppliedModelingLib.Probability.finiteIIDSampleMap
(AppliedModelingLib.Probability.euclideanDyadicCubeShellScaleExtension dimension shell)
(AppliedModelingLib.Probability.finiteIIDSampleOnFinsetToFin
(AppliedModelingLib.empiricalSampleIndicesInSet sample
(AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell))
sample)))}
a

```

Direct retained dependencies

- [AppliedModelingLib.Probability.HalfOpenUnitCube](#)
- [AppliedModelingLib.Probability.dyadicPartitionL1Deviation](#)
- [AppliedModelingLib.Probability.euclideanDyadicCubeShell](#)
- [AppliedModelingLib.Probability.euclideanDyadicCubeShellCompactLaw](#)
- [AppliedModelingLib.Probability.euclideanDyadicCubeShellScaleExtension](#)
- [AppliedModelingLib.Probability.finiteIIDSampleMap](#)
- [AppliedModelingLib.Probability.finiteIIDSampleOnFinsetToFin](#)
- [AppliedModelingLib.empiricalSampleIndicesInSet](#)

AppliedModelingLib.Probability.pOneFournierGuillinSelectedShellPartitionCertificate

Declaration location: reusable library

Lean declaration body

```

type:
(dimension : ℕ) →
MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension)) →
(count : ℕ) → ℝ → ℝ → (ℕ → ℕ) → ℝ → (Fin count → EuclideanSpace ℝ (Fin dimension)) → Prop

value:
fun (dimension : ℕ) (law : MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension))) (count : ℕ)
(eta deviation : ℝ) (depth : ℕ → ℕ) (confidence : ℝ) (a : Fin count → EuclideanSpace ℝ (Fin dimension)) =>
((⋃ (shell : ℕ),
  AppliedModelingLib.Probability.finiteIIDEuclideanDyadicCubeShellMassRelativeTailTotalized dimension law count
  shell
  (AppliedModelingLib.Probability.pOneFournierGuillinShellToleranceWithEta eta deviation
  ((↑ law).real (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell)) shell)) u
⋃ (shell : ℕ),
  ⋃ level ∈ Finset.range (depth shell),
  AppliedModelingLib.Probability.finiteIIDRealizedCompactShellDyadicPartitionTail dimension shell law count
  (level + 1)
  (AppliedModelingLib.Probability.selectedShellDyadicGeometricConfidenceThreshold dimension count (level + 1)
  ((↑ law).real (AppliedModelingLib.Probability.euclideanDyadicCubeShell dimension shell))
  (AppliedModelingLib.Probability.selectedShellGeometricConfidence confidence shell)))
a

```

Direct retained dependencies

- [AppliedModelingLib.Probability.euclideanDyadicCubeShell](#)
- [AppliedModelingLib.Probability.finiteIIDEuclideanDyadicCubeShellMassRelativeTailTotalized](#)
- [AppliedModelingLib.Probability.finiteIIDRealizedCompactShellDyadicPartitionTail](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinShellToleranceWithEta](#)
- [AppliedModelingLib.Probability.selectedShellDyadicGeometricConfidenceThreshold](#)
- [AppliedModelingLib.Probability.selectedShellGeometricConfidence](#)

AppliedModelingLib.empiricalSampleLaw

Declaration location: reusable library

Lean declaration body

```

type:
{Sample : Type u_1} → {count : ℕ} → [Nonempty (Fin count)] → (Fin count → Sample) → PMF Sample

value:
fun {Sample : Type u_1} {count : ℕ} [Nonempty (Fin count)] (sample : Fin count → Sample) =>
PMF.map sample (PMF.uniformOfFintype (Fin count))

```

AppliedModelingLib.empiricalSampleProbabilityMeasure

Declaration location: reusable library

Lean declaration body

```
type:
{Sample : Type u_1} →
[inst : MeasurableSpace Sample] →
{count : ℕ} → [Nonempty (Fin count)] → (Fin count → Sample) → MeasureTheory.ProbabilityMeasure Sample

value:
fun {Sample : Type u_1} [MeasurableSpace Sample] {count : ℕ} [Nonempty (Fin count)] (sample : Fin count → Sample) =>
((AppliedModelingLib.empiricalSampleLaw sample).toMeasure,
AppliedModelingLib.empiricalSampleProbabilityMeasure._proof_1 sample)
```

Direct retained dependencies

- [AppliedModelingLib.empiricalSampleLaw](#)

AppliedModelingLib.empiricalSampleProbabilityMeasureOfPos

Declaration location: reusable library

Lean declaration body

```
type:
{Sample : Type u_1} →
[inst : MeasurableSpace Sample] →
{count : ℕ} → 0 < count → (Fin count → Sample) → MeasureTheory.ProbabilityMeasure Sample

value:
fun {Sample : Type u_1} [MeasurableSpace Sample] {count : ℕ} (hcount : 0 < count) (sample : Fin count → Sample) =>
AppliedModelingLib.empiricalSampleProbabilityMeasure sample
```

Direct retained dependencies

- [AppliedModelingLib.empiricalSampleProbabilityMeasure](#)

AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace

Declaration location: reusable library

Lean declaration body

```
type:
(BatchIndex : ℕ → Type u_1) →
(Data : Type u_2) →
[MeasurableSpace Data] →
(round : ℕ) → MeasurableSpace (AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data round)

value:
fun (BatchIndex : ℕ → Type u_1) (Data : Type u_2) [MeasurableSpace Data] (round : ℕ) =>
Nat.casesAuxOn (motive := fun (a : ℕ) =>
round = a → MeasurableSpace (AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data round)) round
(fun (h : round = 0) =>
AppliedModelingLib.heterogeneousBatchTraceCoordinateFintype._proof_1 round h ▶ id inferInstance)
(fun (n : ℕ) (h : round = n + 1) =>
AppliedModelingLib.heterogeneousBatchTraceCoordinateFintype._proof_2 round n h ▶ id inferInstance)
(Eq.refl round)
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)

AppliedModelingLib.heterogeneousBatchTraceInfiniteLaw

Declaration location: reusable library

Lean declaration body

```
type:
{Data : Type u_1} →
[inst : MeasurableSpace Data] →
(BatchIndex : ℕ → Type u_2) →
(kernel :
(iteration : ℕ) →
ProbabilityTheory.Kernel
```

```

((i :  $\mathbb{N}$  (Finset.lic iteration))) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data  $\uparrow$  i)
(AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data (iteration + 1))) →
[ $\forall$  (iteration :  $\mathbb{N}$ ), ProbabilityTheory.IsMarkovKernel (kernel iteration)] →
MeasureTheory.Measure ((i :  $\mathbb{N}$ ) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data i)

```

value:

```

fun {Data : Type u_1} [MeasurableSpace Data] (BatchIndex :  $\mathbb{N}$  → Type u_2)
(kernel :
  (iteration :  $\mathbb{N}$ ) →
  ProbabilityTheory.Kernel
  ((i :  $\mathbb{N}$  (Finset.lic iteration))) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data  $\uparrow$  i)
  (AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data (iteration + 1)))
[ $\forall$  (iteration :  $\mathbb{N}$ ), ProbabilityTheory.IsMarkovKernel (kernel iteration)] =>
ProbabilityTheory.Kernel.trajMeasure (MeasureTheory.Measure.dirac PUnit.unit) kernel

```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace](#)

AppliedModelingLib.heterogeneousBatchTraceMeasurableSpace

Declaration location: reusable library

Lean declaration body

```

type:
(BatchIndex :  $\mathbb{N}$  → Type u_1) →
(Data : Type u_2) →
[MeasurableSpace Data] →
(iteration :  $\mathbb{N}$ ) → MeasurableSpace (AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration)

```

value:

```

fun (BatchIndex :  $\mathbb{N}$  → Type u_1) (Data : Type u_2) [MeasurableSpace Data] (iteration :  $\mathbb{N}$ ) => id inferInstance

```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)

AppliedModelingLib.heterogeneousBatchTraceOfInfiniteTrace

Declaration location: reusable library

Lean declaration body

```

type:
{BatchIndex :  $\mathbb{N}$  → Type u_1} →
{Data : Type u_2} →
(iteration :  $\mathbb{N}$ ) →
((i :  $\mathbb{N}$ ) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data i) →
(round : Fin iteration) → BatchIndex  $\uparrow$  round → Data

```

value:

```

fun {BatchIndex :  $\mathbb{N}$  → Type u_1} {Data : Type u_2} (iteration :  $\mathbb{N}$ )
(trace : (i :  $\mathbb{N}$ ) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data i) (round : Fin iteration)
(a : BatchIndex  $\uparrow$  round) =>
trace ( $\uparrow$  round + 1) a

```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)

AppliedModelingLib.heterogeneousBatchTraceOfPrefix

Declaration location: reusable library

Lean declaration body

```

type:
{BatchIndex :  $\mathbb{N}$  → Type u_1} →
{Data : Type u_2} →
(iteration :  $\mathbb{N}$ ) →
((i :  $\mathbb{N}$  (Finset.lic iteration))) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data  $\uparrow$  i) →
(round : Fin iteration) → BatchIndex  $\uparrow$  round → Data

```

value:

```

fun {BatchIndex :  $\mathbb{N}$  → Type u_1} {Data : Type u_2} (iteration :  $\mathbb{N}$ )
(historyPrefix :
  (i :  $\mathbb{N}$  (Finset.lic iteration))) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data  $\uparrow$  i)
  (round : Fin iteration) (a : BatchIndex  $\uparrow$  round) =>
historyPrefix ( $\uparrow$  round + 1, AppliedModelingLib.heterogeneousBatchTraceOfPrefix._proof_1 iteration round) a

```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)

AppliedModelingLib.heterogeneousBatchTraceBadEventPair

Declaration location: reusable library

Lean declaration body

```
type:
{Data : Type u_1} →
  (BatchIndex : ℕ → Type u_2) →
    ((iteration : ℕ) →
      AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration → (BatchIndex iteration → Data) → Prop) →
      (iteration : ℕ) →
        ((i : 1 (Finset.lic iteration)) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data ↑i) ×
          AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data (iteration + 1) →
            Prop

value:
fun {Data : Type u_1} (BatchIndex : ℕ → Type u_2)
  (bad :
    (iteration : ℕ) →
      AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration → (BatchIndex iteration → Data) → Prop)
  (iteration : ℕ)
  (a :
    ((i : 1 (Finset.lic iteration)) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data ↑i) ×
      AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data (iteration + 1)) =>
  {pair :
    ((i : 1 (Finset.lic iteration)) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data ↑i) ×
      AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data (iteration + 1) |
    bad iteration (AppliedModelingLib.heterogeneousBatchTraceOfPrefix iteration pair.1) pair.2}
  a
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)
- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.heterogeneousBatchTraceOfPrefix](#)

AppliedModelingLib.heterogeneousBatchTraceOneObservationKernel

Declaration location: reusable library

Lean declaration body

```
type:
{State : Type u_1} →
{Data : Type u_2} →
  [inst : MeasurableSpace State] →
  [inst_1 : MeasurableSpace Data] →
    (BatchIndex : ℕ → Type u_3) →
      ProbabilityTheory.Kernel State Data →
        (stateOfHistory :
          (iteration : ℕ) → AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration → State) →
          (iteration : ℕ) →
            Measurable (stateOfHistory iteration) →
              ProbabilityTheory.Kernel
                ((i : 1 (Finset.lic iteration)) →
                  AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data ↑i)
                  Data

value:
fun {State : Type u_1} {Data : Type u_2} [MeasurableSpace State] [MeasurableSpace Data] (BatchIndex : ℕ → Type u_3)
  (dataKernel : ProbabilityTheory.Kernel State Data)
  (stateOfHistory : (iteration : ℕ) → AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration → State)
  (iteration : ℕ) (hstate : Measurable (stateOfHistory iteration)) =>
  dataKernel.comap
    (fun
      (historyPrefix :
        (i : 1 (Finset.lic iteration)) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data ↑i) =>
        stateOfHistory iteration (AppliedModelingLib.heterogeneousBatchTraceOfPrefix iteration historyPrefix))
      (AppliedModelingLib.heterogeneousBatchTraceOneObservationKernel._proof_1 BatchIndex stateOfHistory iteration hstate)
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)
- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace](#)
- [AppliedModelingLib.heterogeneousBatchTraceMeasurableSpace](#)
- [AppliedModelingLib.heterogeneousBatchTraceOfPrefix](#)

AppliedModelingLib.heterogeneousBatchTraceSnoc

Declaration location: reusable library

Lean declaration body

```
type:
{BatchIndex : ℕ → Type u_1} →
{Data : Type u_2} →
{iteration : ℕ} →
  AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration →
  (BatchIndex iteration → Data) → (round : Fin (iteration + 1)) → BatchIndex ↑ round → Data

value:
fun {BatchIndex : ℕ → Type u_1} {Data : Type u_2} {iteration : ℕ}
  (history : AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration)
  (batch : BatchIndex iteration → Data) (round : Fin (iteration + 1)) (a : BatchIndex ↑ round) =>
  Fin.lastCases (motive := fun (x : Fin (iteration + 1)) => BatchIndex ↑ x → Data) batch
  (fun (previous : Fin iteration) => history previous) round a
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)

AppliedModelingLib.hilbertProjection

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
[inst : NormedAddCommGroup Parameter] →
[inst_1 : InnerProductSpace ℝ Parameter] →
[CompleteSpace Parameter] →
  (domain : Set Parameter) → domain.Nonempty → IsClosed domain → Convex ℝ domain → Parameter → Parameter

value:
fun {Parameter : Type u_1} [NormedAddCommGroup Parameter] [InnerProductSpace ℝ Parameter] [CompleteSpace Parameter]
  (domain : Set Parameter) (hnonempty : domain.Nonempty) (hclosed : IsClosed domain) (hconvex : Convex ℝ domain)
  (a : Parameter) =>
  Classical.choose (AppliedModelingLib.hilbertProjection._proof_1 domain hnonempty hclosed hconvex a)
```

AppliedModelingLib.IsMeasurePerformativelyStable

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] → AppliedModelingLib.MeasurePerformativeModel Parameter Data → Parameter → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
  (model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (parameter : Parameter) =>
  ∀ (candidate : Parameter),
  AppliedModelingLib.measureDecoupledPerformativeRisk model parameter parameter ≤
  AppliedModelingLib.measureDecoupledPerformativeRisk model parameter candidate
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.measureDecoupledPerformativeRisk](#)

AppliedModelingLib.IsMeasurePerformativelyStableOn

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
  AppliedModelingLib.MeasurePerformativeModel Parameter Data → Set Parameter → Parameter → Prop

value:
```

```

fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
  (model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (domain : Set Parameter)
  (parameter : Parameter) =>
  parameter ∈ domain ∧
  ∀ candidate ∈ domain,
  AppliedModelingLib.measureDecoupledPerformativeRisk model parameter parameter ≤
  AppliedModelingLib.measureDecoupledPerformativeRisk model parameter candidate

```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.measureDecoupledPerformativeRisk](#)

AppliedModelingLib.IsMeasureRepeatedRiskMinimization

Declaration location: reusable library

Lean declaration body

```

type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
AppliedModelingLib.MeasurePerformativeModel Parameter Data → (Parameter → Parameter) → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
  (model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (update : Parameter → Parameter) =>
  ∀ (deployed candidate : Parameter),
  AppliedModelingLib.measureDecoupledPerformativeRisk model deployed (update deployed) ≤
  AppliedModelingLib.measureDecoupledPerformativeRisk model deployed candidate

```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.measureDecoupledPerformativeRisk](#)

AppliedModelingLib.IsMeasureRepeatedRiskMinimizationOn

Declaration location: reusable library

Lean declaration body

```

type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
AppliedModelingLib.MeasurePerformativeModel Parameter Data → Set Parameter → (Parameter → Parameter) → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
  (model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (domain : Set Parameter)
  (update : Parameter → Parameter) =>
  ∀ deployed ∈ domain,
  update deployed ∈ domain ∧
  ∀ candidate ∈ domain,
  AppliedModelingLib.measureDecoupledPerformativeRisk model deployed (update deployed) ≤
  AppliedModelingLib.measureDecoupledPerformativeRisk model deployed candidate

```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.measureDecoupledPerformativeRisk](#)

AppliedModelingLib.measureEmpiricalDecoupledPerformativeRisk

Declaration location: reusable library

Lean declaration body

```

type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
AppliedModelingLib.MeasurePerformativeModel Parameter Data → MeasureTheory.ProbabilityMeasure Data → Parameter → ℝ

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
  (model : AppliedModelingLib.MeasurePerformativeModel Parameter Data)
  (empiricalLaw : MeasureTheory.ProbabilityMeasure Data) (evaluatedParameter : Parameter) =>
  ∫ (datum : Data), model.loss datum evaluatedParameter ∂ ↑ empiricalLaw

```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)

AppliedModelingLib.IsMeasureEmpiricalRiskMinimizerOn

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
AppliedModelingLib.MeasurePerformativeModel Parameter Data →
Set Parameter → MeasureTheory.ProbabilityMeasure Data → Parameter → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (domain : Set Parameter)
(empiricalLaw : MeasureTheory.ProbabilityMeasure Data) (minimizer : Parameter) =>
minimizer ∈ domain ∧
∀ candidate ∈ domain,
AppliedModelingLib.measureEmpiricalDecoupledPerformativeRisk model empiricalLaw minimizer ≤
AppliedModelingLib.measureEmpiricalDecoupledPerformativeRisk model empiricalLaw candidate
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.measureEmpiricalDecoupledPerformativeRisk](#)

AppliedModelingLib.IsHeterogeneousMeasureRepeatedEmpiricalRiskMinimizationTrajectoryOn

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
AppliedModelingLib.MeasurePerformativeModel Parameter Data →
Set Parameter →
(sampleCount : ℕ → ℕ) →
(∀ (iteration : ℕ), 0 < sampleCount iteration) →
((iteration : ℕ) →
AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (sampleCount round)) Data
iteration →
Parameter) →
Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (domain : Set Parameter) (sampleCount : ℕ → ℕ)
(hcountPositive : ∀ (iteration : ℕ), 0 < sampleCount iteration)
(deployedOfHistory :
(iteration : ℕ) →
AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (sampleCount round)) Data iteration →
Parameter) =>
∀ (iteration : ℕ)
(history : AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (sampleCount round)) Data iteration)
(batch : Fin (sampleCount iteration) → Data),
AppliedModelingLib.IsMeasureEmpiricalRiskMinimizerOn model domain
(AppliedModelingLib.empiricalSampleProbabilityMeasureOfPos (hcountPositive iteration) batch)
(deployedOfHistory (iteration + 1) (AppliedModelingLib.heterogeneousBatchTraceSnoc history batch))
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)
- [AppliedModelingLib.IsMeasureEmpiricalRiskMinimizerOn](#)
- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.empiricalSampleProbabilityMeasureOfPos](#)
- [AppliedModelingLib.heterogeneousBatchTraceSnoc](#)

AppliedModelingLib.measureEmpiricalPopulationGradient

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : NormedAddCommGroup Parameter] →
[InnerProductSpace ℝ Parameter] →
[inst : MeasurableSpace Data] →
(Data → Parameter → Parameter) → MeasureTheory.ProbabilityMeasure Data → Parameter → Parameter

value:
fun {Parameter : Type u_1} {Data : Type u_2} [NormedAddCommGroup Parameter] [InnerProductSpace ℝ Parameter]
[MeasurableSpace Data] (gradient : Data → Parameter → Parameter)
(empiricalLaw : MeasureTheory.ProbabilityMeasure Data) (evaluatedParameter : Parameter) =>
f (datum : Data), gradient datum evaluatedParameter ∂ f empiricalLaw
```

AppliedModelingLib.measurePerformativeDeployedLaw

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
AppliedModelingLib.MeasurePerformativeModel Parameter Data →
(BatchIndex : ℕ → Type u_3) →
((iteration : ℕ) → AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration → Parameter) →
(iteration : ℕ) →
AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration →
MeasureTheory.ProbabilityMeasure Data

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (BatchIndex : ℕ → Type u_3)
(deployedOfHistory :
(iteration : ℕ) → AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration → Parameter)
(iteration : ℕ) (a : AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration) =>
model.dataLaw (deployedOfHistory iteration a)
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)
- [AppliedModelingLib.MeasurePerformativeModel](#)

AppliedModelingLib.measurePerformativeHeterogeneousBatchTraceInfiniteLaw

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Parameter] →
[inst_1 : MeasurableSpace Data] →
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) →
AppliedModelingLib.MeasurePerformativeSamplingKernel model →
(sampleCount : ℕ → ℕ) →
(deployedOfHistory :
(iteration : ℕ) →
AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (sampleCount round)) Data
iteration →
Parameter) →
(∀ (iteration : ℕ), Measurable (deployedOfHistory iteration)) →
MeasureTheory.Measure
((i : ℕ) →
AppliedModelingLib.HeterogeneousBatchTraceCoordinate (fun (round : ℕ) => Fin (sampleCount round))
Data i)

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Parameter] [MeasurableSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data)
(sampling : AppliedModelingLib.MeasurePerformativeSamplingKernel model) (sampleCount : ℕ → ℕ)
(deployedOfHistory :
(iteration : ℕ) →
AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (sampleCount round)) Data iteration →
Parameter)
(hmeasurableDeployed : ∀ (iteration : ℕ), Measurable (deployedOfHistory iteration)) =>
AppliedModelingLib.heterogeneousBatchTraceInfiniteLaw (fun (round : ℕ) => Fin (sampleCount round))
fun (iteration : ℕ) =>
AppliedModelingLib.Probability.finiteIDBatchKernel (sampleCount iteration)
(AppliedModelingLib.heterogeneousBatchTraceOneObservationKernel (fun (round : ℕ) => Fin (sampleCount round))
sampling.kernel deployedOfHistory iteration (hmeasurableDeployed iteration))
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)
- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.MeasurePerformativeSamplingKernel](#)
- [AppliedModelingLib.Probability.finiteIIDBatchKernel](#)
- [AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace](#)
- [AppliedModelingLib.heterogeneousBatchTraceInfiniteLaw](#)
- [AppliedModelingLib.heterogeneousBatchTraceMeasurableSpace](#)
- [AppliedModelingLib.heterogeneousBatchTraceOneObservationKernel](#)

AppliedModelingLib.measurePerformativeRisk

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] → AppliedModelingLib.MeasurePerformativeModel Parameter Data → Parameter → ℝ

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (parameter : Parameter) =>
AppliedModelingLib.measureDecoupledPerformativeRisk model parameter parameter
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.measureDecoupledPerformativeRisk](#)

AppliedModelingLib.IsMeasurePerformativelyOptimal

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] → AppliedModelingLib.MeasurePerformativeModel Parameter Data → Parameter → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (parameter : Parameter) =>
∀ (candidate : Parameter),
AppliedModelingLib.measurePerformativeRisk model parameter ≤
AppliedModelingLib.measurePerformativeRisk model candidate
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.measurePerformativeRisk](#)

AppliedModelingLib.measureSampledGradientDescentUpdate

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : NormedAddCommGroup Parameter] →
[InnerProductSpace ℝ Parameter] →
[MeasurableSpace Data] →
(Data → Parameter → Parameter) →
(Parameter → Parameter) →
ℝ → {sampleCount : ℕ} → 0 < sampleCount → (Fin sampleCount → Data) → Parameter → Parameter

value:
fun {Parameter : Type u_1} {Data : Type u_2} [NormedAddCommGroup Parameter] [InnerProductSpace ℝ Parameter]
[MeasurableSpace Data] (gradient : Data → Parameter → Parameter) (project : Parameter → Parameter) (stepSize : ℝ)
{sampleCount : ℕ} (hsampleCount : 0 < sampleCount) (sample : Fin sampleCount → Data) (a : Parameter) =>
project
(a -
stepSize •
AppliedModelingLib.measureEmpiricalPopulationGradient gradient
(AppliedModelingLib.empiricalSampleProbabilityMeasureOfPos hsampleCount sample) a)
```

Direct retained dependencies

- [AppliedModelingLib.empiricalSampleProbabilityMeasureOfPos](#)
- [AppliedModelingLib.measureEmpiricalPopulationGradient](#)

AppliedModelingLib.pOneFournierGuillinAdaptiveEuclideanSelectedShellPartitionCertificateBadEvent

Declaration location: reusable library

Lean declaration body

```
type:
(dimension : ℕ) →
ℕ →
ℝ →
(ℕ → ℝ) →
(countSchedule : ℕ → ℕ) →
(iteration : ℕ) →
AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (countSchedule round))
(EuclideanSpace ℝ (Fin dimension)) iteration →
MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension))) →
(iteration : ℕ) →
AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (countSchedule round))
(EuclideanSpace ℝ (Fin dimension)) iteration →
(Fin (countSchedule iteration) → EuclideanSpace ℝ (Fin dimension)) → Prop

value:
fun (dimension cutoff : ℕ) (eta deviation : ℝ) (confidence : ℕ → ℝ) (countSchedule : ℕ → ℕ)
(law :
(iteration : ℕ) →
AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (countSchedule round))
(EuclideanSpace ℝ (Fin dimension)) iteration →
MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension)))
(iteration : ℕ)
(a :
AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (countSchedule round))
(EuclideanSpace ℝ (Fin dimension)) iteration)
(a_1 : Fin (countSchedule iteration) → EuclideanSpace ℝ (Fin dimension)) =>
AppliedModelingLib.Probability.pOneFournierGuillinSelectedShellPartitionCertificate dimension (law iteration a)
(countSchedule iteration) eta deviation
(AppliedModelingLib.Probability.selectedShellGeometricEffectiveHeadDepth dimension cutoff (countSchedule iteration)
(law iteration a) (confidence iteration))
(confidence iteration) a_1)
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinSelectedShellPartitionCertificate](#)
- [AppliedModelingLib.Probability.selectedShellGeometricEffectiveHeadDepth](#)

AppliedModelingLib.pOneFournierGuillinAdaptiveEuclideanWassersteinOneThreshold

Declaration location: reusable library

Lean declaration body

```
type:
  (dimension : ℕ) →
    ℕ →
      ℝ →
        ℝ →
          ℝ →
            (ℕ → ℝ) →
              (countSchedule : ℕ → ℕ) →
                ((iteration : ℕ) →
                  AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (countSchedule round))
                    (EuclideanSpace ℝ (Fin dimension)) iteration →
                      MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension))) →
                    (iteration : ℕ) →
                      AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (countSchedule round))
                        (EuclideanSpace ℝ (Fin dimension)) iteration →
                          ENNReal

value:
  fun (dimension cutoff : ℕ) (alpha gamma momentBound deviation : ℝ) (confidence : ℕ → ℝ) (countSchedule : ℕ → ℕ)
    (law :
      (iteration : ℕ) →
        AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (countSchedule round))
          (EuclideanSpace ℝ (Fin dimension)) iteration →
            MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension)))
      (iteration : ℕ)
      (a :
        AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (countSchedule round))
          (EuclideanSpace ℝ (Fin dimension)) iteration) =>
        ENNReal.ofReal (√ ↑ dimension * deviation) +
        ENNReal.ofReal
          (2 * √ ↑ dimension *
            (32 / 15 *
              AppliedModelingLib.Probability.fifthOrderExponentialRadialTailConstant (law iteration a) alpha gamma /
              16 ^ cutoff)) +
        AppliedModelingLib.Probability.selectedShellGeometricHeadSchedule_byDimension dimension cutoff
          (countSchedule iteration) (confidence iteration) (↑ (Nat.factorial 5) * gamma-1 ^ 5 * momentBound)
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)
- [AppliedModelingLib.Probability.fifthOrderExponentialRadialTailConstant](#)
- [AppliedModelingLib.Probability.selectedShellGeometricHeadSchedule_byDimension](#)

AppliedModelingLib.pOneFournierGuillinAdaptiveEuclideanWassersteinOneBadEvent

Declaration location: reusable library

Lean declaration body

```
type:
  (dimension : ℕ) →
    ℕ →
      ℝ →
        ℝ →
          ℝ →
            (ℕ → ℝ) →
              (countSchedule : ℕ → ℕ) →
                (∀ (round : ℕ), 0 < countSchedule round) →
                  ((iteration : ℕ) →
                    AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (countSchedule round))
                      (EuclideanSpace ℝ (Fin dimension)) iteration →
                        MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension))) →
                    (iteration : ℕ) →
                      AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (countSchedule round))
                        (EuclideanSpace ℝ (Fin dimension)) iteration →
                          (Fin (countSchedule iteration) → EuclideanSpace ℝ (Fin dimension)) → Prop

value:
  fun (dimension cutoff : ℕ) (alpha gamma momentBound deviation : ℝ) (confidence : ℕ → ℝ) (countSchedule : ℕ → ℕ)
    (hcountPositive : ∀ (round : ℕ), 0 < countSchedule round)
    (law :
      (iteration : ℕ) →
        AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (countSchedule round))
          (EuclideanSpace ℝ (Fin dimension)) iteration →
            MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin dimension)))
      (iteration : ℕ)
      (a :
        AppliedModelingLib.HeterogeneousBatchTrace (fun (round : ℕ) => Fin (countSchedule round))
          (EuclideanSpace ℝ (Fin dimension)) iteration)
      (a_1 : Fin (countSchedule iteration) → EuclideanSpace ℝ (Fin dimension)) =>
    AppliedModelingLib.pOneFournierGuillinAdaptiveEuclideanWassersteinOneThreshold dimension cutoff alpha gamma
      momentBound deviation confidence countSchedule law iteration a <
    AppliedModelingLib.ProbabilityCoupling.wassersteinOne (law iteration a)
      (AppliedModelingLib.empiricalSampleProbabilityMeasureOfPos (hcountPositive iteration) a_1)
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)
- [AppliedModelingLib.ProbabilityCoupling.wassersteinOne](#)
- [AppliedModelingLib.empiricalSampleProbabilityMeasureOfPos](#)
- [AppliedModelingLib.pOneFournierGuillinAdaptiveEuclideanWassersteinOneThreshold](#)

AppliedModelingLib.pOneFournierGuillinTheorem310CappedDeviation

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → ℝ → ℝ → ℝ → ℝ → ℝ → ℝ

value:
fun (dimension : ℕ) (eta alpha gamma momentBound scaledDeviation : ℝ) =>
  AppliedModelingLib.Probability.pOneFournierGuillinUniformCappedDeviation dimension eta alpha gamma momentBound
  scaledDeviation
```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinUniformCappedDeviation](#)

AppliedModelingLib.pmfExp

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → [Fintype α] → PMF α → (α → ℝ) → ℝ

value:
fun {α : Type u_1} [Fintype α] (μ : PMF α) (f : α → ℝ) => ∑ a : α, (μ a).toReal * f a
```

AppliedModelingLib.FiniteCoupling.expectedCost

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} →
{β : Type u_2} →
[Fintype α] → [Fintype β] → {μ : PMF α} → {ν : PMF β} → AppliedModelingLib.FiniteCoupling μ ν → (α → β → ℝ) → ℝ

value:
fun {α : Type u_1} {β : Type u_2} [Fintype α] [Fintype β] {μ : PMF α} {ν : PMF β}
  (coupling : AppliedModelingLib.FiniteCoupling μ ν) (cost : α → β → ℝ) =>
  AppliedModelingLib.pmfExp coupling.law fun (pair : α × β) => cost pair.1 pair.2
```

Direct retained dependencies

- [AppliedModelingLib.FiniteCoupling](#)
- [AppliedModelingLib.pmfExp](#)

AppliedModelingLib.FiniteCoupling.transportCostSet

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → {β : Type u_2} → [Fintype α] → [Fintype β] → PMF α → PMF β → (α → β → ℝ) → ℝ → Prop

value:
fun {α : Type u_1} {β : Type u_2} [Fintype α] [Fintype β] (μ : PMF α) (ν : PMF β) (cost : α → β → ℝ) (a : ℝ) =>
  {bound : ℝ | ∃ (coupling : AppliedModelingLib.FiniteCoupling μ ν), coupling.expectedCost cost = bound} a
```

Direct retained dependencies

- [AppliedModelingLib.FiniteCoupling](#)
- [AppliedModelingLib.FiniteCoupling.expectedCost](#)

AppliedModelingLib.FiniteCoupling.finiteWassersteinOne

Declaration location: reusable library

Lean declaration body

```
type:
{Data : Type u_3} → [Fintype Data] → [DecidableEq Data] → [MetricSpace Data] → PMF Data → PMF Data → ℝ

value:
fun {Data : Type u_3} [Fintype Data] [DecidableEq Data] [MetricSpace Data] (μ v : PMF Data) =>
  sInf (AppliedModelingLib.FiniteCoupling.transportCostSet μ v fun (left right : Data) => dist left right)
```

Direct retained dependencies

- [AppliedModelingLib.FiniteCoupling.transportCostSet](#)

AppliedModelingLib.IsFiniteWassersteinSensitive

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : Fintype Data] →
[inst_1 : DecidableEq Data] →
[MetricSpace Parameter] →
[MetricSpace Data] → AppliedModelingLib.FinitePerformativeModel Parameter Data → ℝ → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [Fintype Data] [DecidableEq Data] [MetricSpace Parameter]
  [MetricSpace Data] (model : AppliedModelingLib.FinitePerformativeModel Parameter Data) (sensitivity : ℝ) =>
  ∀ (first second : Parameter),
    AppliedModelingLib.FiniteCoupling.finiteWassersteinOne (model.dataLaw first) (model.dataLaw second) ≤
    sensitivity * dist first second
```

Direct retained dependencies

- [AppliedModelingLib.FiniteCoupling.finiteWassersteinOne](#)
- [AppliedModelingLib.FinitePerformativeModel](#)

AppliedModelingLib.performativeRisk

Declaration location: reusable library

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : Fintype Data] →
[inst_1 : DecidableEq Data] → AppliedModelingLib.FinitePerformativeModel Parameter Data → Parameter → ℝ

value:
fun {Parameter : Type u_1} {Data : Type u_2} [Fintype Data] [DecidableEq Data]
  (model : AppliedModelingLib.FinitePerformativeModel Parameter Data) (parameter : Parameter) =>
  AppliedModelingLib.pmfExp (model.dataLaw parameter) fun (datum : Data) => model.loss datum parameter
```

Direct retained dependencies

- [AppliedModelingLib.FinitePerformativeModel](#)
- [AppliedModelingLib.pmfExp](#)

AppliedModelingLib.proposition36aDomain

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \text{Prop}$ 

value:
fun (a :  $\mathbb{R}$ ) => Set.lcc (-1) 1 a
```

AppliedModelingLib.proposition36aMeasureGradient

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (beta datum _candidate :  $\mathbb{R}$ ) => beta * datum
```

AppliedModelingLib.proposition36aMeasureLaw

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{MeasureTheory.ProbabilityMeasure } \mathbb{R}$ 

value:
fun (epsilon theta :  $\mathbb{R}$ ) => MeasureTheory.diracProba (epsilon * theta)
```

AppliedModelingLib.proposition36aMeasureLoss

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (beta datum candidate :  $\mathbb{R}$ ) => beta * datum * candidate
```

AppliedModelingLib.proposition36aMeasureModel

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{AppliedModelingLib.MeasurePerformativeModel } \mathbb{R} \mathbb{R}$ 

value:
fun (epsilon beta :  $\mathbb{R}$ ) =>
{ dataLaw := AppliedModelingLib.proposition36aMeasureLaw epsilon,
  loss := AppliedModelingLib.proposition36aMeasureLoss beta,
  loss_integrable := AppliedModelingLib.proposition36aMeasureModel._proof_1 epsilon beta }
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.proposition36aMeasureLaw](#)
- [AppliedModelingLib.proposition36aMeasureLoss](#)

AppliedModelingLib.proposition36aRRMUpdate

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (theta :  $\mathbb{R}$ ) => if theta < 0 then 1 else -1
```

AppliedModelingLib.proposition36bLowerEndpoint

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (epsilon :  $\mathbb{R}$ ) => -(1 / (2 * epsilon))
```

AppliedModelingLib.IsProposition36bLargePenalty

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{Prop}$ 

value:
fun (epsilon gamma penalty :  $\mathbb{R}$ ) =>
  gamma * (1 - AppliedModelingLib.proposition36bLowerEndpoint epsilon)  $\leq$  2 * penalty * epsilon  $\wedge$  2 * gamma  $\leq$  penalty
```

Direct retained dependencies

- [AppliedModelingLib.proposition36bLowerEndpoint](#)

AppliedModelingLib.proposition36bDomain

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{Prop}$ 

value:
fun (epsilon a :  $\mathbb{R}$ ) => Set.Icc (AppliedModelingLib.proposition36bLowerEndpoint epsilon) 2 a
```

Direct retained dependencies

- [AppliedModelingLib.proposition36bLowerEndpoint](#)

AppliedModelingLib.proposition36bMeasureLaw

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{MeasureTheory.ProbabilityMeasure } \mathbb{R}$ 

value:
fun (epsilon theta :  $\mathbb{R}$ ) => MeasureTheory.diracProba (epsilon * theta)
```

AppliedModelingLib.proposition36bMeasureLoss

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (gamma penalty datum candidate :  $\mathbb{R}$ ) => penalty * max (-1) (datum * candidate) + gamma / 2 * (candidate - 1) ^ 2
```

AppliedModelingLib.proposition36bMeasureModel

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{AppliedModelingLib.MeasurePerformativeModel } \mathbb{R} \mathbb{R}$ 

value:
fun (epsilon gamma penalty :  $\mathbb{R}$ ) =>
{ dataLaw := AppliedModelingLib.proposition36bMeasureLaw epsilon,
  loss := AppliedModelingLib.proposition36bMeasureLoss gamma penalty,
  loss_integrable := AppliedModelingLib.proposition36bMeasureModel._proof_1 epsilon gamma penalty }
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.proposition36bMeasureLaw](#)
- [AppliedModelingLib.proposition36bMeasureLoss](#)

AppliedModelingLib.proposition36bRRMUpdate

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (epsilon deployed :  $\mathbb{R}$ ) => if deployed = 2 then AppliedModelingLib.proposition36bLowerEndpoint epsilon else 2
```

Direct retained dependencies

- [AppliedModelingLib.proposition36bLowerEndpoint](#)

AppliedModelingLib.proposition36cMeasureLaw

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{MeasureTheory.ProbabilityMeasure } \mathbb{R}$ 

value:
fun (epsilon theta :  $\mathbb{R}$ ) => MeasureTheory.diracProba (1 + epsilon * theta)
```

AppliedModelingLib.proposition36cRegularityGradient

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (beta gamma datum candidate :  $\mathbb{R}$ ) => gamma * candidate - beta * datum
```

AppliedModelingLib.proposition36cRegularityLoss

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (beta gamma datum candidate :  $\mathbb{R}$ ) => gamma / 2 * candidate ^ 2 - beta * datum * candidate
```

AppliedModelingLib.proposition36cRegularityMeasureModel

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \text{AppliedModelingLib.MeasurePerformativeModel } \mathbb{R} \mathbb{R}$ 

value:
fun (epsilon beta gamma :  $\mathbb{R}$ ) =>
{ dataLaw := AppliedModelingLib.proposition36cMeasureLaw epsilon,
  loss := AppliedModelingLib.proposition36cRegularityLoss beta gamma,
  loss_integrable := AppliedModelingLib.proposition36cRegularityMeasureModel._proof_1 epsilon beta gamma }
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.proposition36cMeasureLaw](#)
- [AppliedModelingLib.proposition36cRegularityLoss](#)

AppliedModelingLib.proposition36cRegularityRRMUpdate

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (epsilon beta gamma theta :  $\mathbb{R}$ ) => beta / gamma * (1 + epsilon * theta)
```

AppliedModelingLib.proposition42BernoulliParameter

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \text{Bool} \rightarrow \text{NNReal}$ 

value:
fun (bias :  $\mathbb{R}$ ) (feature : Bool) => (if feature = true then 1 / 2 + bias else 1 / 2 - bias).toNNReal
```

AppliedModelingLib.proposition42BoolMetricSpace

Declaration location: reusable library

Lean declaration body

```
type:
MetricSpace Bool

value:
MetricSpace.induced (fun (value : Bool) => if value = true then 1 else 0)
  AppliedModelingLib.proposition42BoolMetricSpace._proof_1 inferInstance
```

AppliedModelingLib.proposition42Gradient

Declaration location: reusable library

Lean declaration body

```
type:
Bool  $\times$  Bool  $\rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
fun (datum : Bool  $\times$  Bool) (theta :  $\mathbb{R}$ ) =>
(2 * if datum.1 = true then 1 else -1) *
((theta * if datum.1 = true then 1 else -1) + 1 / 2 - if datum.2 = true then 1 else 0)
```

AppliedModelingLib.proposition42Law

Declaration location: reusable library

Lean declaration body

```
type:
(bias : ℝ) → |bias| ≤ 1 / 2 → PMF (Bool × Bool)

value:
fun (bias : ℝ) (hbias : |bias| ≤ 1 / 2) =>
(PMF.uniformOfFintype Bool).bind fun (feature : Bool) =>
  PMF.map (fun (label : Bool) => (feature, label))
    (PMF.bernoulli (AppliedModelingLib.proposition42BernoulliParameter bias feature)
      (AppliedModelingLib.proposition42BernoulliParameter_le_one bias feature hbias))
```

Direct retained dependencies

- [AppliedModelingLib.proposition42BernoulliParameter](#)

AppliedModelingLib.proposition42Loss

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → Bool × Bool → ℝ

value:
fun (theta : ℝ) (datum : Bool × Bool) =>
  ((if datum.2 = true then 1 else 0) - ((theta * if datum.1 = true then 1 else -1) + 1 / 2)) ^ 2
```

AppliedModelingLib.correctedProposition42Model

Declaration location: reusable library

Lean declaration body

```
type:
(mu epsilon : ℝ) →
|mu| ≤ 1 / 2 →
|mu + epsilon| ≤ 1 / 2 →
AppliedModelingLib.FinitePerformativeModel { theta : ℝ // theta ∈ Set.Icc 0 1 } (Bool × Bool)

value:
fun (mu epsilon : ℝ) (hmu : |mu| ≤ 1 / 2) (hmuEpsilon : |mu + epsilon| ≤ 1 / 2) =>
{
  dataLaw := fun (theta : { theta : ℝ // theta ∈ Set.Icc 0 1 }) =>
    AppliedModelingLib.proposition42Law (mu + epsilon * ↑theta)
    (AppliedModelingLib.correctedProposition42Model. proof_1 mu epsilon hmu hmuEpsilon theta),
  loss := fun (datum : Bool × Bool) (theta : { theta : ℝ // theta ∈ Set.Icc 0 1 }) =>
    AppliedModelingLib.proposition42Loss (↑theta) datum }
}
```

Direct retained dependencies

- [AppliedModelingLib.FinitePerformativeModel](#)
- [AppliedModelingLib.proposition42Law](#)
- [AppliedModelingLib.proposition42Loss](#)

AppliedModelingLib.theorem310FailureBudget

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → ℕ → ℝ

value:
fun (p : ℝ) (iteration : ℕ) => 6 * p / (Real.pi ^ 2 * ↑(iteration + 1) ^ 2)
```

AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConcreteCountSchedule

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow (\mathbb{N} \rightarrow \mathbb{R}) \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 

value:
fun (dimension cutoff :  $\mathbb{N}$ ) (eta alpha gamma tailBound deviation scaledDeviation p :  $\mathbb{R}$ ) (radiusSchedule :  $\mathbb{N} \rightarrow \mathbb{R}$ )
  (a :  $\mathbb{N}$ ) =>
  AppliedModelingLib.Probability.pOneFournierGuillinAllDimensionalAllShellConcreteCountSchedule dimension cutoff eta
    alpha gamma tailBound deviation scaledDeviation (AppliedModelingLib.theorem310FailureBudget p) radiusSchedule a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinAllDimensionalAllShellConcreteCountSchedule](#)
- [AppliedModelingLib.theorem310FailureBudget](#)

AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 

value:
fun (dimension cutoff :  $\mathbb{N}$ ) (eta alpha gamma tailBound deviation scaledDeviation p headTolerance :  $\mathbb{R}$ ) (a :  $\mathbb{N}$ ) =>
  AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConcreteCountSchedule dimension cutoff eta alpha gamma
    tailBound deviation scaledDeviation p (fun (x :  $\mathbb{N}$ ) => headTolerance) a
```

Direct retained dependencies

- [AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConcreteCountSchedule](#)

AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 

value:
fun (dimension cutoff :  $\mathbb{N}$ ) (eta alpha gamma momentBound tailBound scaledDeviation p headTolerance :  $\mathbb{R}$ ) (a :  $\mathbb{N}$ ) =>
  AppliedModelingLib.Probability.pOneFournierGuillinSupercriticalCappedAllShellConcreteCountSchedule dimension cutoff
    eta alpha gamma momentBound tailBound scaledDeviation (AppliedModelingLib.theorem310FailureBudget p)
      (fun (x :  $\mathbb{N}$ ) => headTolerance) a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.pOneFournierGuillinSupercriticalCappedAllShellConcreteCountSchedule](#)
- [AppliedModelingLib.theorem310FailureBudget](#)

AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalConfidenceSchedule

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{N} \rightarrow \mathbb{R}$ 

value:
fun (p :  $\mathbb{R}$ ) (a :  $\mathbb{N}$ ) =>
  AppliedModelingLib.Math.expConfidenceForHalfBudget (AppliedModelingLib.theorem310FailureBudget p a)
```

Direct retained dependencies

- [AppliedModelingLib.Math.expConfidenceForHalfBudget](#)
- [AppliedModelingLib.theorem310FailureBudget](#)

PZMH20PerformativePrediction.DomainRelative.frozenGradient

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : NormedAddCommGroup Parameter] →
[InnerProductSpace ℝ Parameter] →
[inst : MeasurableSpace Data] →
{domain : Set Parameter} →
AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain →
(Data → ↑ domain → Parameter) → ↑ domain → ↑ domain → Parameter

value:
fun {Parameter : Type u_1} {Data : Type u_2} [NormedAddCommGroup Parameter] [InnerProductSpace ℝ Parameter]
[MeasurableSpace Data] {domain : Set Parameter}
(model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain)
(gradient : Data → ↑ domain → Parameter) (deployed evaluated : ↑ domain) =>
λ (datum : Data), gradient datum evaluated ∂ ↑ (model.dataLaw deployed)
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModelOn](#)

PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : NormedAddCommGroup Parameter] →
[inst_1 : InnerProductSpace ℝ Parameter] →
[CompleteSpace Parameter] →
[inst_3 : MeasurableSpace Data] →
{domain : Set Parameter} →
AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain →
(Data → ↑ domain → Parameter) →
domain.Nonempty → IsClosed domain → Convex ℝ domain → ℝ → ↑ domain → ↑ domain

value:
fun {Parameter : Type u_1} {Data : Type u_2} [NormedAddCommGroup Parameter] [InnerProductSpace ℝ Parameter]
[CompleteSpace Parameter] [MeasurableSpace Data] {domain : Set Parameter}
(model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain)
(gradient : Data → ↑ domain → Parameter) (hnonempty : domain.Nonempty) (hclosed : IsClosed domain)
(hconvex : Convex ℝ domain) (stepSize : ℝ) (a : ↑ domain) =>
(AppliedModelingLib.hilbertProjection domain hnonempty hclosed hconvex
(↑ a - stepSize • PZMH20PerformativePrediction.DomainRelative.frozenGradient model gradient a),
PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn._proof_1 model gradient hnonempty hclosed hconvex stepSize
a)
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModelOn](#)
- [AppliedModelingLib.hilbertProjection](#)
- [PZMH20PerformativePrediction.DomainRelative.frozenGradient](#)

PZMH20PerformativePrediction.EmpiricalMinimizerPerturbation.IsPointwiseGradientStronglyConvexOn

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : NormedAddCommGroup Parameter] →
[InnerProductSpace ℝ Parameter] →
(Data → Parameter → ℝ) → (Data → Parameter → Parameter) → Set Parameter → ℝ → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [NormedAddCommGroup Parameter] [InnerProductSpace ℝ Parameter]
(loss : Data → Parameter → ℝ) (gradient : Data → Parameter → Parameter) (domain : Set Parameter) (modulus : ℝ) =>
λ (datum : Data),
λ center ∈ domain,
λ candidate ∈ domain,
loss datum candidate ≥
loss datum center + inner ℝ (gradient datum center) (candidate - center) +
modulus / 2 * ‖candidate - center‖ ^ 2
```

PZMH20PerformativePrediction.IsFiniteWassersteinSensitive

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : Fintype Data] →
[inst_1 : DecidableEq Data] →
[MetricSpace Parameter] →
[MetricSpace Data] → AppliedModelingLib.FinitePerformativeModel Parameter Data → ℝ → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [Fintype Data] [DecidableEq Data] [MetricSpace Parameter]
[MetricSpace Data] (model : AppliedModelingLib.FinitePerformativeModel Parameter Data) (sensitivity : ℝ) =>
AppliedModelingLib.IsFiniteWassersteinSensitive model sensitivity
```

Direct retained dependencies

- [AppliedModelingLib.FinitePerformativeModel](#)
- [AppliedModelingLib.IsFiniteWassersteinSensitive](#)

PZMH20PerformativePrediction.IsMeasurePointwiseGradientStronglyConvex

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
[inst_1 : NormedAddCommGroup Parameter] →
[InnerProductSpace ℝ Parameter] →
AppliedModelingLib.MeasurePerformativeModel Parameter Data → (Data → Parameter → Parameter) → ℝ → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data] [NormedAddCommGroup Parameter]
[InnerProductSpace ℝ Parameter] (model : AppliedModelingLib.MeasurePerformativeModel Parameter Data)
(gradient : Data → Parameter → Parameter) (modulus : ℝ) =>
AppliedModelingLib.IsMeasurePointwiseGradientStronglyConvex model gradient modulus
```

Direct retained dependencies

- [AppliedModelingLib.IsMeasurePointwiseGradientStronglyConvex](#)
- [AppliedModelingLib.MeasurePerformativeModel](#)

PZMH20PerformativePrediction.IsMeasureWassersteinSensitiveOn

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[MetricSpace Parameter] →
[inst : MeasurableSpace Data] →
[MetricSpace Data] → AppliedModelingLib.MeasurePerformativeModel Parameter Data → Set Parameter → ℝ → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MetricSpace Parameter] [MeasurableSpace Data] [MetricSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (domain : Set Parameter) (sensitivity : ℝ) =>
AppliedModelingLib.IsMeasureWassersteinSensitiveOn model domain sensitivity
```

Direct retained dependencies

- [AppliedModelingLib.IsMeasureWassersteinSensitiveOn](#)
- [AppliedModelingLib.MeasurePerformativeModel](#)

PZMH20PerformativePrediction.measureLossOnDomain

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
{domain : Set Parameter} →
AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain → Data → Parameter → ℝ

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data] {domain : Set Parameter}
(model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain) (datum : Data)
(parameter : Parameter) =>
if hparameter : parameter ∈ domain then model.loss datum (parameter, hparameter) else 0
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModelOn](#)

PZMH20PerformativePrediction.proposition36bExactDiracEndpointCounterexampleSpec

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℝ → ℝ → Prop

value:
fun (epsilon gamma penalty : ℝ) =>
0 < epsilon →
0 < gamma →
AppliedModelingLib.IsProposition36bLargePenalty epsilon gamma penalty →
PZMH20PerformativePrediction.IsMeasureWassersteinSensitive
(AppliedModelingLib.proposition36bMeasureModel epsilon gamma penalty) epsilon ∧
(∀ (datum : ℝ),
StrongConvexOn Set.univ gamma fun (candidate : ℝ) =>
AppliedModelingLib.proposition36bMeasureLoss gamma penalty datum candidate) ∧
¬DifferentiableAt ℝ
(fun (candidate : ℝ) =>
AppliedModelingLib.proposition36bMeasureLoss gamma penalty (epsilon * 2) candidate)
(AppliedModelingLib.proposition36bLowerEndpoint epsilon) ∧
((AppliedModelingLib.proposition36bRRMUpdate epsilon 2 ∈ AppliedModelingLib.proposition36bDomain epsilon ∧
∀ candidate ∈ AppliedModelingLib.proposition36bDomain epsilon,
AppliedModelingLib.measureDecoupledPerformativeRisk
(AppliedModelingLib.proposition36bMeasureModel epsilon gamma penalty) 2
(AppliedModelingLib.proposition36bRRMUpdate epsilon 2) ≤
AppliedModelingLib.measureDecoupledPerformativeRisk
(AppliedModelingLib.proposition36bMeasureModel epsilon gamma penalty) 2 candidate) ∧
AppliedModelingLib.proposition36bRRMUpdate epsilon
(AppliedModelingLib.proposition36bLowerEndpoint epsilon) ∈
AppliedModelingLib.proposition36bDomain epsilon ∧
∀ candidate ∈ AppliedModelingLib.proposition36bDomain epsilon,
AppliedModelingLib.measureDecoupledPerformativeRisk
(AppliedModelingLib.proposition36bMeasureModel epsilon gamma penalty)
(AppliedModelingLib.proposition36bLowerEndpoint epsilon)
(AppliedModelingLib.proposition36bRRMUpdate epsilon
(AppliedModelingLib.proposition36bLowerEndpoint epsilon)) ≤
AppliedModelingLib.measureDecoupledPerformativeRisk
(AppliedModelingLib.proposition36bMeasureModel epsilon gamma penalty)
(AppliedModelingLib.proposition36bLowerEndpoint epsilon) candidate) ∧
¬∃ (limit : ℝ),
Filter.Tendsto
(fun (iteration : ℕ) => (AppliedModelingLib.proposition36bRRMUpdate epsilon)^[iteration] 2)
Filter.atTop (nhds limit)
```

Direct retained dependencies

- [AppliedModelingLib.IsProposition36bLargePenalty](#)
- [AppliedModelingLib.measureDecoupledPerformativeRisk](#)
- [AppliedModelingLib.proposition36bDomain](#)
- [AppliedModelingLib.proposition36bLowerEndpoint](#)
- [AppliedModelingLib.proposition36bMeasureLoss](#)
- [AppliedModelingLib.proposition36bMeasureModel](#)
- [AppliedModelingLib.proposition36bRRMUpdate](#)
- [PZMH20PerformativePrediction.IsMeasureWassersteinSensitive](#)

PZMH20PerformativePrediction.strategicClassificationDataMap

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
{domain : Set Parameter} → (↑ domain → Feature → Feature) → ↑ domain → Feature × Label → Feature × Label

value:
fun {Parameter : Type u_1} {Feature : Type u_2} {Label : Type u_3} {domain : Set Parameter}
  (response : ↑ domain → Feature → Feature) (parameter : ↑ domain) (a : Feature × Label) =>
  (response parameter a.1, a.2)
```

PZMH20PerformativePrediction.strategicClassificationInducedLaw

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
[inst : MeasurableSpace Feature] →
[inst_1 : MeasurableSpace Label] →
{domain : Set Parameter} →
MeasureTheory.ProbabilityMeasure (Feature × Label) →
(response : ↑ domain → Feature → Feature) →
(∀ (parameter : ↑ domain), Measurable (response parameter)) →
↑ domain → MeasureTheory.ProbabilityMeasure (Feature × Label)

value:
fun {Parameter : Type u_1} {Feature : Type u_2} {Label : Type u_3} [MeasurableSpace Feature] [MeasurableSpace Label]
  {domain : Set Parameter} (population : MeasureTheory.ProbabilityMeasure (Feature × Label))
  (response : ↑ domain → Feature → Feature) (hresponse : ∀ (parameter : ↑ domain), Measurable (response parameter))
  (parameter : ↑ domain) =>
  population.map
  (PZMH20PerformativePrediction.strategicClassificationInducedLaw._proof_1 population response hresponse parameter)
```

Direct retained dependencies

- [PZMH20PerformativePrediction.strategicClassificationDataMap](#)

PZMH20PerformativePrediction.MeasureStrategicClassificationProfile

Declaration location: paper-local

Lean declaration body

```
field population:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
[inst : MeasurableSpace Feature] →
[inst_1 : MeasurableSpace Label] →
{domain : Set Parameter} →
PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain →
MeasureTheory.ProbabilityMeasure (Feature × Label)

field response:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
[inst : MeasurableSpace Feature] →
[inst_1 : MeasurableSpace Label] →
{domain : Set Parameter} →
PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain →
↑ domain → Feature → Feature

field utility:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
[inst : MeasurableSpace Feature] →
[inst_1 : MeasurableSpace Label] →
{domain : Set Parameter} →
PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain →
Feature → ↑ domain → ℝ

field cost:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
[inst : MeasurableSpace Feature] →
[inst_1 : MeasurableSpace Label] →
{domain : Set Parameter} →
PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain →
Feature → Feature → ℝ
```

```

field loss:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
[inst : MeasurableSpace Feature] →
[inst_1 : MeasurableSpace Label] →
{domain : Set Parameter} →
PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain →
Feature × Label →  $\mathbb{R}$ 

```

```

field response_measurable:
 $\forall$  {Parameter : Type u_1} {Feature : Type u_2} {Label : Type u_3} [inst : MeasurableSpace Feature]
[inst_1 : MeasurableSpace Label] {domain : Set Parameter}
(self : PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain)
(parameter :  $\uparrow$  domain), Measurable (self.response parameter)

```

```

field loss_integrable:
 $\forall$  {Parameter : Type u_1} {Feature : Type u_2} {Label : Type u_3} [inst : MeasurableSpace Feature]
[inst_1 : MeasurableSpace Label] {domain : Set Parameter}
(self : PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain)
(deployed evaluated :  $\uparrow$  domain),
MeasureTheory.Integrable (fun (datum : Feature × Label) => self.loss datum evaluated)
 $\uparrow$  (PZMH20PerformativePrediction.strategicClassificationInducedLaw self.population self.response
self.response_measurable deployed)

```

Direct retained dependencies

- [PZMH20PerformativePrediction.strategicClassificationInducedLaw](#)

PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.IsFollowerBestResponse

Declaration location: paper-local

Lean declaration body

```

type:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
[inst : MeasurableSpace Feature] →
[inst_1 : MeasurableSpace Label] →
{domain : Set Parameter} →
PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain → Prop

```

```

value:
fun {Parameter : Type u_1} {Feature : Type u_2} {Label : Type u_3} [MeasurableSpace Feature] [MeasurableSpace Label]
{domain : Set Parameter}
(profile : PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain) =>
 $\forall$  (parameter :  $\uparrow$  domain) (initial alternative : Feature),
profile.utility alternative parameter - profile.cost alternative initial  $\leq$ 
profile.utility (profile.response parameter initial) parameter -
profile.cost (profile.response parameter initial) initial

```

Direct retained dependencies

- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile](#)

PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.inducedLaw

Declaration location: paper-local

Lean declaration body

```

type:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
[inst : MeasurableSpace Feature] →
[inst_1 : MeasurableSpace Label] →
{domain : Set Parameter} →
PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain →
 $\uparrow$  domain → MeasureTheory.ProbabilityMeasure (Feature × Label)

```

```

value:
fun {Parameter : Type u_1} {Feature : Type u_2} {Label : Type u_3} [MeasurableSpace Feature] [MeasurableSpace Label]
{domain : Set Parameter}
(profile : PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain)
(parameter :  $\uparrow$  domain) =>
PZMH20PerformativePrediction.strategicClassificationInducedLaw profile.population profile.response
profile.response_measurable parameter

```

Direct retained dependencies

- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile](#)
- [PZMH20PerformativePrediction.strategicClassificationInducedLaw](#)

PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.performativeModelOn

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
[inst : MeasurableSpace Feature] →
[inst_1 : MeasurableSpace Label] →
{domain : Set Parameter} →
PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain →
AppliedModelingLib.MeasurePerformativeModelOn Parameter (Feature × Label) domain

value:
fun {Parameter : Type u_1} {Feature : Type u_2} {Label : Type u_3} [MeasurableSpace Feature] [MeasurableSpace Label]
{domain : Set Parameter}
(profile : PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain) =>
{ dataLaw := profile.inducedLaw, loss := profile.loss,
loss_integrable := fun (deployed evaluated : ↑ domain) => profile.loss_integrable deployed evaluated }
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModelOn](#)
- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile](#)
- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.inducedLaw](#)

PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.leaderRisk

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
[inst : MeasurableSpace Feature] →
[inst_1 : MeasurableSpace Label] →
{domain : Set Parameter} →
PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain →
↑ domain → ℝ

value:
fun {Parameter : Type u_1} {Feature : Type u_2} {Label : Type u_3} [MeasurableSpace Feature] [MeasurableSpace Label]
{domain : Set Parameter}
(profile : PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain)
(parameter : ↑ domain) =>
profile.performativeModelOn.performativeRisk parameter
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModelOn.performativeRisk](#)
- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile](#)
- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.performativeModelOn](#)

PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.IsLeaderOptimal

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
[inst : MeasurableSpace Feature] →
[inst_1 : MeasurableSpace Label] →
{domain : Set Parameter} →
PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain →
↑ domain → Prop

value:
fun {Parameter : Type u_1} {Feature : Type u_2} {Label : Type u_3} [MeasurableSpace Feature] [MeasurableSpace Label]
{domain : Set Parameter}
(profile : PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain)
(parameter : ↑ domain) =>
∀ (candidate : ↑ domain), profile.leaderRisk parameter ≤ profile.leaderRisk candidate
```

Direct retained dependencies

- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile](#)
- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.leaderRisk](#)

PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.IsStackelbergEquilibrium

Declaration location: paper-local

Lean declaration body

```
type:
{Parameter : Type u_1} →
{Feature : Type u_2} →
{Label : Type u_3} →
[inst : MeasurableSpace Feature] →
[inst_1 : MeasurableSpace Label] →
{domain : Set Parameter} →
PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain →
↑ domain → Prop

value:
fun {Parameter : Type u_1} {Feature : Type u_2} {Label : Type u_3} [MeasurableSpace Feature] [MeasurableSpace Label]
{domain : Set Parameter}
(profile : PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain)
(parameter : ↑ domain) =>
profile.IsFollowerBestResponse ∧ profile.IsLeaderOptimal parameter
```

Direct retained dependencies

- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile](#)
- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.IsFollowerBestResponse](#)
- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.IsLeaderOptimal](#)

Material library prerequisites

AppliedModelingLib.IsJointlySmoothGradient

Source locator: cited publication, lines 373-415

Verbatim paper-source connection

Theorem 3.10. Suppose that the loss $\ell(z; \theta)$ is β -jointly smooth (A1) and γ -strongly convex (A2), and that there exist $\alpha > 1, \mu > 0$ such that $\xi_{\alpha, \mu} = \sup_{\theta \in \Theta} \int_{\mathbb{R}^m} \ell(x) dD(\theta)$ is finite. If the distribution $D(\cdot)$ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then RGD with step size $\eta \leq \frac{2}{\beta + \gamma}$ satisfies the following:

(a) $\|k_{\theta} - \theta\|_2 \leq \epsilon(1.5\eta\beta + \beta)$ with probability $1 - p$, RERM satisfies, $\log \frac{1}{\delta} \|k_{\theta} - \theta\|_2 \leq \theta_{PS}^2$ for all $t > \frac{\log \frac{1}{\delta} \|k_{\theta} - \theta\|_2}{1 - 2\epsilon\beta}$.

(b) The iterates θ_t of RGD converge to a unique performance stable point θ_{PS} at a linear rate, $\log \frac{1}{\delta} \|k_{\theta} - \theta_{PS}\|_2 \leq \theta_{PS}^2$ for all $t > \frac{\log \frac{1}{\delta} \|k_{\theta} - \theta_{PS}\|_2}{1 - 2\epsilon\beta}$.

The conclusion of Theorem 3.8 is a strict generalization of a classical optimization result which considers a static objective, in which case the rate of contraction is $1 - \eta(\beta + \gamma)$ for a constant choice of step size $\eta \leq \frac{2}{\beta + \gamma}$.

(see for example Theorem 2.1.15 in Nesterov (2013) or Lemma 3.7 in Hardt et al. (2016b)). Our rate exactly matches this standard result in the case that $\epsilon = 0$. The proof of Theorem 3.8 can be found in Appendix E.3.

Proof sketch. The basic idea behind these results is the following. While $\|k_{\theta} - \theta_{PS}\|_2 > \delta$, the sample size nt is sufficiently large to ensure a behavior similar to that on a

Formalized review target The archival source above is not asserted equivalent to this formalized target.

The REGD premise bundle for Theorem 3.10(b) specifies data dimension greater than two, the concrete capped count and inverse-square confidence schedules, a shared bounded exponential moment and tail envelope, measurable adaptive sampling and bad events, joint-gradient smoothness and pointwise strong convexity with Wasserstein sensitivity, a convex domain with a variational nonexpansive projection, gradient integrability and pointwise loss differentiation, a checked empirical-gradient trajectory and a stable point, and the step-size, numerical W1 budget, contraction slack, absorbing-radius, and initial-distance conditions. The trajectory theorem derives population contraction and a finite entry iteration; neither a caller-supplied small-deviation gate nor a supplied burn-in bound is a premise. The model and expected-loss integrability are defined for every ambient parameter. Joint-gradient smoothness and pointwise strong convexity quantify over all ambient parameters and data points; Wasserstein sensitivity, gradient integrability, and loss differentiation quantify over all ambient parameters. These global hypotheses are stronger in scope than the source assumptions on the parameter domain and the union of its induced supports; their necessity for the source conclusion is unresolved.

Archival source anchor: cited publication, lines 373-415

Lean-expanded library semantic target

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[Inst : NormedAddCommGroup Parameter] →
[InnerProductSpace ℝ Parameter] → [MetricSpace Data] → (Data → Parameter → Parameter) → NNReal → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [NormedAddCommGroup Parameter] [InnerProductSpace ℝ Parameter]
[MetricSpace Data] (gradient : Data → Parameter → Parameter) (smoothness : NNReal) =>
(∀ (datum : Data) (first second : Parameter),
  ||gradient datum first - gradient datum second|| ≤ ↑smoothness * ||first - second||) ∧
  AppliedModelingLib.IsDataGradientLipschitz gradient smoothness
```

Exact Lean library declaration

```
/-- The two-coordinate joint-smoothness condition used in the PZ paper. -/
def IsJointlySmoothGradient {Parameter Data : Type*}
[NormedAddCommGroup Parameter] [InnerProductSpace ℝ Parameter] [MetricSpace Data]
(gradient : Data → Parameter → Parameter) (smoothness : NNReal) : Prop :=
(∀ (datum : Data) (first second : Parameter),
  ||gradient datum first - gradient datum second|| ≤
    (smoothness : ℝ) * ||first - second||) ∧
  IsDataGradientLipschitz gradient smoothness
```

Direct retained dependencies

- [AppliedModelingLib.IsDataGradientLipschitz](#)

Recorded source-to-library screening **Verdict:** matches_approved_corrected_target

Reason: The expanded definition is exactly the two A1 Lipschitz inequalities for the supplied gradient: uniform smoothness in the parameter coordinate and, through IsDataGradientLipschitz, in the data coordinate. Its graph-bound uses separately require HasGradientAt to identify this field with the loss gradient, and the definition's global parameter/data quantifiers are the stronger scope expressly disclosed in the corrected Theorem 3.10 target.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

-/
 structure MeasurePerformativeModel (Parameter Data : Type*) [MeasurableSpace Data] where
 dataLaw : Parameter → ProbabilityMeasure Data
 loss : Data → Parameter → ℝ
 loss_integrable : ∀ distributionParameter evaluatedParameter,
 Integrable (fun datum => loss datum evaluatedParameter)
 (dataLaw distributionParameter : Measure Data)

Recorded source-to-library screening Verdict: matches_approved_corrected_target

Reason: The expanded structure consists of a probability data law for every deployed parameter, a loss for every datum and evaluated parameter, and integrability of that loss under every deployed/evaluated pair. Those are exactly the globally defined model and expected-loss integrability conditions disclosed in the corrected Theorem 3.10 target and assumed by every displayed material use.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.IsMeasurePointwiseGradientStronglyConvex

Source locator: cited publication, lines 373-415

Verbatim paper-source connection

Theorem 3.10. Suppose that the loss $\ell(z; \theta)$ is β -jointly smooth (A1) and γ -strongly convex (A2), and that there exist $\alpha > 1, \mu > 0$ such that $\xi_{\alpha, \mu} = \mathbb{E} \mu |x| \mathbb{E} D(\theta)$ is finite. If the distribution μ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then RGD with step size $\eta \leq \beta + \gamma$ satisfies the following:

(a) $\|k_{\theta} - \theta_{PS}\|_2^2 \leq \epsilon(1.5\eta\beta^2 + \beta) \log \frac{1}{\delta}$ for all $t > \frac{\log \frac{1}{\delta} \log \frac{1}{\epsilon\delta}}{\eta(\beta + \gamma - \epsilon(1.5\eta\beta^2 + \beta))}$.

(b) The iterates θ_t of RGD converge to a unique performance stable point θ_{PS} at a linear rate, $\|k_{\theta_t} - \theta_{PS}\|_2^2 \leq \epsilon(3\eta\beta^2 + 2\beta) \log \frac{1}{\delta}$ for all $t > \frac{\log \frac{1}{\delta} \log \frac{1}{\epsilon\delta}}{\eta(\beta + \gamma - \epsilon(3\eta\beta^2 + 2\beta))}$.

The conclusion of Theorem 3.8 is a strict generalization of a classical optimization result which considers a static objective, in which case the rate of contraction is $1 - \eta(\beta + \gamma)$ for a constant choice of step size $\eta \leq \beta + \gamma$.

(see for example Theorem 2.1.15 in Nesterov (2013) or Lemma 3.7 in Hardt et al. (2016b)). Our rate exactly matches this standard result in the case that $\epsilon = 0$. The proof of Theorem 3.8 can be found in Appendix E.3.

Proof sketch. The basic idea behind these results is the following. While $\|k_{\theta_t} - \theta_{PS}\|_2^2 > \delta$, the sample size nt is sufficiently large to ensure a behavior similar to that on a

sider running RERM or RGD with $nt = O(\epsilon\delta)m \log p$ samples at time t . (a) If the map $D(\cdot)$ is ϵ -sensitive with $\epsilon < 2\beta$, then with probability $1 - p$, RERM satisfies, $\log \frac{1}{\delta} \log \frac{1}{\epsilon\delta} \log \frac{1}{\epsilon\delta}$ $\|k_{\theta_t} - \theta_{PS}\|_2^2 \leq \epsilon(1.5\eta\beta^2 + \beta) \log \frac{1}{\delta}$ for all $t > \frac{\log \frac{1}{\delta} \log \frac{1}{\epsilon\delta}}{\eta(\beta + \gamma - \epsilon(1.5\eta\beta^2 + \beta))}$.

(b) If the map $D(\cdot)$ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then with probability $1 - p$, REGD with satisfies, $\log \frac{1}{\delta} \log \frac{1}{\epsilon\delta} \log \frac{1}{\epsilon\delta}$ $\|k_{\theta_t} - \theta_{PS}\|_2^2 \leq \epsilon(3\eta\beta^2 + 2\beta) \log \frac{1}{\delta}$ for all $t > \frac{\log \frac{1}{\delta} \log \frac{1}{\epsilon\delta}}{\eta(\beta + \gamma - \epsilon(3\eta\beta^2 + 2\beta))}$.

The conclusion of Theorem 3.8 is a strict generalization of a classical optimization result which considers a static objective, in which case the rate of contraction is $1 - \eta(\beta + \gamma)$ for a constant choice of step size $\eta \leq \beta + \gamma$.

(see for example Theorem 2.1.15 in Nesterov (2013) or Lemma 3.7 in Hardt et al. (2016b)). Our rate exactly matches this standard result in the case that $\epsilon = 0$. The proof of Theorem 3.8 can be found in Appendix E.3.

Proof sketch. The basic idea behind these results is the following. While $\|k_{\theta_t} - \theta_{PS}\|_2^2 > \delta$, the sample size nt is sufficiently large to ensure a behavior similar to that on a

Formalized review target

The REGD premise bundle for Theorem 3.10(b) specifies data dimension greater than two, the concrete capped count and inverse-square confidence schedules, a shared bounded exponential moment and tail envelope, measurable adaptive sampling and bad events, joint-gradient smoothness and pointwise strong convexity with Wasserstein sensitivity, a convex domain with a variational nonexpansive projection, gradient integrability and pointwise loss differentiation, a checked empirical-gradient trajectory and a stable point, and the step-size, numerical W1 budget, contraction slack, absorbing-radius, and initial-distance conditions. The trajectory theorem derives population contraction and a finite entry iteration; neither a caller-supplied small-deviation gate nor a supplied burn-in bound is a premise. The model and expected-loss integrability are defined for every ambient parameter. Joint-gradient smoothness and pointwise strong convexity quantify over all ambient parameters and data points; Wasserstein sensitivity, gradient integrability, and loss differentiation quantify over all ambient parameters. These global hypotheses are stronger in scope than the source assumptions on the parameter domain and the union of its induced supports; their necessity for the source conclusion is unresolved.

Archival source anchor: cited publication, lines 373-415

Lean-expanded library semantic target

```
type:
  {Parameter : Type u_1} →
  {Data : Type u_2} →
  [inst : MeasurableSpace Data] →
  [inst_1 : NormedAddCommGroup Parameter] →
  [InnerProductSpace ℝ Parameter] →
  AppliedModelingLib.MeasurePerformativeModel Parameter Data → (Data → Parameter → Parameter) → ℝ → Prop

value:
  fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data] [NormedAddCommGroup Parameter]
    [InnerProductSpace ℝ Parameter] (model : AppliedModelingLib.MeasurePerformativeModel Parameter Data)
      (gradient : Data → Parameter → Parameter) (modulus : ℝ) =>
    ∀ (datum : Data) (center candidate : Parameter),
      model.loss datum candidate ≥
      model.loss datum center + inner ℝ (gradient datum center) (candidate - center) +
      modulus / 2 * ‖candidate - center‖ ^ 2
```

Exact Lean library declaration

/--
The pointwise form of the source's A2 strong-convexity assumption for an arbitrary probability data law. Together with integrability of the gradient, it implies strong monotonicity of the frozen expected gradient below.
-/

```

def IsMeasurePointwiseGradientStronglyConvex {Parameter Data : Type*}
  [MeasurableSpace Data] [NormedAddCommGroup Parameter] [InnerProductSpace ℝ Parameter]
  (model : MeasurePerformativeModel Parameter Data)
  (gradient : Data → Parameter → Parameter) (modulus : ℝ) : Prop :=
  ∀ datum center candidate,
  model.loss datum candidate ≥
  model.loss datum center +
  ⟨gradient datum center, candidate - center⟩_ℝ +
  modulus / 2 * ‖candidate - center‖ ^ 2

```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)

Recorded source-to-library screening Verdict: matches_approved corrected_target

Reason: The expanded definition states the A2 lower bound with modulus / 2 times squared parameter distance for every datum, center, and candidate, using the supplied gradient at the center. The graph-bound carriers separately identify that gradient with HasGradientAt; the all-ambient quantifier scope is exactly the stronger scope disclosed by the corrected target.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.IsMeasureWassersteinSensitive

Source locator: cited publication, lines 373-415

Verbatim paper-source connection

Theorem 3.10. Suppose that the loss $\ell(z; \theta)$ is β -jointly smooth (A1) and γ -strongly convex (A2), and that there exist $\alpha > 1, \mu > 0$ such that $\xi_{\alpha, \mu} = \mathbb{R}^m \mathbb{E}[\|x\|] \mathbb{E}[\|D\ell(\theta)\|]$ is finite. If the distribution μ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then RGD with step size $\eta \leq \beta + \gamma$ satisfies the following:

(a) If the map $D(\cdot)$ is ϵ -sensitive with $\epsilon < 2\beta$, then with probability $1 - p$, RERM satisfies,

$$\|k\theta_t - \theta_{PS}\|_2 \leq \epsilon(1.5\eta\beta^2 + \beta) \quad \text{for all } t > \frac{\log 16 \|k\theta_0 - \theta_{PS}\|_2}{1 - 2\epsilon\beta}$$

(b) The iterates θ_t of RGD converge to a unique performance stable point θ_{PS} at a linear rate,

$$\|k\theta_t - \theta_{PS}\|_2 \leq \epsilon(1.5\eta\beta^2 + \beta) \quad \text{for all } t > \frac{\log 16 \|k\theta_0 - \theta_{PS}\|_2}{1 - 2\epsilon\beta}$$

The conclusion of Theorem 3.8 is a strict generalization of a classical optimization result which considers a static objective, in which case the rate of contraction is $1 - \eta(\beta + \gamma)$ for a constant choice of step size $\eta \leq \beta + \gamma$.

(see for example Theorem 2.1.15 in Nesterov (2013) or Lemma 3.7 in Hardt et al. (2016b)). Our rate exactly matches this standard result in the case that $\epsilon = 0$. The proof of Theorem 3.8 can be found in Appendix E.3.

Proof sketch. The basic idea behind these results is the following. While $\|k\theta_t - \theta_{PS}\|_2 > \delta$, the sample size nt is sufficiently large to ensure a behavior similar to that on a

Formalized review target

The REGD premise bundle for Theorem 3.10(b) specifies data dimension greater than two, the concrete capped count and inverse-square confidence schedules, a shared bounded exponential moment and tail envelope, measurable adaptive sampling and bad events, joint-gradient smoothness and pointwise strong convexity with Wasserstein sensitivity, a convex domain with a variational nonexpansive projection, gradient integrability and pointwise loss differentiation, a checked empirical-gradient trajectory and a stable point, and the step-size, numerical W_1 budget, contraction slack, absorbing-radius, and initial-distance conditions. The trajectory theorem derives population contraction and a finite entry iteration; neither a caller-supplied small-deviation gate nor a supplied burn-in bound is a premise. The model and expected-loss integrability are defined for every ambient parameter. Joint-gradient smoothness and pointwise strong convexity quantify over all ambient parameters and data points; Wasserstein sensitivity, gradient integrability, and loss differentiation quantify over all ambient parameters. These global hypotheses are stronger in scope than the source assumptions on the parameter domain and the union of its induced supports; their necessity for the source conclusion is unresolved.

Archival source anchor: cited publication, lines 373-415

Lean-expanded library semantic target

```

type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[MetricSpace Parameter] →
[inst : MeasurableSpace Data] →
[MetricSpace Data] → AppliedModelingLib.MeasurePerformativeModel Parameter Data → ℝ → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MetricSpace Parameter] [MeasurableSpace Data] [MetricSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (sensitivity : ℝ) =>
  ∀ (first second : Parameter),
  ∃ (hfinite :
    (AppliedModelingLib.ProbabilityCoupling.expectedDistanceCosts (model.dataLaw first)
      (model.dataLaw second)).Nonempty),
    AppliedModelingLib.ProbabilityCoupling.wassersteinOneReal (model.dataLaw first) (model.dataLaw second) hfinite ≤
      sensitivity * dist first second

```

Exact Lean library declaration

/--
The source paper's general-distribution Wasserstein-1 sensitivity condition.
For every pair of deployed parameters, the induced laws have a finite
expected-distance coupling and their real W_1 cost is bounded by the displayed
parameter-distance modulus.

```

-/
def IsMeasureWassersteinSensitive {Parameter Data : Type*}
  [MetricSpace Parameter] [MeasurableSpace Data] [MetricSpace Data]
  (model : MeasurePerformativeModel Parameter Data) (sensitivity : ℝ) : Prop :=
  ∀ first second,
  ∃ hfinite : (ProbabilityCoupling.expectedDistanceCosts
    (model.dataLaw first) (model.dataLaw second)).Nonempty,
  ProbabilityCoupling.wassersteinOneReal (model.dataLaw first) (model.dataLaw second) hfinite ≤
  sensitivity * dist first second

```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.ProbabilityCoupling.expectedDistanceCosts](#)
- [AppliedModelingLib.ProbabilityCoupling.wassersteinOneReal](#)

Recorded source-to-library screening Verdict: matches_approved_corrected_target

Reason: For every pair of ambient parameters, the expanded definition supplies a finite expected-distance coupling and bounds the resulting real W1 infimum by sensitivity times parameter distance. This is the finite-first-moment Wasserstein-1 reading and global scope expressly disclosed in the corrected Theorem 3.10 premises and used unchanged by the carriers.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source locator: cited publication, lines 151-172

Verbatim paper-source connection

Recently, there has been increased interest within the algorithmic fairness community in classification dynamics. See, for example, Liu et al. (2018), Hu & Chen (2018), and Hashimoto et al. (2018). The latter work considers repeated risk minimization, but from the perspective of what it does to a measure of disparity between groups.

Causal inference. The reader familiar with causality can think of $D(\theta)$ as the interventional distribution over instances Z resulting from a do-intervention that sets the

model parameters to θ in some underlying causal graph.

Importantly, this mapping $D(\cdot)$ remains fixed and does not change over time or by intervention: deploying the same classifier at two different points in time must induce the same distribution over observations Z . While causal in-

ference focuses on estimating properties of interventional distributions such as treatment effects (Pearl, 2009; Imbens & Rubin, 2015), our focus is on a new stability notion and

We contrast this with the performative optimum. As introduced previously, in settings where predictions support decisions, the manifested distribution over features and outcomes is in part determined by the deployed classifier. Instead of considering a fixed distribution D , each classifier f_θ induces a potentially different distribution $D(\theta)$ over

instances z . A predictor must therefore be evaluated with regard to the expected loss over the distribution $D(\theta)$ it induces: its performative risk.

Definition 2.1 (performative optimality and risk). A classifier f_θ is performatively optimal if the following relationship holds:

$$\theta_{\text{PO}} = \arg \min_{\theta} \mathbb{E}_{Z \sim D(\theta)} \ell(Z; \theta).$$

We define $\text{PR}(\theta) = \mathbb{E}_{Z \sim D(\theta)} \ell(Z; \theta)$ as the performative risk; then, $\theta_{\text{PO}} = \arg \min_{\theta} \text{PR}(\theta)$.

[Next verbatim source excerpt]

Throughout our presentation, we focus on predictive models that are parametrized by a vector $\theta \in \Theta$, where the parameter space $\Theta \subseteq \mathbb{R}^d$ is a closed, convex set. We use capital letters to denote random variables and their lowercase counterparts to denote realizations of these variables. We consider instances $z = (x, y)$ defined as feature, outcome pairs, where $x \in \mathbb{R}^{m-1}$ and $y \in \mathbb{R}$. Whenever we define a

Here, ϵ represents the performative aspect of the model. If $\epsilon = 0$, outcomes are independent of the assigned scores and the problem reduces to standard supervised learning where the optimal predictor is the conditional expectation $f_{\text{SL}}(x) = \mathbb{E}[Y | X = x] = \mu_x$, with $\text{SL} = \mu$. In the performative setting with $\epsilon \neq 0$, the optimal prediction θ_{PO} balances between its predictive accuracy as well

[Next verbatim source excerpt]

Definition 2.3 (performative stability and decoupled risk). A classifier f_θ is performatively stable if the following relationship holds:

$$\theta_{\text{PS}} = \arg \min_{\theta} \mathbb{E}_{Z \sim D(\theta_{\text{PS}})} \ell(Z; \theta).$$

We define $\text{DPR}(\theta, \theta_0) = \mathbb{E}_{Z \sim D(\theta)} \ell(Z; \theta_0)$ as the decoupled performative risk; then, $\theta_{\text{PS}} = \arg \min_{\theta} \text{DPR}(\theta_{\text{PS}}, \theta)$.

Points Having introduced our framework for performative prediction, we now address some of the basic questions that arise in this setting and examine the behavior of common machine learning practices, such as retraining, through the lens of performativity. We begin by analyzing the behavior of

these procedures when they operate at a population level and then extend our analysis to finite samples.

Lean declaration type (metadata)

```
field dataLaw:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
{domain : Set Parameter} →
AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain →
↑ domain → MeasureTheory.ProbabilityMeasure Data

field loss:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
{domain : Set Parameter} →
AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain → Data → ↑ domain → ℝ

field loss_integrable:
∀ {Parameter : Type u_1} {Data : Type u_2} [inst : MeasurableSpace Data] {domain : Set Parameter}
(self : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain)
(distributionParameter evaluatedParameter : ↑ domain),
MeasureTheory.Integrable (fun datum : Data => self.loss datum evaluatedParameter)
↑ (self.dataLaw distributionParameter)
```

Exact Lean library declaration

```
/-- A performative model defined only on a designated parameter domain. -/
structure MeasurePerformativeModelOn (Parameter Data : Type*) [MeasurableSpace Data]
(domain : Set Parameter) where
dataLaw : domain → ProbabilityMeasure Data
loss : Data → domain → ℝ
loss_integrable : ∀ distributionParameter evaluatedParameter,
Integrable (fun datum => loss datum evaluatedParameter)
(dataLaw distributionParameter : Measure Data)
```

Recorded source-to-library screening Verdict: matches

Reason: Compared domain relative performative model at cited publication:151-172,188-194,219-228 with the three record-field displays and every listed risk/predicate/use display. dataLaw is a fixed probability-law map on Theta, and loss has a datum and a feasible parameter as its arguments. The independently quantified distributionParameter and

evaluatedParameter in loss integrable cover exactly the two parameters of the source DPR expectation, with PR as its diagonal. These fields give the source risks as real expectations without requiring a distribution or loss value outside Theta. The displayed paper use supplies closedness and convexity of the domain, a positive A2 modulus, the uniform data-Lipschitz and nonnegative sensitivity hypotheses, and the optimal/stable domain members. Its A2 prerequisite binds the loss derivative within this same domain; the explicit zero extension is read only within the domain and therefore adds no off-domain regularity condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.MeasurePerformativeModelOn.IsDataLossLipschitz

Source locator: cited publication, lines 468-480

Verbatim paper-source connection

tee that the resulting classifier has low performative risk. In particular, our next result demonstrates that if the loss function $\ell(z; \theta)$ is Lipschitz in z and γ -strongly convex, then all performatively stable points and performative optima lie in a small neighborhood around each other. Moreover, the theorem holds for cases where performative optima and performatively stable points are not necessarily unique.

Theorem 4.3. Suppose that the loss $\ell(z; \theta)$ is L_z -Lipschitz in z , γ -strongly convex (A2), and that the distribution map

¹ In particular, they are guaranteed to exist over the extended real line, i.e. we allow $\theta \in (\mathbb{R} \cup \{\pm\infty\})^d$.

said to have a Stackelberg structure since agents adapt their features only after the bank has deployed their classifier.

The optimal strategy for the institution in a strategic classification setting is to deploy the solution corresponding to the Stackelberg equilibrium, defined as the classifier θ^* which achieves minimal loss over the induced distribution $D(\theta)$

in which agents have strategically adapted their features in response to θ^* . In fact, we see that this equilibrium notion exactly matches our definition of performative optimality:

θ^* is a Stackelberg equilibrium $\Leftrightarrow \theta^* \in \arg \min \text{PR}(\theta)$.

Lean-expanded library semantic target

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
[MetricSpace Data] →
{domain : Set Parameter} → AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain → NNReal → Prop
```

```
value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data] [MetricSpace Data] {domain : Set Parameter}
(model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain) (constant : NNReal) =>
  ∀ (parameter : ↑ domain), LipschitzWith constant fun (datum : Data) => model.loss datum parameter
```

Exact Lean library declaration

```
-- Data-coordinate loss Lipschitzness on a model's parameter domain. -/
def IsDataLossLipschitz
  {Parameter Data : Type*} [MeasurableSpace Data] [MetricSpace Data]
  {domain : Set Parameter} (model : MeasurePerformativeModelOn Parameter Data domain)
  (constant : NNReal) : Prop :=
  ∀ parameter, LipschitzWith constant (fun datum => model.loss datum parameter)
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModelOn](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared theorem 4.3 data lipschitz premise at cited publication:468-480 with the expanded predicate, model fields, and theorem-use display. For every parameter in Theta, the same constant bounds the Lipschitz modulus of datum -> loss datum parameter. Thus the target is the uniform data-coordinate inequality $|\text{loss}(z, \theta) - \text{loss}(z', \theta)| \leq L_z \text{dist}(z, z')$, as required by the source premise. The NNReal parameter permits $L_z=0$ and supplies its nonnegativity; no parameter-coordinate Lipschitz condition is introduced. The use retains this same constant and the domain membership of all parameters. Model integrability provides the real expectations required by the displayed risk comparisons.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.MeasurePerformativeModelOn.decoupledPerformativeRisk

Source locator: cited publication, lines 219-228

Verbatim paper-source connection

Definition 2.3 (performative stability and decoupled risk). A classifier $f_{\theta PS}$ is performatively stable if the following relationship holds:

$$\theta PS = \arg \min_{\theta} E_{Z \sim D(\theta PS)} \ell(Z; \theta).$$

We define $DPR(\theta, \theta_0) = E_{Z \sim D(\theta)} \ell(Z; \theta_0)$ as the decoupled performative risk; then, $\theta PS = \arg \min_{\theta} DPR(\theta PS, \theta)$.

Points
Having introduced our framework for performative prediction, we now address some of the basic questions that arise in this setting and examine the behavior of common machine learning practices, such as retraining, through the lens of performativity. We begin by analyzing the behavior of these procedures when they operate at a population level and then extend our analysis to finite samples.

Lean-expanded library semantic target

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
{domain : Set Parameter} →
AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain → ↑ domain → ↑ domain → ℝ

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data] {domain : Set Parameter}
(model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain) (deployed evaluated : ↑ domain) =>
f (datum : Data), model.loss datum evaluated ∂ ↑ (model.dataLaw deployed)
```

Exact Lean library declaration

```
/-- Decoupled performative risk for a model whose parameter carrier is a set. -/
noncomputable def decoupledPerformativeRisk
{Parameter Data : Type*} [MeasurableSpace Data] {domain : Set Parameter}
(model : MeasurePerformativeModelOn Parameter Data domain)
(deployed evaluated : domain) : ℝ :=
f datum, model.loss datum evaluated ∂ (model.dataLaw deployed : Measure Data)
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModelOn](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared definition 2.3 performative stability and decoupled risk at cited publication:219-228 with the expanded integral and every listed model, risk, optimality, stability, and theorem-use display. The deployed argument selects dataLaw, while the evaluated argument selects the loss parameter, giving $DPR(\theta, \theta') = E$ under $D(\theta)$ of $\ell(Z, \theta')$. The model ensures integrability for these two independently chosen feasible arguments. The displayed diagonal call gives PR; the stability calls keep the law argument fixed; and optimality compares diagonals. All these uses preserve the source argument order, candidate quantifier, and feasible parameter set.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.MeasurePerformativeModelOn.performativeRisk

Source locator: cited publication, lines 161-172

Verbatim paper-source connection

Definition 2.1 (performative optimality and risk). A classifier f is performatively optimal if the following relationship holds:

Importantly, this mapping $D(\cdot)$ remains fixed and does not change over time or by intervention: deploying the same classifier at two different points in time must induce the same distribution over observations Z . While causal inference focuses on estimating properties of interventional distributions such as treatment effects (Pearl, 2009; Imbens & Rubin, 2015), our focus is on a new stability notion and

$$\theta_{PO} = \arg \min_{\theta} \mathbb{E}_{Z \sim D(\theta)} \ell(Z; \theta).$$

We define $PR(\theta) = \mathbb{E}_{Z \sim D(\theta)} \ell(Z; \theta)$ as the performative risk; then, $\theta_{PO} = \arg \min_{\theta} PR(\theta)$.

Lean-expanded library semantic target

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
{domain : Set Parameter} → AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain → ↑domain → ℝ

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data] {domain : Set Parameter}
(model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain) (parameter : ↑domain) =>
model.decoupledPerformativeRisk parameter parameter
```

Exact Lean library declaration

```
-- Performative risk for a model whose parameter carrier is a set. -/
noncomputable def performativeRisk
{Parameter Data : Type*} [MeasurableSpace Data] {domain : Set Parameter}
(model : MeasurePerformativeModelOn Parameter Data domain)
(parameter : domain) : ℝ :=
model.decoupledPerformativeRisk parameter parameter
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModelOn](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.decoupledPerformativeRisk](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared definition_2_1_performative_optimality_and_risk at cited publication:161-172 with the expanded definition and every listed decoupled-risk, model, optimality, and theorem-use display. The same feasible parameter is passed as both arguments of decoupledPerformativeRisk, so $PR(\theta)$ is exactly $\mathbb{E}$ under $D(\theta)$ of $\ell(Z, \theta)$. In optimality comparisons, the induced law and evaluated parameter therefore change together, preserving the source objective. The model supplies the needed diagonal integrability; neither this definition nor the displayed use introduces a unique minimizer or evaluates a parameter outside Θ .

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.MeasurePerformativeModelOn.IsPerformativelyOptimal

Source locator: cited publication, lines 161-172

Verbatim paper-source connection

Definition 2.1 (performative optimality and risk). A classifier f is performatively optimal if the following relationship holds:

Importantly, this mapping $D(\cdot)$ remains fixed and does not change over time or by intervention: deploying the same classifier at two different points in time must induce the same distribution over observations Z . While causal inference focuses on estimating properties of interventional distributions such as treatment effects (Pearl, 2009; Imbens & Rubin, 2015), our focus is on a new stability notion and

$$\theta_{PO} = \arg \min_{\theta} E_{Z \sim D(\theta)} \ell(Z; \theta).$$

We define $PR(\theta) = E_{Z \sim D(\theta)} \ell(Z; \theta)$ as the performative risk; then, $\theta_{PO} = \arg \min_{\theta} PR(\theta)$.

Lean-expanded library semantic target

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
{domain : Set Parameter} → AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain → ↑ domain → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data] {domain : Set Parameter}
(model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain) (optimal : ↑ domain) =>
∀ (candidate : ↑ domain), model.performativeRisk optimal ≤ model.performativeRisk candidate
```

Exact Lean library declaration

```
-- Performative optimality for a model defined only on its parameter domain. -/
def IsPerformativelyOptimal
{Parameter Data : Type*} [MeasurableSpace Data] {domain : Set Parameter}
(model : MeasurePerformativeModelOn Parameter Data domain)
(optimal : domain) : Prop :=
∀ candidate, model.performativeRisk optimal ≤ model.performativeRisk candidate
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModelOn](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.performativeRisk](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared definition 2.1_performative_optimality_and_risk at cited publication:161-172 with the expanded predicate, both risk definitions, the model fields, and the listed theorem use. Expanding the two risk definitions gives $E_{\text{under } D(\text{optimal})} \text{loss}(Z, \text{optimal}) \leq E_{\text{under } D(\text{candidate})} \text{loss}(Z, \text{candidate})$ for every candidate in Θ . Both the law and evaluation argument vary together on the candidate side, exactly the source argmin of PR. The supplied optimal member satisfies all comparisons; neither this predicate nor its paper use asserts uniqueness or constructs an optimum. The model covers the integrability of every diagonal expectation, and all minimization arguments stay inside the source domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.MeasurePerformativeModelOn.IsPerformativelyStable

Source locator: cited publication, lines 219-228

Verbatim paper-source connection

Definition 2.3 (performative stability and decoupled risk). A classifier θ_{PS} is performatively stable if the following relationship holds:

$$\theta_{PS} = \arg \min_{\theta} E_{Z \sim D(\theta)} \ell(Z; \theta).$$

We define $DPR(\theta, \theta_0) = E_{Z \sim D(\theta)} \ell(Z; \theta_0)$ as the decoupled performative risk; then, $\theta_{PS} = \arg \min_{\theta} DPR(\theta_{PS}, \theta)$.

Points
Having introduced our framework for performative prediction, we now address some of the basic questions that arise in this setting and examine the behavior of common machine learning practices, such as retraining, through the lens of performativity. We begin by analyzing the behavior of these procedures when they operate at a population level and then extend our analysis to finite samples.

Lean-expanded library semantic target

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
{domain : Set Parameter} → AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain → ↑domain → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data] {domain : Set Parameter}
(model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain) (stable : ↑domain) =>
  ∀ (candidate : ↑domain),
    model.decoupledPerformativeRisk stable stable ≤ model.decoupledPerformativeRisk stable candidate
```

Exact Lean library declaration

```
/- Performative stability for a model defined only on its parameter domain. -/
def IsPerformativelyStable
  {Parameter Data : Type*} [MeasurableSpace Data] {domain : Set Parameter}
  (model : MeasurePerformativeModelOn Parameter Data domain)
  (stable : domain) : Prop :=
  ∀ candidate, model.decoupledPerformativeRisk stable stable ≤
    model.decoupledPerformativeRisk stable candidate
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModelOn](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.decoupledPerformativeRisk](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared definition 2_3_performative_stability_and_decoupled_risk at cited publication:219-228 with the expanded predicate and all listed model, decoupled-risk, and theorem-use displays. The resulting inequality is E under $D(\text{stable})$ of $\text{loss}(Z, \text{stable}) \leq E$ under $D(\text{stable})$ of $\text{loss}(Z, \text{candidate})$ for every candidate in Θ . The induced distribution is fixed on both sides, and only the evaluated classifier changes, exactly the source $\arg \min_{\theta} \text{DPR}(\theta_{PS}, \theta)$. The supplied stable domain member need not be unique, and no existence assertion is substituted for the predicate. Universal cross-law integrability in the model covers these fixed-law real expectations.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.MeasurePerformativeModelOn.IsPointwiseGradientStronglyConvex

Source locator: cited publication, lines 270-282

Verbatim paper-source connection

(A1) (joint smoothness) A loss function $\ell(z; \theta)$ is β -jointly smooth if $\forall \theta, \theta_0 \in \Theta$ and $z, z_0 \in Z$:
The main message of this theorem is that in performative prediction, if the loss function is sufficiently “nice” and the distribution map is sufficiently (in)sensitive, then one need only retrain a classifier a small number of times before it converges to a unique stable point. Here, we provide a proof sketch illustrating the main ideas behind the theorem and defer the full proof to Appendix E.1.

(A2) (strong convexity) A loss function $\ell(z; \theta)$ is γ -strongly convex if $\forall \theta, \theta_0 \in \Theta$ and $z \in Z$:
Proof Sketch. Fix $\theta, \theta_0 \in \Theta$. Let $f(\phi) = \mathbb{E} Z \sim D(\theta) \ell(Z; \phi)$
 $\ell(z; \theta) > \ell(z; \theta_0) + r\theta \ell(z; \theta_0) > (\theta - \theta_0) + k\theta - \theta_0 k^2$.
and $f_0(\phi) = \mathbb{E} Z \sim D(\theta_0) \ell(Z; \phi)$. By applying standard prop-

Lean-expanded library semantic target

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
[inst_1 : NormedAddCommGroup Parameter] →
[InnerProductSpace ℝ Parameter] →
{domain : Set Parameter} →
AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain →
(Data → ↑domain → Parameter) → ℝ → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data] [NormedAddCommGroup Parameter]
[InnerProductSpace ℝ Parameter] {domain : Set Parameter}
(model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain)
(gradient : Data → ↑domain → Parameter) (modulus : ℝ) =>
(∀ (datum : Data) (center : ↑domain),
HasFDerivWithinAt
(fun (parameter : Parameter) =>
if hparameter : parameter ∈ domain then model.loss datum (parameter, hparameter) else 0)
(innerSL ℝ (gradient datum center)) domain (center : ↑domain)) ∧
∀ (datum : Data) (center candidate : ↑domain),
model.loss datum candidate ≥
model.loss datum center + inner ℝ (gradient datum center) (↑candidate - ↑center) +
modulus / 2 * ||↑candidate - ↑center|| ^ 2
```

Exact Lean library declaration

```
-- A2 uses the actual loss derivative on the feasible parameter domain.
The zero extension is read only along the domain; no off-domain loss value or
regularity is required. The lower-model inequality uses this same derivative. -/
def IsPointwiseGradientStronglyConvex
{Parameter Data : Type*} [MeasurableSpace Data]
[NormedAddCommGroup Parameter] [InnerProductSpace ℝ Parameter]
{domain : Set Parameter} (model : MeasurePerformativeModelOn Parameter Data domain)
(gradient : Data → domain → Parameter) (modulus : ℝ) : Prop := by
classical
exact
(∀ datum (center : domain),
HasFDerivWithinAt
(fun parameter => if hparameter : parameter ∈ domain then
model.loss datum (parameter, hparameter) else 0)
(innerSL ℝ (gradient datum center)) domain (center : Parameter)) ∧
∀ datum center candidate,
model.loss datum candidate ≥
model.loss datum center +
⟨gradient datum center, (candidate : Parameter) - (center : Parameter)⟩_ℝ +
modulus / 2 * ||(candidate : Parameter) - (center : Parameter)|| ^ 2
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModelOn](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared the A2 formula in assumptions `a1` `a2` `joint_smoothness` and `strong_convexity` at cited publication: 270-282 with the full expanded conjunctive predicate, its model prerequisite, and the theorem-4.3 use. For every datum and feasible center, `HasFDerivWithinAt` identifies the derivative of the loss on `Theta` with the linear functional `v -> inner(gradient datum center, v)`. The subsequent inequality uses exactly this same vector, every feasible candidate, the displacement candidate-center, and modulus/2 times its squared norm. This gives the actual first-order quadratic lower model in source A2 rather than an unrestricted supporting-vector condition. The zero extension agrees with the loss at every point used by the within-domain derivative, so it imposes no differentiability outside `Theta` or discontinuity condition across its boundary. On lower-dimensional domains, any undetermined normal component has zero pairing with feasible displacements and does not change A2. The paper use requires this whole predicate and a strictly positive modulus on its closed convex domain; A1 smoothness is not inadvertently added. In particular, $\theta^2 + |\theta|$ on $[-1, 1]$ cannot satisfy the derivative conjunct at its interior point 0.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

AppliedModelingLib.MeasurePerformativeModelOn.IsWassersteinSensitive

Source locator: cited publication, lines 242-249

Verbatim paper-source connection

formatively optimal classifiers need not be performatively stable and performatively stable classifiers need not be performatively optimal. We illustrate this point in the context

$$W_1(D(\theta), D(\theta_0)) \leq \epsilon \text{ for } \theta_0 \in \Theta,$$

of our previous biased coin toss example.

Example 2.2 (continued). Consider again our model of a biased coin toss. In order for a classifier f_θ to be perfor-

Definition 3.1 (ϵ -sensitivity). We say that a distribution map $D(\cdot)$ is ϵ -sensitive if for all $\theta, \theta_0 \in \Theta$:

where W_1 denotes the earth mover's distance.

Lean-expanded library semantic target

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[MetricSpace Parameter] →
[inst : MeasurableSpace Data] →
[MetricSpace Data] →
{domain : Set Parameter} → AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain → ℝ → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MetricSpace Parameter] [MeasurableSpace Data] [MetricSpace Data]
{domain : Set Parameter} (model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain)
(sensitivity : ℝ) =>
∀ (first second : ↑ domain),
∃ (hfinite :
(AppliedModelingLib.ProbabilityCoupling.expectedDistanceCosts (model.dataLaw first)
(model.dataLaw second)).Nonempty),
AppliedModelingLib.ProbabilityCoupling.wassersteinOneReal (model.dataLaw first) (model.dataLaw second) hfinite ≤
sensitivity * dist ↑ first ↑ second
```

Exact Lean library declaration

```
/-- Wasserstein sensitivity for a model defined only on its parameter domain. -/
def IsWassersteinSensitive
{Parameter Data : Type*} [MetricSpace Parameter] [MeasurableSpace Data]
[MetricSpace Data] {domain : Set Parameter}
(model : MeasurePerformativeModelOn Parameter Data domain)
(sensitivity : ℝ) : Prop :=
∀ first second,
∃ hfinite : (ProbabilityCoupling.expectedDistanceCosts
(model.dataLaw first) (model.dataLaw second)).Nonempty,
ProbabilityCoupling.wassersteinOneReal
(model.dataLaw first) (model.dataLaw second) hfinite ≤
sensitivity * dist (first : Parameter) (second : Parameter)
```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModelOn](#)
- [AppliedModelingLib.ProbabilityCoupling.expectedDistanceCosts](#)
- [AppliedModelingLib.ProbabilityCoupling.wassersteinOneReal](#)

Recorded source-to-library screening Verdict: matches

Reason: Compared definition_3_1_epsilon_sensitivity at cited publication:242-249 with the expanded predicate and every listed coupling/model/use display. The joint probability law has exactly the two prescribed marginals. expectedDistanceCosts consists of its finite expected distances, and wassersteinOneReal takes their infimum with an explicit nonempty finite-cost witness. The target is therefore the source earth-mover bound $W_1(D(\text{first}), D(\text{second})) \leq \epsilon$ for all feasible parameter pairs. A finite source bound corresponds to this finite-cost infimum; infinite-cost couplings cannot lower it. In the inner-product paper use, parameter distance equals the norm of the difference, and the explicit sensitivity premise permits precisely nonnegative epsilon, including 0. No attainment of the infimum, finite support, or off-domain distribution is required.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.measureDecoupledPerformativeRisk

Source locator: cited publication, lines 219-228

Verbatim paper-source connection

Definition 2.3 (performative stability and decoupled risk). Points
 A classifier $f_{\theta PS}$ is performatively stable if the following Having introduced our framework for performative predic-
 relationship holds: tion, we now address some of the basic questions that arise

$$\theta PS = \arg \min_{\theta} E_{Z \sim D(\theta PS)} \ell(Z; \theta).$$
 in this setting and examine the behavior of common ma-
chine learning practices, such as retraining, through the lens
of performativity. We begin by analyzing the behavior of
 We define $DPR(\theta, \theta_0) = E_{Z \sim D(\theta)} \ell(Z; \theta_0)$ as these procedures when they operate at a population level
 the decoupled performative risk; then, $\theta PS =$ and then extend our analysis to finite samples.
 $\arg \min_{\theta} DPR(\theta PS, \theta).$

Lean-expanded library semantic target

```

type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
AppliedModelingLib.MeasurePerformativeModel Parameter Data → Parameter → Parameter → ℝ

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data)
(distributionParameter evaluatedParameter : Parameter) =>
f (datum : Data), model.loss datum evaluatedParameter ∂ (model.dataLaw distributionParameter)

```

Exact Lean library declaration

```

/-- General-distribution frozen population risk. -/
noncomputable def measureDecoupledPerformativeRisk {Parameter Data : Type*} [MeasurableSpace Data]
(model : MeasurePerformativeModel Parameter Data)
(distributionParameter evaluatedParameter : Parameter) : ℝ :=
f datum, model.loss datum evaluatedParameter ∂ (model.dataLaw distributionParameter : Measure Data)

```

Direct retained dependencies

- [AppliedModelingLib.MeasurePerformativeModel](#)

Recorded source-to-library screening Verdict: matches

Reason: Definition 2.3 defines $DPR(\theta, \theta') = E_{Z \sim D(\theta)} \ell(Z; \theta')$. The expanded Lean declaration is exactly the corresponding integral of `model.loss z evaluatedParameter` under `model.dataLaw distributionParameter`, preserving the distribution/evaluation parameter order; the model integrability condition only makes the expectation real-valued and does not change the definition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-specific semantic prerequisites

PZMH20PerformativePrediction.IsMeasurePerformativelyOptimal

Paper-local declaration:

PZMH20PerformativePrediction.IsMeasurePerformativelyOptimal

Source locator: cited publication, lines 161-172

Verbatim paper-source connection

Definition 2.1 (performative optimality and risk). A classifier f_{θ} is performatively optimal if the following relationship holds:

Importantly, this mapping $D(\cdot)$ remains fixed and does not change over time or by intervention: deploying the same classifier at two different points in time must induce the same distribution over observations Z . While causal inference focuses on estimating properties of interventional distributions such as treatment effects (Pearl, 2009; Imbens & Rubin, 2015), our focus is on a new stability notion and

$$\theta_{PO} = \arg \min_{\theta} \mathbb{E}_{Z \sim D(\theta)} \ell(Z; \theta).$$

We define $PR(\theta) = \mathbb{E}_{Z \sim D(\theta)} \ell(Z; \theta)$ as the performative risk; then, $\theta_{PO} = \arg \min_{\theta} PR(\theta)$.

Lean-expanded paper semantic target

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] → AppliedModelingLib.MeasurePerformativeModel Parameter Data → Parameter → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (parameter : Parameter) =>
AppliedModelingLib.IsMeasurePerformativelyOptimal model parameter
```

Direct retained dependencies

- [AppliedModelingLib.IsMeasurePerformativelyOptimal](#)
- [AppliedModelingLib.MeasurePerformativeModel](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Definition 2.1's $\theta_{PO} = \arg \min_{\theta} PR(\theta)$ is exactly the expanded universal minimizer condition for `measurePerformativeRisk`. The explicit probability law and loss integrability merely realize the displayed expectation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PZMH20PerformativePrediction.IsMeasureRRMOn

Paper-local declaration:

PZMH20PerformativePrediction.IsMeasureRRMOn

Source locator: cited publication, lines 296-303

Verbatim paper-source connection

Definition 3.3 (RRM). Repeated risk minimization (RRM) refers to the procedure where, starting from an initial model θ_0 we perform the following sequence of updates for $t > 0$:

well as the first-order conditions of optimality for convex functions, a short calculation reveals that

$$(G(\theta) - G(\theta_0)) \geq \inf_{\theta \in \Theta} \langle G(\theta_0), \theta - \theta_0 \rangle \geq -\epsilon \beta \|G(\theta) - G(\theta_0)\|_2 \|\theta - \theta_0\|_2.$$

$\theta_{t+1} = G(\theta_t) = \arg \min_{\theta \in \Theta} E_{Z \sim D(\theta_t)} \ell(Z; \theta)$. Claim (a) then follows by combining the previous two inequalities and rearranging. Intuitively, strong convexity forces the iterates

Lean-expanded paper semantic target

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] →
AppliedModelingLib.MeasurePerformativeModel Parameter Data → Set Parameter → (Parameter → Parameter) → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (domain : Set Parameter)
(update : Parameter → Parameter) =>
AppliedModelingLib.IsMeasureRepeatedRiskMinimizationOn model domain update
```

Direct retained dependencies

- [AppliedModelingLib.IsMeasureRepeatedRiskMinimizationOn](#)
- [AppliedModelingLib.MeasurePerformativeModel](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Definition 3.3's $G(\theta_t) = \arg \min_{\theta \in \Theta} E_{Z \sim D(\theta_t)} \ell(Z; \theta)$ matches the selected frozen-risk minimizer over domain, including the requirement that the update remains in the parameter domain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PZMH20PerformativePrediction.IsMeasureStable

Paper-local declaration:

PZMH20PerformativePrediction.IsMeasureStable

Source locator: cited publication, lines 219-228

Verbatim paper-source connection

Definition 2.3 (performative stability and decoupled risk). Points
A classifier $f_{\theta PS}$ is performatively stable if the following Having introduced our framework for performative prediction, we now address some of the basic questions that arise in this setting and examine the behavior of common machine learning practices, such as retraining, through the lens of performativity. We begin by analyzing the behavior of
relationship holds:
$$\theta PS = \arg \min_{\theta} \mathbb{E}_{Z \sim D(\theta PS)} \ell(Z; \theta).$$

We begin by analyzing the behavior of
these procedures when they operate at a population level and then extend our analysis to finite samples.
We define $DPR(\theta, \theta_0) = \mathbb{E}_{Z \sim D(\theta)} \ell(Z; \theta_0)$ as the decoupled performative risk; then, $\theta PS = \arg \min_{\theta} DPR(\theta PS, \theta).$

Lean-expanded paper semantic target

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[inst : MeasurableSpace Data] → AppliedModelingLib.MeasurePerformativeModel Parameter Data → Parameter → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MeasurableSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (parameter : Parameter) =>
AppliedModelingLib.IsMeasurePerformativelyStable model parameter
```

Direct retained dependencies

- [AppliedModelingLib.IsMeasurePerformativelyStable](#)
- [AppliedModelingLib.MeasurePerformativeModel](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Definition 2.3's $\theta PS = \arg \min_{\theta} DPR(\theta PS, \theta)$ is the expanded condition that the frozen law at parameter has no lower-risk candidate.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

PZMH20PerformativePrediction.IsMeasureWassersteinSensitive

Paper-local declaration:

PZMH20PerformativePrediction.IsMeasureWassersteinSensitive

Source locator: cited publication, lines 242-249

Verbatim paper-source connection

formatively optimal classifiers need not be performatively stable and performatively stable classifiers need not be performatively optimal. We illustrate this point in the context

Definition 3.1 (ϵ -sensitivity). We say that a distribution map $D(\cdot)$ is ϵ -sensitive if for all $\theta, \theta_0 \in \Theta$:

$$W_1(D(\theta), D(\theta_0)) \leq \epsilon \|\theta - \theta_0\|_2,$$



of our previous biased coin toss example.

Example 2.2 (continued). Consider again our model of a biased coin toss. In order for a classifier f_θ to be per-

where W_1 denotes the earth mover's distance.

Lean-expanded paper semantic target

```
type:
{Parameter : Type u_1} →
{Data : Type u_2} →
[MetricSpace Parameter] →
[inst : MeasurableSpace Data] →
[MetricSpace Data] → AppliedModelingLib.MeasurePerformativeModel Parameter Data → ℝ → Prop

value:
fun {Parameter : Type u_1} {Data : Type u_2} [MetricSpace Parameter] [MeasurableSpace Data] [MetricSpace Data]
(model : AppliedModelingLib.MeasurePerformativeModel Parameter Data) (sensitivity : ℝ) =>
AppliedModelingLib.IsMeasureWassersteinSensitive model sensitivity
```

Direct retained dependencies

- [AppliedModelingLib.IsMeasureWassersteinSensitive](#)
- [AppliedModelingLib.MeasurePerformativeModel](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Definition 3.1's $W_1(D(\theta), D(\theta')) \leq \epsilon \|\theta - \theta'\|_2$ is represented by a finite expected-distance coupling and its real W_1 infimum bounded by sensitivity * dist; finiteness is required for the source's finite displayed bound.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData

Source locator: cited publication, lines 373-415

Verbatim paper-source connection

Theorem 3.10. Suppose that the loss $\ell(z; \theta)$ is β -jointly smooth (A1) and γ -strongly convex (A2), and that there exist $\alpha > 1, \mu > 0$ such that $\xi_{\alpha, \mu} = \sup_{\theta \in \Theta} \int_{\mathcal{D}} \ell(z; \theta) dD(\theta)$ is finite. If the distribution $D(\cdot)$ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then RGD with step size $\eta \leq \frac{2}{6\beta + \gamma}$ satisfies the following:

(a) If the map $D(\cdot)$ is ϵ -sensitive with $\epsilon < 2\beta$, then with probability $1 - p$, RERM satisfies,

$$k\theta_t - \theta_{PS} k^2 \leq \frac{\log \frac{1}{1 - 2\epsilon\beta} k\theta_0 - \theta_{PS} k^2}{1 - 2\epsilon\beta}, \text{ for all } t > \frac{\log \frac{1}{1 - 2\epsilon\beta} k\theta_0 - \theta_{PS} k^2}{\eta \beta + \gamma - \epsilon(1.5\eta\beta + \beta)}.$$

(b) If the map $D(\cdot)$ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then with probability $1 - p$, REGD with step size $\eta \leq \frac{2}{6\beta + \gamma}$ satisfies,

$$k\theta_t - \theta_{PS} k^2 \leq \frac{\log \frac{1}{1 - 2\epsilon\beta} k\theta_0 - \theta_{PS} k^2}{1 - 2\epsilon\beta}, \text{ for all } t > \frac{\log \frac{1}{1 - 2\epsilon\beta} k\theta_0 - \theta_{PS} k^2}{\eta \beta + \gamma - \epsilon(3\eta\beta + 2\beta)}.$$

The conclusion of Theorem 3.8 is a strict generalization of a classical optimization result which considers a static objective, in which case the rate of contraction is $1 - \eta \beta + \gamma$ for a constant choice of step size $\eta \leq \frac{2}{6\beta + \gamma}$. (see for example Theorem 2.1.15 in Nesterov (2013) or Lemma 3.7 in Hardt et al. (2016b)). Our rate exactly matches this standard result in the case that $\epsilon = 0$. The proof of Theorem 3.8 can be found in Appendix E.3.

Proof sketch. The basic idea behind these results is the following. While $k\theta_t - \theta_{PS} k^2 > \delta$, the sample size nt is sufficiently large to ensure a behavior similar to that on a

Formalized review target The archival source above is not asserted equivalent to this formalized target.

The REGD premise bundle for Theorem 3.10(b) specifies data dimension greater than two, the concrete capped count and inverse-square confidence schedules, a shared bounded exponential moment and tail envelope, measurable adaptive sampling and bad events, joint-gradient smoothness and pointwise strong convexity with Wasserstein sensitivity, a convex domain with a variational nonexpansive projection, gradient integrability and pointwise loss differentiation, a checked empirical-gradient trajectory and a stable point, and the step-size, numerical W_1 budget, contraction slack, absorbing-radius, and initial-distance conditions. The trajectory theorem derives population contraction and a finite entry iteration; neither a caller-supplied small-deviation gate nor a supplied burn-in bound is a premise. The model and expected-loss integrability are defined for every ambient parameter. Joint-gradient smoothness and pointwise strong convexity quantify over all ambient parameters and data points; Wasserstein sensitivity, gradient integrability, and loss differentiation quantify over all ambient parameters. These global hypotheses are stronger in scope than the source assumptions on the parameter domain and the union of its induced supports; their necessity for the source conclusion is unresolved.

Archival source anchor: cited publication, lines 373-415

Lean-elaborated paper signature

```
field dimension:
{Parameter : Type u_1} →
  [inst : MeasurableSpace Parameter] →
    [inst_1 : NormedAddCommGroup Parameter] →
      [inst_2 : InnerProductSpace ℝ Parameter] →
        [inst_3 : CompleteSpace Parameter] →
          PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → ℝ

field cutoff:
{Parameter : Type u_1} →
  [inst : MeasurableSpace Parameter] →
    [inst_1 : NormedAddCommGroup Parameter] →
      [inst_2 : InnerProductSpace ℝ Parameter] →
        [inst_3 : CompleteSpace Parameter] →
          PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → ℝ

field hdimension:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
  [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
  (self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter), 2 < self.dimension

field eta:
{Parameter : Type u_1} →
```



```

field htailEnvelope:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
    [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
    (self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter)
    (law : MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin self.dimension))),
    AppliedModelingLib.Probability.HasBoundedExponentialRadialMoment law self.alpha self.gamma self.momentBound →
    AppliedModelingLib.Probability.fifthOrderExponentialRadialTailConstant law self.alpha self.gamma ≤ self.tailBound

field p:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
      [inst_1 : NormedAddCommGroup Parameter] →
        [inst_2 : InnerProductSpace ℝ Parameter] →
          [inst_3 : CompleteSpace Parameter] →
            PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → ℝ

field headTolerance:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
      [inst_1 : NormedAddCommGroup Parameter] →
        [inst_2 : InnerProductSpace ℝ Parameter] →
          [inst_3 : CompleteSpace Parameter] →
            PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → ℝ

field hp:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
    [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
    (self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter), 0 < self.p

field hheadTolerance:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
    [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
    (self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter), 0 < self.headTolerance

field htailBound_moment:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
    [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
    (self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
    self.tailBound = ↑(Nat.factorial 5) * self.gamma-1 ^ 5 * self.momentBound

field model:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
      [inst_1 : NormedAddCommGroup Parameter] →
        [inst_2 : InnerProductSpace ℝ Parameter] →
          [inst_3 : CompleteSpace Parameter] →
            (self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter) →
              AppliedModelingLib.MeasurePerformativeModel Parameter (EuclideanSpace ℝ (Fin self.dimension))

field sampling:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
      [inst_1 : NormedAddCommGroup Parameter] →
        [inst_2 : InnerProductSpace ℝ Parameter] →
          [inst_3 : CompleteSpace Parameter] →
            (self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter) →
              AppliedModelingLib.MeasurePerformativeSamplingKernel self.model

field gradient:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
      [inst_1 : NormedAddCommGroup Parameter] →
        [inst_2 : InnerProductSpace ℝ Parameter] →
          [inst_3 : CompleteSpace Parameter] →
            (self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter) →
              EuclideanSpace ℝ (Fin self.dimension) → Parameter → Parameter

field smoothness:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
      [inst_1 : NormedAddCommGroup Parameter] →
        [inst_2 : InnerProductSpace ℝ Parameter] →
          [inst_3 : CompleteSpace Parameter] →
            PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → NNReal

field hjoint:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
    [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
    (self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
    AppliedModelingLib.IsJointlySmoothGradient self.gradient self.smoothness

field hsmoothness:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
    [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
    (self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter), self.smoothness ≠ 0

field sensitivity:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
      [inst_1 : NormedAddCommGroup Parameter] →
        [inst_2 : InnerProductSpace ℝ Parameter] →
          [inst_3 : CompleteSpace Parameter] →
            PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → ℝ

field modulus:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
      [inst_1 : NormedAddCommGroup Parameter] →
        [inst_2 : InnerProductSpace ℝ Parameter] →

```

```

[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → ℝ

field hsensitivity_nonneg:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter), 0 ≤ self.sensitivity

field hsensitive:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
PZMH20PerformativePrediction.IsMeasureWassersteinSensitive self.model self.sensitivity

field hmodulus_pos:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter), 0 < self.modulus

field hmodulus_le:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
self.modulus ≤ ↑self.smoothness

field hstrong:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
PZMH20PerformativePrediction.IsMeasurePointwiseGradientStronglyConvex self.model self.gradient self.modulus

field domain:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → Set Parameter

field hconvex:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter), Convex ℝ self.domain

field project:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → Parameter → Parameter

field hproject:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
LipschitzWith 1 self.project

field hprojectVariational:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
AppliedModelingLib.IsVariationalEuclideanProjectionOn self.domain self.project

field stepSize:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → ℝ

field hstepSize:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter), 0 ≤ self.stepSize

field hstepSize_pos:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter), 0 < self.stepSize

field hstepSize_le:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
self.stepSize ≤ 2 / (self.modulus + ↑self.smoothness)

field hsensitivity_small:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
self.sensitivity < self.modulus / ((self.modulus + ↑self.smoothness) * (1 + 3 / 2 * self.stepSize * ↑self.smoothness))

field hintegrable:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter)
(distributionParameter evaluatedParameter : Parameter),
MeasureTheory.Integrable

```

```

(fun (datum : EuclideanSpace ℝ (Fin self.dimension)) => self.gradient datum evaluatedParameter)
↑ (self.model.dataLaw distributionParameter)

field hloss_meas:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAAllRoundTrajectoryData Parameter)
(distributionParameter evaluatedParameter : Parameter),
∀ (parameter : Parameter) in hnds evaluatedParameter,
MeasureTheory.AEStronglyMeasurable
(fun (datum : EuclideanSpace ℝ (Fin self.dimension)) => self.model.loss datum parameter)
↑ (self.model.dataLaw distributionParameter)

field hpointwise:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAAllRoundTrajectoryData Parameter)
(datum : EuclideanSpace ℝ (Fin self.dimension)) (parameter : Parameter),
HasGradientAt (fun (theta : Parameter) => self.model.loss datum theta) (self.gradient datum parameter) parameter

field hcountPositive:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAAllRoundTrajectoryData Parameter) (round : ℕ),
0 <
AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule self.dimension self.cutoff
self.eta self.alpha self.gamma self.momentBound self.tailBound self.scaledDeviation self.p self.headTolerance
round

field deployedOfHistory:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAAllRoundTrajectoryData Parameter) →
(iteration : ℕ) →
AppliedModelingLib.HeterogeneousBatchTrace
(fun (round : ℕ) =>
Fin
(AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule
self.dimension self.cutoff self.eta self.alpha self.gamma self.momentBound self.tailBound
self.scaledDeviation self.p self.headTolerance round))
(EuclideanSpace ℝ (Fin self.dimension)) iteration →
Parameter

field hmeasurableDeployed:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAAllRoundTrajectoryData Parameter) (iteration : ℕ),
Measurable (self.deployedOfHistory iteration)

field hempirical:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAAllRoundTrajectoryData Parameter) (iteration : ℕ)
(history :
AppliedModelingLib.HeterogeneousBatchTrace
(fun (round : ℕ) =>
Fin
(AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule self.dimension
self.cutoff self.eta self.alpha self.gamma self.momentBound self.tailBound self.scaledDeviation self.p
self.headTolerance round))
(EuclideanSpace ℝ (Fin self.dimension)) iteration)
(batch :
Fin
(AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule self.dimension
self.cutoff self.eta self.alpha self.gamma self.momentBound self.tailBound self.scaledDeviation self.p
self.headTolerance iteration) →
EuclideanSpace ℝ (Fin self.dimension)),
self.deployedOfHistory (iteration + 1) (AppliedModelingLib.heterogeneousBatchTraceSnoc history batch) =
AppliedModelingLib.measureSampledGradientDescentUpdate self.gradient self.project self.stepSize
(self.hcountPositive iteration) batch (self.deployedOfHistory iteration history)

field hintegrableEmpirical:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAAllRoundTrajectoryData Parameter) (iteration : ℕ)
(history :
AppliedModelingLib.HeterogeneousBatchTrace
(fun (round : ℕ) =>
Fin
(AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule self.dimension
self.cutoff self.eta self.alpha self.gamma self.momentBound self.tailBound self.scaledDeviation self.p
self.headTolerance round))
(EuclideanSpace ℝ (Fin self.dimension)) iteration)
(batch :
Fin
(AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule self.dimension
self.cutoff self.eta self.alpha self.gamma self.momentBound self.tailBound self.scaledDeviation self.p
self.headTolerance iteration) →
EuclideanSpace ℝ (Fin self.dimension)),
MeasureTheory.Integrable
(fun (datum : EuclideanSpace ℝ (Fin self.dimension)) =>
self.gradient datum (self.deployedOfHistory iteration history))
↑ (AppliedModelingLib.empiricalSampleProbabilityMeasureOfPos (self.hcountPositive iteration) batch)

field stable:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →

```

```

[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → Parameter

field hstablePerformative:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
AppliedModelingLib.IsMeasurePerformativelyStableOn self.model self.domain self.stable

field contraction:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → ℝ

field outerContraction:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → ℝ

field wassersteinBound:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → ℝ

field radius:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → ℝ

field hcontraction_eq:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
self.contraction =
1 -
self.stepSize *
(self.modulus * ↑self.smoothness / (self.modulus + ↑self.smoothness) -
self.sensitivity * (3 / 2 * self.stepSize * ↑self.smoothness ^ 2 + ↑self.smoothness))

field hcontraction_le_outer:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
self.contraction ≤ self.outerContraction

field hbound_nonneg:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
0 ≤ self.wassersteinBound

field hbudget:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
√↑self.dimension * self.deviation + 2 * √↑self.dimension * (32 / 15 * self.tailBound / 16 ^ self.cutoff) +
self.headTolerance ≤
self.wassersteinBound

field herror_outer:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
self.stepSize * ↑self.smoothness * self.wassersteinBound ≤ (self.outerContraction - self.contraction) * self.radius

field herror_absorbed:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
self.stepSize * ↑self.smoothness * self.wassersteinBound ≤ (1 - self.contraction) * self.radius

field initialDistance:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter → ℝ

field houter_lt_one:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
self.outerContraction < 1

field hradius_pos:

```

```

∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAAllRoundTrajectoryData Parameter), 0 < self.radius

field hinitialDistance:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAAllRoundTrajectoryData Parameter)
(history :
  AppliedModelingLib.HeterogeneousBatchTrace
    (fun (round : ℕ) =>
      Fin
        (AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule self.dimension
          self.cutoff self.eta self.alpha self.gamma self.momentBound self.tailBound self.scaledDeviation self.p
            self.headTolerance round))
        (EuclideanSpace ℝ (Fin self.dimension)) 0),
  dist (self.deployedOfHistory 0 history) self.stable ≤ self.initialDistance

field hmoment:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAAllRoundTrajectoryData Parameter) (iteration : ℕ)
(history :
  AppliedModelingLib.HeterogeneousBatchTrace
    (fun (round : ℕ) =>
      Fin
        (AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule self.dimension
          self.cutoff self.eta self.alpha self.gamma self.momentBound self.tailBound self.scaledDeviation self.p
            self.headTolerance round))
        (EuclideanSpace ℝ (Fin self.dimension)) iteration),
  AppliedModelingLib.Probability.HasBoundedExponentialRadialMoment
    (self.model.dataLaw (self.deployedOfHistory iteration history)) self.alpha self.gamma self.momentBound

field hevent:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310CorrectedRGDAAllRoundTrajectoryData Parameter) (iteration : ℕ),
MeasurableSet
  (AppliedModelingLib.heterogeneousBatchTraceBadEventPair
    (fun (round : ℕ) =>
      Fin
        (AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule self.dimension
          self.cutoff self.eta self.alpha self.gamma self.momentBound self.tailBound self.scaledDeviation self.p
            self.headTolerance round))
        (AppliedModelingLib.pOneFournierGuillinAdaptiveEuclideanSelectedShellPartitionCertificateBadEvent self.dimension
          self.cutoff self.eta
            (AppliedModelingLib.pOneFournierGuillinTheorem310CappedDeviation self.dimension self.eta self.alpha self.gamma
              self.momentBound self.scaledDeviation)
            (AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalConfidenceSchedule self.p)
            (AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule self.dimension
              self.cutoff self.eta self.alpha self.gamma self.momentBound self.tailBound self.scaledDeviation self.p
                self.headTolerance)
            (AppliedModelingLib.measurePerformativeDeployedLaw self.model
              (fun (round : ℕ) =>
                Fin
                  (AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule self.dimension
                    self.cutoff self.eta self.alpha self.gamma self.momentBound self.tailBound self.scaledDeviation self.p
                      self.headTolerance round))
                  self.deployedOfHistory))
                iteration))
  )

```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)
- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.IsJointlySmoothGradient](#)
- [AppliedModelingLib.IsMeasurePerformativelyStableOn](#)
- [AppliedModelingLib.IsVariationalEuclideanProjectionOn](#)
- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.MeasurePerformativeSamplingKernel](#)
- [AppliedModelingLib.Probability.HasBoundedExponentialRadialMoment](#)
- [AppliedModelingLib.Probability.fifthOrderExponentialRadialTailConstant](#)
- [AppliedModelingLib.empiricalSampleProbabilityMeasureOfPos](#)
- [AppliedModelingLib.heterogeneousBatchTraceBadEventPair](#)
- [AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace](#)
- [AppliedModelingLib.heterogeneousBatchTraceMeasurableSpace](#)
- [AppliedModelingLib.heterogeneousBatchTraceSnoc](#)
- [AppliedModelingLib.measurePerformativeDeployedLaw](#)

- [AppliedModelingLib.measureSampledGradientDescentUpdate](#)
- [AppliedModelingLib.pOneFournierGuillinAdaptiveEuclideanSelectedShellPartitionCertificateBadEvent](#)
- [AppliedModelingLib.pOneFournierGuillinTheorem310CappedDeviation](#)
- [AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule](#)
- [AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalConfidenceSchedule](#)
- [PZMH20PerformativePrediction.IsMeasurePointwiseGradientStronglyConvex](#)
- [PZMH20PerformativePrediction.IsMeasureWassersteinSensitive](#)

Recorded source-to-declaration screening Verdict: matches_approved_corrected_target

Reason: The expanded carrier matches the disclosed corrected REGD premise bundle: dimension greater than two, the capped count and inverse-square failure schedules, a uniform adaptive exponential-moment/tail envelope, measurable sampling and bad events, globally scoped gradient regularity and integrability, the convex-domain variational nonexpansive projection, the source step-size and sensitivity inequalities, the empirical REGD recurrence, a supplied stable point, and the stated contraction, W_1 -budget, absorption, radius, and initial-distance conditions. Its graph-bound use derives a deterministic finite entry iteration and all-subsequent-round containment with probability at least $1 - p$; no entry bound or small-deviation gate is supplied by the caller.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData

Source locator: cited publication, lines 373-415

Verbatim paper-source connection

Theorem 3.10. Suppose that the loss $\ell(z; \theta)$ is β -jointly smooth (A1) and γ -strongly convex (A2), and that there exist $\alpha > 1, \mu > 0$ such that $\xi_{\alpha, \mu} = \mathbb{E} \mu[x] \frac{dD(\theta)}{d\theta}$ is finite. If the distribution $D(\cdot)$ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then RGD with step size $\eta \leq \frac{2}{\beta + \gamma}$ satisfies the following:

(a) $k \text{Ggd}(\theta) - \text{Ggd}(\theta) \leq k^2$ with probability $1 - p$, RERM satisfies,

(b) The iterates θ_t of RGD converge to a unique performatively stable point θ_{PS} at a linear rate,

(c) If the map $D(\cdot)$ is ϵ -sensitive with $\epsilon < 2\beta$, then with probability $1 - p$, REGD with step size $\eta \leq \frac{2}{\beta + \gamma}$ satisfies,

The conclusion of Theorem 3.8 is a strict generalization of a classical optimization result which considers a static objective, in which case the rate of contraction is $1 - \eta\beta$ for a constant choice of step size $\eta \leq \frac{2}{\beta + \gamma}$.

(see for example Theorem 2.1.15 in Nesterov (2013) or Lemma 3.7 in Hardt et al. (2016b)). Our rate exactly matches this standard result in the case that $\epsilon = 0$. The proof of Theorem 3.8 can be found in Appendix E.3.

Proof sketch. The basic idea behind these results is the following. While $k\theta_t - \theta_{PS} \leq \delta$, the sample size nt is sufficiently large to ensure a behavior similar to that on a

Formalized review target The archival source above is not asserted equivalent to this formalized target.

The RERM premise bundle for the approved Theorem 3.10(a) recovery fixes the explicit all-positive-dimensional selected-shell count schedule and

- inverse-square confidence allocation; a shared bounded exponential-moment and tail envelope along every adaptive history; measurable adaptive
- sampling and raw-Wasserstein bad events; a convex parameter domain; A1's data-gradient bound and A2's pointwise strong-convexity lower model on
- that domain; empirical-risk integrability; a checked RERM trajectory and population risk-minimizing update with a performatively stable point; and the
- stated sensitivity, numerical W1 budget, absorbing-radius, and entry conditions. The selected-shell construction derives the raw W1 roundwise failure
- bound and deterministic threshold internally. No data-coordinate loss-Lipschitz bridge, gradient-integrability premise, pointwise first-order certificate,
- or caller-supplied selected-shell certificate is assumed.

Archival source anchor: cited publication, lines 373-415

Lean-elaborated paper signature

field dimension:

```
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter → ℕ
```

field cutoff:

```
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter → ℕ
```

field hdimension:

```
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
0 < self.dimension
```

field eta:

```
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
```

86

```

field htailBound_pos:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
    [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
    (self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
    0 < self.tailBound

field htailEnvelope:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
    [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
    (self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter)
    (law : MeasureTheory.ProbabilityMeasure (EuclideanSpace ℝ (Fin self.dimension))),
    AppliedModelingLib.Probability.HasBoundedExponentialRadialMoment law self.alpha self.gamma self.momentBound →
    AppliedModelingLib.Probability.fifthOrderExponentialRadialTailConstant law self.alpha self.gamma ≤ self.tailBound

field p:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
    [inst_1 : NormedAddCommGroup Parameter] →
    [inst_2 : InnerProductSpace ℝ Parameter] →
    [inst_3 : CompleteSpace Parameter] →
    PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter → ℝ

field headTolerance:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
    [inst_1 : NormedAddCommGroup Parameter] →
    [inst_2 : InnerProductSpace ℝ Parameter] →
    [inst_3 : CompleteSpace Parameter] →
    PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter → ℝ

field hp:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
    [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
    (self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
    0 < self.p

field hheadTolerance:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
    [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
    (self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
    0 < self.headTolerance

field htailBound_moment:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
    [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
    (self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
    self.tailBound = ↑(Nat.factorial 5) * self.gamma-1 ^ 5 * self.momentBound

field model:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
    [inst_1 : NormedAddCommGroup Parameter] →
    [inst_2 : InnerProductSpace ℝ Parameter] →
    [inst_3 : CompleteSpace Parameter] →
    (self :
      PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter) →
    AppliedModelingLib.MeasurePerformativeModel Parameter (EuclideanSpace ℝ (Fin self.dimension))

field sampling:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
    [inst_1 : NormedAddCommGroup Parameter] →
    [inst_2 : InnerProductSpace ℝ Parameter] →
    [inst_3 : CompleteSpace Parameter] →
    (self :
      PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter) →
    AppliedModelingLib.MeasurePerformativeSamplingKernel self.model

field gradient:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
    [inst_1 : NormedAddCommGroup Parameter] →
    [inst_2 : InnerProductSpace ℝ Parameter] →
    [inst_3 : CompleteSpace Parameter] →
    (self :
      PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter) →
    EuclideanSpace ℝ (Fin self.dimension) → Parameter → Parameter

field domain:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
    [inst_1 : NormedAddCommGroup Parameter] →
    [inst_2 : InnerProductSpace ℝ Parameter] →
    [inst_3 : CompleteSpace Parameter] →
    PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter →
    Set Parameter

field hconvex:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
    [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
    (self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
    Convex ℝ self.domain

field smoothness:
  {Parameter : Type u_1} →
    [inst : MeasurableSpace Parameter] →
    [inst_1 : NormedAddCommGroup Parameter] →
    [inst_2 : InnerProductSpace ℝ Parameter] →
    [inst_3 : CompleteSpace Parameter] →

```

PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter → NNReal

field hgradient:

```
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter)
(datum : EuclideanSpace ℝ (Fin self.dimension)),
∀ parameter ∈ self.domain,
HasGradientWithinAt (self.model.loss datum) (self.gradient datum parameter) self.domain parameter
```

field hdata:

```
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter)
(first second : EuclideanSpace ℝ (Fin self.dimension)),
∀ parameter ∈ self.domain,
||self.gradient first parameter - self.gradient second parameter|| ≤ ↑self.smoothness * dist first second
```

field modulus:

```
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter → ℝ
```

field hmodulus:

```
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
0 < self.modulus
```

field hstrong:

```
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
PZMH20PerformativePrediction.EmpiricalMinimizerPerturbation.IsPointwiseGradientStronglyConvexOn self.model.loss
self.gradient self.domain self.modulus
```

field deployedOfHistory:

```
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
(self :
PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter) →
(iteration : ℕ) →
AppliedModelingLib.HeterogeneousBatchTrace
(fun (round : ℕ) =>
Fin
(AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule
self.dimension self.cutoff self.eta self.alpha self.gamma self.tailBound self.deviation
self.scaledDeviation self.p self.headTolerance round))
(EuclideanSpace ℝ (Fin self.dimension)) iteration →
Parameter
```

field hmeasurableDeployed:

```
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter)
(iteration : ℕ), Measurable (self.deployedOfHistory iteration)
```

field hempirical:

```
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
AppliedModelingLib.IsHeterogeneousMeasureRepeatedEmpiricalRiskMinimizationTrajectoryOn self.model self.domain
(AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule self.dimension self.cutoff
self.eta self.alpha self.gamma self.tailBound self.deviation self.scaledDeviation self.p self.headTolerance)
(AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule_pos self.dimension
self.cutoff self.eta self.alpha self.gamma self.tailBound self.deviation self.scaledDeviation self.p
self.headTolerance)
self.deployedOfHistory
```

field populationUpdate:

```
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter →
Parameter → Parameter
```

field hpopulation:

```
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
AppliedModelingLib.IsMeasureRepeatedRiskMinimizationOn self.model self.domain self.populationUpdate
```

field stable:

```
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter →
Parameter
```

field hstable:

```

∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
AppliedModelingLib.IsMeasurePerformativelyStableOn self.model self.domain self.stable

field sensitivity:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter → ℝ

field radius:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter → ℝ

field hsensitivityNonneg:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
0 ≤ self.sensitivity

field hsmoothnessPos:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
0 < ↑self.smoothness

field hsensitivitySmall:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
self.sensitivity < self.modulus / (2 * ↑self.smoothness)

field hsensitive:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
PZMH20PerformativePrediction.IsMeasureWassersteinSensitiveOn self.model self.domain self.sensitivity

field hradius:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
0 < self.radius

field hinitial:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter)
(history :
AppliedModelingLib.HeterogeneousBatchTrace
(fun (round : ℕ) =>
Fin
(AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule self.dimension
self.cutoff self.eta self.alpha self.gamma self.tailBound self.deviation self.scaledDeviation self.p
self.headTolerance round))
(EuclideanSpace ℝ (Fin self.dimension)) 0),
self.deployedOfHistory 0 history ∈ self.domain

field hempiricalIntegrable:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter)
(iteration : ℕ)
(batch :
Fin
(AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule self.dimension
self.cutoff self.eta self.alpha self.gamma self.tailBound self.deviation self.scaledDeviation self.p
self.headTolerance iteration) →
EuclideanSpace ℝ (Fin self.dimension))),
∀ candidate ∈ self.domain,
MeasureTheory.Integrable (fun (datum : EuclideanSpace ℝ (Fin self.dimension)) => self.model.loss datum candidate)
↑ (AppliedModelingLib.empiricalSampleProbabilityMeasureOfPos
(AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule_pos self.dimension
self.cutoff self.eta self.alpha self.gamma self.tailBound self.deviation self.scaledDeviation self.p
self.headTolerance iteration)
batch)

field entryIteration:
{Parameter : Type u_1} →
[inst : MeasurableSpace Parameter] →
[inst_1 : NormedAddCommGroup Parameter] →
[inst_2 : InnerProductSpace ℝ Parameter] →
[inst_3 : CompleteSpace Parameter] →
PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter → ℕ

field hentry:
∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter)
(trace :
(i : ℕ) →
AppliedModelingLib.HeterogeneousBatchTraceCoordinate
(fun (round : ℕ) =>

```

```

    Fin
    (AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule self.dimension
    self.cutoff self.eta self.alpha self.gamma self.tailBound self.deviation self.scaledDeviation self.p
    self.headTolerance round))
  (EuclideanSpace ℝ (Fin self.dimension)) i),
Real.log
  (dist (self.deployedOfHistory 0 (AppliedModelingLib.heterogeneousBatchTraceOfInfiniteTrace 0 trace))
  self.stable /
  self.radius) /
  (1 - 2 * (self.sensitivity * ↑self.smoothness / self.modulus)) ≤
  ↑self.entryIteration

field hmoment:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
  [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
  (self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter)
  (iteration : ℕ)
  (history :
    AppliedModelingLib.HeterogeneousBatchTrace
    (fun (round : ℕ) =>
      Fin
      (AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule self.dimension
      self.cutoff self.eta self.alpha self.gamma self.tailBound self.deviation self.scaledDeviation self.p
      self.headTolerance round))
    (EuclideanSpace ℝ (Fin self.dimension)) iteration),
  AppliedModelingLib.Probability.HasBoundedExponentialRadialMoment
  (self.model.dataLaw (self.deployedOfHistory iteration history)) self.alpha self.gamma self.momentBound

field hbudget:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
  [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
  (self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
  √↑self.dimension * self.deviation + 2 * √↑self.dimension * (32 / 15 * self.tailBound / 16 ^ self.cutoff) +
  self.headTolerance ≤
  self.sensitivity * self.radius

field hevent:
  ∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
  [inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
  (self : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter)
  (iteration : ℕ),
  MeasurableSet
  (AppliedModelingLib.heterogeneousBatchTraceBadEventPair
  (fun (round : ℕ) =>
    Fin
    (AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule self.dimension
    self.cutoff self.eta self.alpha self.gamma self.tailBound self.deviation self.scaledDeviation self.p
    self.headTolerance round))
  (AppliedModelingLib.pOneFournierGuillinAdaptiveEuclideanWassersteinOneBadEvent self.dimension self.cutoff
  self.alpha self.gamma self.momentBound self.deviation
  (AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalConfidenceSchedule self.p)
  (AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule self.dimension
  self.cutoff self.eta self.alpha self.gamma self.tailBound self.deviation self.scaledDeviation self.p
  self.headTolerance)
  (AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule_pos self.dimension
  self.cutoff self.eta self.alpha self.gamma self.tailBound self.deviation self.scaledDeviation self.p
  self.headTolerance)
  (AppliedModelingLib.measurePerformativeDeployedLaw self.model
  (fun (round : ℕ) =>
    Fin
    (AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule self.dimension
    self.cutoff self.eta self.alpha self.gamma self.tailBound self.deviation self.scaledDeviation self.p
    self.headTolerance round))
  self.deployedOfHistory))
  iteration)

```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)
- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.IsHeterogeneousMeasureRepeatedEmpiricalRiskMinimizationTrajectoryOn](#)
- [AppliedModelingLib.IsMeasurePerformativelyStableOn](#)
- [AppliedModelingLib.IsMeasureRepeatedRiskMinimizationOn](#)
- [AppliedModelingLib.MeasurePerformativeModel](#)
- [AppliedModelingLib.MeasurePerformativeSamplingKernel](#)
- [AppliedModelingLib.Probability.HasBoundedExponentialRadialMoment](#)
- [AppliedModelingLib.Probability.fifthOrderExponentialRadialTailConstant](#)
- [AppliedModelingLib.empiricalSampleProbabilityMeasureOfPos](#)
- [AppliedModelingLib.heterogeneousBatchTraceBadEventPair](#)
- [AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace](#)
- [AppliedModelingLib.heterogeneousBatchTraceMeasurableSpace](#)

- [AppliedModelingLib.heterogeneousBatchTraceOfInfiniteTrace](#)
- [AppliedModelingLib.measurePerformativeDeployedLaw](#)
- [AppliedModelingLib.pOneFournierGuillinAdaptiveEuclideanWassersteinOneBadEvent](#)
- [AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule](#)
- [AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalConfidenceSchedule](#)
- [PZMH20PerformativePrediction.EmpiricalMinimizerPerturbation.IsPointwiseGradientStronglyConvexOn](#)
- [PZMH20PerformativePrediction.IsMeasureWassersteinSensitiveOn](#)

Recorded source-to-declaration screening Verdict: matches_approved_corrected_target

Reason: The constructor makes every approved RERM premise explicit: positive-dimensional fixed selected-shell schedule, uniform adaptive moment/tail envelope, measurable sampling and raw-Wasserstein events, convex-domain A1/A2 conditions, empirical and population RERM updates, stability, sensitivity/radius/budget, integrability, and entry. It neither permits an arbitrary schedule nor assumes a finite-IID raw-event bound, threshold, or selected-shell certificate; those are derived by the all-dimensional concentration route.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Main-text source claims

Review row 1

Source locator: cited publication, lines 264-269

Verbatim source input

In addition to this assumption on the distribution map, we will often make standard assumptions on the loss function $\ell(z; \theta)$ which hold for broad classes of losses. Each technical result to follow will invoke some subset of them. To simplify our presentation, let $Z = \cup_{\theta \in \Theta} \text{supp}(D(\theta))$.

(b) If $\epsilon < \beta\gamma$, the iterates θ_t of RRM converge to a unique performatively stable point θ_{PS} at a linear rate,

$$\log(16k\theta_0 - \theta_{PS}k_2) \leq (1 - \epsilon\beta\gamma)^t$$

[Next verbatim source excerpt]

Theorem 3.5. Suppose that the loss $\ell(z; \theta)$ is β -jointly smooth (A1) and γ -strongly convex (A2). If the distribution map $D(\cdot)$ is ϵ -sensitive, then the following is true:

Proposition 3.6 suggests a fundamental difference between strong and weak convexity in our framing of performative prediction (weak meaning $\gamma = 0$). In supervised learning, using strongly convex losses generally guarantees a

(a) $kG(\theta) - G(\theta_0) \leq k_2 \epsilon \beta\gamma k\theta - \theta_0 k_2$, $\forall \theta, \theta_0 \in \Theta$.

[form-feed in source extraction]

Performative Prediction

faster rate of optimization, yet asymptotically, the solution achieved with either strongly or weakly convex losses is globally optimal. However, in our framework, strong convexity is in fact necessary to guarantee convergence of repeated risk minimization, even for arbitrarily smooth losses and an arbitrarily small sensitivity parameter.

3.4. Finite-Sample Analysis

We now extend our main results regarding the convergence of RRM and RGD to the finite-sample regime. To do so, we leverage the fact that, under mild regularity conditions, the empirical distribution D_n given by n samples drawn i.i.d. from a true distribution D is with high probability close to D in earth mover's distance (Fournier & Guillin,

Settled source reading The source parameter space Θ is represented by the subtype of its closed convex finite-dimensional Euclidean domain.

Expanded PaperInterface specification

```
∀ {Parameter : Type u_1} {Data : Type u_2} [inst : NormedAddCommGroup Parameter]
[inst_1 : InnerProductSpace ℝ Parameter] [CompleteSpace Parameter] [FiniteDimensional ℝ Parameter]
[inst_4 : MeasurableSpace Data] [inst_5 : MetricSpace Data] (domain : Set Parameter)
(model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain) (gradient : Data → ↑ domain → Parameter)
{smoothness : NNReal},
((∀ (datum : Data) (first second : ↑ domain),
  ||gradient datum first - gradient datum second|| ≤ ↑smoothness * dist first second) ∧
  ∀ (first second : Data) (parameter : ↑ domain),
  ||gradient first parameter - gradient second parameter|| ≤ ↑smoothness * dist first second) →
  ∀ {sensitivity modulus : ℝ},
  0 ≤ sensitivity →
  0 < modulus →
  model.IsWassersteinSensitive sensitivity →
  IsClosed domain →
  Convex ℝ domain →
  ∀ (update : ↑ domain → ↑ domain),
  (∀ (deployed candidate : ↑ domain),
    model.decoupledPerformativeRisk deployed (update deployed) ≤
    model.decoupledPerformativeRisk deployed candidate) →
  model.IsPointwiseGradientStronglyConvex gradient modulus →
  sensitivity * ↑smoothness / modulus < 1 →
  ∀ (initial : ↑ domain),
  (∀ (first second : ↑ domain),
    ||↑(update first) - ↑(update second)|| ≤
    sensitivity * ↑smoothness / modulus * ||↑first - ↑second||) ∧
  ∃ (stable : ↑ domain),
  model.IsPerformativelyStable stable ∧
  (∀ (other : ↑ domain), model.IsPerformativelyStable other → other = stable) ∧
  Filter.Tendsto (fun (iteration : ℕ) => update^[iteration] initial) Filter.atTop
    (nhds stable) ∧
  (∀ (iteration : ℕ),
    dist (update^[iteration] initial) stable ≤
    (sensitivity * ↑smoothness / modulus) ^ iteration * dist initial stable) ∧
  ∀ radius > 0,
  ∀ (iteration : ℕ),
  Real.log (dist initial stable / radius) /
    (1 - sensitivity * ↑smoothness / modulus) ≤
    ↑iteration →
    dist (update^[iteration] initial) stable ≤ radius
```

Retained governing declarations

- [AppliedModelingLib.MeasurePerformativeModelOn](#)

- [AppliedModelingLib.MeasurePerformativeModelOn.IsPerformativelyStable](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsPointwiseGradientStronglyConvex](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsWassersteinSensitive](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.decoupledPerformativeRisk](#)

Lean-elaborated claim roles

```

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: NormedAddCommGroup Parameter
4. parameter: InnerProductSpace ℝ Parameter
5. assumption: CompleteSpace Parameter
6. assumption: FiniteDimensional ℝ Parameter
7. parameter: MeasurableSpace Data
8. parameter: MetricSpace Data
9. parameter: Set Parameter
10. parameter: AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain
11. parameter: Data → ↑domain → Parameter
12. parameter: NNReal
13. assumption: (∀ (datum : Data) (first second : ↑domain),
  ||gradient datum first - gradient datum second|| ≤ ↑smoothness * dist first second) ∧
  ∀ (first second : Data) (parameter : ↑domain),
  ||gradient first parameter - gradient second parameter|| ≤ ↑smoothness * dist first second
14. parameter: ℝ
15. parameter: ℝ
16. assumption: 0 ≤ sensitivity
17. assumption: 0 < modulus
18. assumption: model.IsWassersteinSensitive sensitivity
19. assumption: IsClosed domain
20. assumption: Convex ℝ domain
21. parameter: ↑domain → ↑domain
22. assumption: ∀ (deployed candidate : ↑domain),
  model.decoupledPerformativeRisk deployed (update deployed) ≤ model.decoupledPerformativeRisk deployed candidate
23. assumption: model.IsPointwiseGradientStronglyConvex gradient modulus
24. assumption: sensitivity * ↑smoothness / modulus < 1
25. parameter: ↑domain
26. conclusion: (∀ (first second : ↑domain),
  ||↑(update first) - ↑(update second)|| ≤ sensitivity * ↑smoothness / modulus * ||↑first - ↑second||) ∧
  ∃ (stable : ↑domain),
  model.IsPerformativelyStable stable ∧
  (∀ (other : ↑domain), model.IsPerformativelyStable other → other = stable) ∧
  Filter.Tendsto (fun (iteration : ℕ) => update^[iteration] initial) Filter.atTop (nhds stable) ∧
  (∀ (iteration : ℕ),
    dist (update^[iteration] initial) stable ≤
    (sensitivity * ↑smoothness / modulus) ^ iteration * dist initial stable) ∧
  ∀ radius > 0,
  ∀ (iteration : ℕ),
    Real.log (dist initial stable / radius) / (1 - sensitivity * ↑smoothness / modulus) ≤ ↑iteration →
    dist (update^[iteration] initial) stable ≤ radius

```

Lean proof endpoint (built separately)

PZMH20PerformativePrediction.measureWassersteinConvexRRMConvergesLinearlyProof

Recorded source-to-Spec screening Verdict: matches

Reason: Under the recorded source-domain convention, the exact Spec includes the finite-dimensional Euclidean parameter domain, on-domain laws/losses/A1/A2/Wasserstein sensitivity, a domain-valued RRM minimizer, the strict epsilon beta over gamma factor, contraction, unique stability, geometric convergence, and the logarithmic entry bound. It imposes no arbitrary off-domain extension and matches the literal Theorem 3.5 statement.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 373-415

Verbatim source input

Theorem 3.10. Suppose that the loss $\ell(z; \theta)$ is β -jointly smooth (A1) and γ -strongly convex (A2), and that there exist $\alpha > 1, \mu > 0$ such that $\xi_{\alpha, \mu} = \mathbb{E} \int_0^{\infty} \mathbb{E} \|\nabla \ell(x_t; \theta)\|^2 dt$ is finite. If the distribution $\mathcal{D}$ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then RGD with step size $\eta \leq \frac{1}{6(\beta + \gamma)}$ satisfies the following:

(a) If the map $D(\cdot)$ is ϵ -sensitive with $\epsilon < 2\beta$, then with probability $1 - p$, RERM satisfies,

$$\|k\theta_t - \theta_{PS}\|_2 \leq \frac{\log 16 k\theta_0 - \theta_{PS}}{1 - 2\epsilon\beta} \leq \delta, \text{ for all } t > \frac{\log 16 k\theta_0 - \theta_{PS}}{1 - 2\epsilon\beta}.$$

(b) If the map $D(\cdot)$ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then with probability $1 - p$, REGD with step size $\eta \leq \frac{1}{6(\beta + \gamma)}$ satisfies,

$$\|k\theta_t - \theta_{PS}\|_2 \leq \frac{\log 16 k\theta_0 - \theta_{PS}}{1 - 2\epsilon\beta} \leq \delta, \text{ for all } t > \frac{\log 16 k\theta_0 - \theta_{PS}}{1 - 2\epsilon\beta}.$$

The conclusion of Theorem 3.8 is a strict generalization of a classical optimization result which considers a static objective, in which case the rate of contraction is $1 - \eta\beta$ for a constant choice of step size $\eta \leq \frac{1}{6(\beta + \gamma)}$.

(see for example Theorem 2.1.15 in Nesterov (2013) or Lemma 3.7 in Hardt et al. (2016b)). Our rate exactly matches this standard result in the case that $\epsilon = 0$. The proof of Theorem 3.8 can be found in Appendix E.3.

Proof sketch. The basic idea behind these results is the following. While $\|k\theta_t - \theta_{PS}\|_2 > \delta$, the sample size nt is sufficiently large to ensure a behavior similar to that on a

Settled source reading The source parameter space Θ is represented by the subtype of its closed convex finite-dimensional Euclidean domain.

Expanded PaperInterface specification

```

∀ {Parameter : Type u_1} {Data : Type u_2} [inst : NormedAddCommGroup Parameter]
[inst_1 : InnerProductSpace ℝ Parameter] [inst_2 : CompleteSpace Parameter] [FiniteDimensional ℝ Parameter]
[inst_4 : MeasurableSpace Data] [inst_5 : MetricSpace Data] {domain : Set Parameter}
(model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain) (gradient : Data → ↑ domain → Parameter)
{smoothness : NNReal},
((∀ (datum : Data) (first second : ↑ domain),
  ||gradient datum first - gradient datum second|| ≤ ↑smoothness * ||↑first - ↑second||) ∧
  ∀ (first second : Data) (parameter : ↑ domain),
  ||gradient first parameter - gradient second parameter|| ≤ ↑smoothness * dist first second) →
  (∀ (deployed evaluated : ↑ domain),
    MeasureTheory.Integrable (fun (datum : Data) => gradient datum evaluated) ↑(model.dataLaw deployed)) →
  ∀ {sensitivity modulus : ℝ},
  0 ≤ sensitivity →
  model.IsWassersteinSensitive sensitivity →
  0 < modulus →
  model.IsPointwiseGradientStronglyConvex gradient modulus →
  ∀ (hclosed : IsClosed domain) (hnonempty : domain.Nonempty) (hconvex : Convex ℝ domain) (stepSize : ℝ),
  0 < stepSize →
  stepSize ≤ 2 / (modulus + ↑smoothness) →
  sensitivity < modulus / ((modulus + ↑smoothness) * (1 + 3 / 2 * stepSize * ↑smoothness)) →
  ∀ (initial : ↑ domain),
  have factor : ℝ :=
    1 -
    stepSize *
    (modulus * ↑smoothness / (modulus + ↑smoothness) -
    sensitivity * (3 / 2 * stepSize * ↑smoothness ^ 2 + ↑smoothness));
  (∀ (first second : ↑ domain),
    dist
    (PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn model gradient hnonempty
    hclosed hconvex stepSize first)
    (PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn model gradient hnonempty
    hclosed hconvex stepSize second) ≤
    factor * dist first second) ∧
  ∃ (stable : ↑ domain),
  model.IsPerformativelyStable stable ∧
  (∀ (other : ↑ domain), model.IsPerformativelyStable other → other = stable) ∧
  Filter.Tendsto

```

```

(fun (iteration : ℕ) =>
  (PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn model gradient
    hnonempty hclosed hconvex stepSize)^[iteration]
  initial)
Filter.atTop (nhds stable) ∧
(∀ (iteration : ℕ),
  dist
    ((PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn model gradient
      hnonempty hclosed hconvex stepSize)^[iteration]
      initial)
    stable ≤
    factor ^ iteration * dist initial stable) ∧
∀ radius > 0,
  ∀ (iteration : ℕ),
  Real.log (dist initial stable / radius) / (1 - factor) ≤ ↑iteration →
  dist
    ((PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn model gradient
      hnonempty hclosed hconvex stepSize)^[iteration]
      initial)
    stable ≤
    radius

```

Retained governing declarations

- [AppliedModelingLib.MeasurePerformativeModelOn](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsPerformativelyStable](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsPointwiseGradientStronglyConvex](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsWassersteinSensitive](#)
- [PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn](#)

Lean-elaborated claim roles

```

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: NormedAddCommGroup Parameter
4. parameter: InnerProductSpace ℝ Parameter
5. assumption: CompleteSpace Parameter
6. assumption: FiniteDimensional ℝ Parameter
7. parameter: MeasurableSpace Data
8. parameter: MetricSpace Data
9. parameter: Set Parameter
10. parameter: AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain
11. parameter: Data → ↑domain → Parameter
12. parameter: NNReal
13. assumption: (∀ (datum : Data) (first second : ↑domain),
  ||gradient datum first - gradient datum second|| ≤ ↑smoothness * ||↑first - ↑second||) ∧
  ∀ (first second : Data) (parameter : ↑domain),
  ||gradient first parameter - gradient second parameter|| ≤ ↑smoothness * dist first second
14. assumption: ∀ (deployed evaluated : ↑domain),
  MeasureTheory.Integrable (fun (datum : Data) => gradient datum evaluated) ↑(model.dataLaw deployed)
15. parameter: ℝ
16. parameter: ℝ
17. assumption: 0 ≤ sensitivity
18. assumption: model.IsWassersteinSensitive sensitivity
19. assumption: 0 < modulus
20. assumption: model.IsPointwiseGradientStronglyConvex gradient modulus
21. assumption: IsClosed domain
22. assumption: domain.Nonempty
23. assumption: Convex ℝ domain
24. parameter: ℝ
25. assumption: 0 < stepSize
26. assumption: stepSize ≤ 2 / (modulus + ↑smoothness)
27. assumption: sensitivity < modulus / ((modulus + ↑smoothness) * (1 + 3 / 2 * stepSize * ↑smoothness))
28. parameter: ↑domain
29. conclusion: (∀ (first second : ↑domain),
  dist
    (PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn model gradient hnonempty hclosed hconvex stepSize
      first)
    (PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn model gradient hnonempty hclosed hconvex stepSize
      second) ≤
    (1 -
      stepSize *
      (modulus * ↑smoothness / (modulus + ↑smoothness) -
        sensitivity * (3 / 2 * stepSize * ↑smoothness ^ 2 + ↑smoothness))) *
    dist first second) ∧
  ∃ (stable : ↑domain),
  model.IsPerformativelyStable stable ∧
  (∀ (other : ↑domain), model.IsPerformativelyStable other → other = stable) ∧
  Filter.Tendsto
    (fun (iteration : ℕ) =>
      (PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn model gradient hnonempty hclosed hconvex
        stepSize)^[iteration]
        initial)
    Filter.atTop (nhds stable) ∧
  (∀ (iteration : ℕ),
    dist
      ((PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn model gradient hnonempty hclosed hconvex
        stepSize)^[iteration]
        initial)
      stable ≤
      (1 -

```

```

      stepSize *
      (modulus * ↑smoothness / (modulus + ↑smoothness) -
      sensitivity * (3 / 2 * stepSize * ↑smoothness ^ 2 + ↑smoothness))) ^
iteration *
dist initial stable) ∧
∀ radius > 0,
  Real.log (dist initial stable / radius) /
  (1 -
  (1 -
  stepSize *
  (modulus * ↑smoothness / (modulus + ↑smoothness) -
  sensitivity * (3 / 2 * stepSize * ↑smoothness ^ 2 + ↑smoothness)))) ≤
  ↑ iteration →
dist
  ((PZMH20PerformativePrediction.DomainRelative.rgdUpdateOn model gradient hnonempty hclosed hconvex
  stepSize)^[iteration]
  initial)
  stable ≤
  radius

```

Lean proof endpoint (built separately)

PZMH20PerformativePrediction.measureWassersteinConvexRGDConvergesLinearlyProof

Recorded source-to-Spec screening Verdict: matches

Reason: Under the recorded source-domain convention, the exact Spec has the finite-dimensional Euclidean domain, A1/A2, sensitivity, displayed step-size and sensitivity inequalities, on-domain expected-gradient integrability, contraction, unique stability, convergence, and rate. It has no user-visible $\gamma \leq \beta$ premise: the paired proof derives it on non-singleton domains and proves the singleton conclusion directly, matching the literal source conditions and conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 442-445

Verbatim source input

Proposition 4.1. Let the distribution map $D(\cdot)$ be ϵ -sensitive and $\Theta \subset \mathbb{R}^d$ be compact. If the loss $\ell(z; \theta)$ is convex and jointly continuous in (z, θ) , then there exists a ϵ -performatively stable classifier.

within the context of strategic classification and discuss some of the implications of our theorems for this setting. We begin by formally establishing how strategic classification can be cast as a performative prediction problem and

Settled source reading The source parameter space Θ is represented by the subtype of its closed convex finite-dimensional Euclidean domain.

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For a measure-performative model defined on the nonempty compact convex source parameter domain Θ in a finite-dimensional real space, assume

- the on-domain decoupled expected risk is jointly continuous and every pointwise loss is convex on Θ . Then a performatively stable parameter
- exists in Θ .

Archival source anchor: cited publication, lines 442-445

Expanded PaperInterface specification

```
∀ {Parameter : Type u_1} {Data : Type u_2} [inst : NormedAddCommGroup Parameter] [inst_1 : NormedSpace ℝ Parameter]
[FiniteDimensional ℝ Parameter] [inst_3 : MeasurableSpace Data] {domain : Set Parameter}
(model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain),
IsCompact domain →
domain.Nonempty →
Convex ℝ domain →
(Continuous fun (point : ↑domain × ↑domain) => model.decoupledPerformativeRisk point.1 point.2) →
(∀ (datum : Data), ConvexOn ℝ domain (PZMH20PerformativePrediction.measureLossOnDomain model datum)) →
∃ (parameter : ↑domain), model.IsPerformativelyStable parameter
```

Retained governing declarations

- [AppliedModelingLib.MeasurePerformativeModelOn](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsPerformativelyStable](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.decoupledPerformativeRisk](#)
- [PZMH20PerformativePrediction.measureLossOnDomain](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: NormedAddCommGroup Parameter
4. parameter: NormedSpace ℝ Parameter
5. assumption: FiniteDimensional ℝ Parameter
6. parameter: MeasurableSpace Data
7. parameter: Set Parameter
8. parameter: AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain
9. assumption: IsCompact domain
10. assumption: domain.Nonempty
11. assumption: Convex ℝ domain
12. assumption: Continuous fun (point : ↑domain × ↑domain) => model.decoupledPerformativeRisk point.1 point.2
13. assumption: ∀ (datum : Data), ConvexOn ℝ domain (PZMH20PerformativePrediction.measureLossOnDomain model datum)
14. conclusion: ∃ (parameter : ↑domain), model.IsPerformativelyStable parameter
```

Lean proof endpoint (built separately)

PZMH20PerformativePrediction.measureCompactConvexStablePointExistsProof

Recorded source-to-Spec screening **Verdict:** matches approved corrected target

Reason: The reviewer compared the exact current Spec with the approved Proposition 4.1 target: it has a finite-dimensional real space, a nonempty compact convex source domain, on-domain joint decoupled-risk continuity, and on-domain pointwise loss convexity, and concludes a domain-stable parameter. This is the declared Berge-continuity correction to the printed proposition, not a claim of archival equivalence.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 460-475

Verbatim source input

Proposition 4.2. The performative risk $PR(\theta)$ can be con-
their features in order to improve their outcomes.

cave in θ , even if the loss $\ell(z; \theta)$ is β -jointly smooth (A1),
 γ -strongly convex (A2), and the distribution map $D(\cdot)$ is
 ε -sensitive with $\varepsilon < \beta\gamma$.

However, what we can show is that there are cases where
finding performatively stable points is sufficient to guaran-
tee that the resulting classifier has low performative risk. In
particular, our next result demonstrates that if the loss func-
tion $\ell(z; \theta)$ is Lipschitz in z and γ -strongly convex, then
all performatively stable points and performative optima
lie in a small neighborhood around each other. Moreover,
the theorem holds for cases where performative optima and
performatively stable points are not necessarily unique.

A classic example of this setting is that of a bank which
uses a machine learning classifier to predict whether or not a

loan applicant is creditworthy. Individual applicants react to
the bank's classifier by manipulating their features with the
hopes of inducing a favorable classification. This game is
said to have a Stackelberg structure since agents adapt their
features only after the bank has deployed their classifier.

The optimal strategy for the institution in a strategic classifi-
cation setting is to deploy the solution corresponding to the
Stackelberg equilibrium, defined as the classifier f_θ which
achieves minimal loss over the induced distribution $D(\theta)$
in which agents have strategically adapted their features in

Expanded PaperInterface specification

```
∃ (mu : ℝ) (epsilon : ℝ) (hmu : |mu| ≤ 1 / 2) (hmuEpsilon : |mu + epsilon| ≤ 1 / 2),
  1 / 2 < epsilon ∧
  epsilon < 2 / 2 ∧
  PZMH20PerformativePrediction.IsFiniteWassersteinSensitive
    (AppliedModelingLib.correctedProposition42Model mu epsilon hmu hmuEpsilon) epsilon ∧
  (∀ (datum : Bool × Bool) (candidate : ℝ),
    HasGradientAt (fun (parameter : ℝ) => AppliedModelingLib.proposition42Loss parameter datum)
      (AppliedModelingLib.proposition42Gradient datum candidate) candidate) ∧
  (∀ (datum : Bool × Bool) (center candidate : ℝ),
    AppliedModelingLib.proposition42Loss candidate datum ≥
      AppliedModelingLib.proposition42Loss center datum +
      AppliedModelingLib.proposition42Gradient datum center * (candidate - center) +
      2 / 2 * (candidate - center) ^ 2) ∧
  AppliedModelingLib.IsJointlySmoothGradient AppliedModelingLib.proposition42Gradient 2 ∧
  (∀ (theta : { theta : ℝ // theta ∈ Set.Icc 0 1 } ),
    AppliedModelingLib.performativeRisk
      (AppliedModelingLib.correctedProposition42Model mu epsilon hmu hmuEpsilon) theta =
      1 / 4 - 2 * ↑theta * mu + (1 - 2 * epsilon) * ↑theta ^ 2) ∧
  ConcaveOn ℝ Set.univ fun (theta : ℝ) => 1 / 4 - 2 * theta * mu + (1 - 2 * epsilon) * theta ^ 2
```

Retained governing declarations

- [AppliedModelingLib.IsJointlySmoothGradient](#)
- [AppliedModelingLib.correctedProposition42Model](#)
- [AppliedModelingLib.performativeRisk](#)
- [AppliedModelingLib.proposition42BoolMetricSpace](#)
- [AppliedModelingLib.proposition42Gradient](#)
- [AppliedModelingLib.proposition42Loss](#)
- [PZMH20PerformativePrediction.IsFiniteWassersteinSensitive](#)

Lean-elaborated claim roles

```
1. conclusion: ∃ (mu : ℝ) (epsilon : ℝ) (hmu : |mu| ≤ 1 / 2) (hmuEpsilon : |mu + epsilon| ≤ 1 / 2),
  1 / 2 < epsilon ∧
  epsilon < 2 / 2 ∧
  PZMH20PerformativePrediction.IsFiniteWassersteinSensitive
    (AppliedModelingLib.correctedProposition42Model mu epsilon hmu hmuEpsilon) epsilon ∧
  (∀ (datum : Bool × Bool) (candidate : ℝ),
    HasGradientAt (fun (parameter : ℝ) => AppliedModelingLib.proposition42Loss parameter datum)
      (AppliedModelingLib.proposition42Gradient datum candidate) candidate) ∧
  (∀ (datum : Bool × Bool) (center candidate : ℝ),
    AppliedModelingLib.proposition42Loss candidate datum ≥
      AppliedModelingLib.proposition42Loss center datum +
      AppliedModelingLib.proposition42Gradient datum center * (candidate - center) +
      2 / 2 * (candidate - center) ^ 2) ∧
  AppliedModelingLib.IsJointlySmoothGradient AppliedModelingLib.proposition42Gradient 2 ∧
  (∀ (theta : { theta : ℝ // theta ∈ Set.Icc 0 1 } ),
    AppliedModelingLib.performativeRisk
      (AppliedModelingLib.correctedProposition42Model mu epsilon hmu hmuEpsilon) theta =
      1 / 4 - 2 * ↑theta * mu + (1 - 2 * epsilon) * ↑theta ^ 2) ∧
  ConcaveOn ℝ Set.univ fun (theta : ℝ) => 1 / 4 - 2 * theta * mu + (1 - 2 * epsilon) * theta ^ 2
```

Lean proof endpoint (built separately)

PZMH20PerformativePrediction.proposition42CorrectedFiniteWitness

Recorded source-to-Spec screening Verdict: matches

Reason: The source makes an existential claim that performative risk can be concave while the loss is jointly smooth and strongly convex and the distribution map is epsilon-sensitive below gamma/beta. The expanded target supplies a valid finite model with $\beta = \gamma = 2$ and $1/2 < \epsilon < 1$, ties proposition42Gradient to proposition42Loss by HasGradientAt, states the 2-strong-convexity inequality and 2-joint-smoothness condition, establishes epsilon-Wasserstein sensitivity, identifies the exact performative-risk polynomial on the parameter interval, and proves that polynomial concave. The displayed finite-law, expectation, metric, coupling, and sensitivity definitions give these clauses their stated meanings, so the witness establishes the source conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 418-423

Verbatim source input

population level: as in Theorems 3.5 and 3.8, the iterates θ_t contract toward θ_{PS} . This implies that θ_t eventually enters a δ -ball around θ_{PS} , for some large enough t . Once this happens,

contrary to population-level results, a contraction is no longer guaranteed due to the noise inherent in observing

[Next verbatim source excerpt]

Theorem 4.3. Suppose that the loss $\ell(z; \theta)$ is L_Z -Lipschitz in z , γ -strongly convex (A2), and that the distribution map

¹ In particular, they are guaranteed to exist over the extended real line, i.e. we allow $\theta \in (\mathbb{R} \cup \{\pm\infty\})^d$.

$D(\cdot)$ is ε -sensitive. Then, for every performatively stable point θ_{PS} and every performative optimum θ_{PO} :

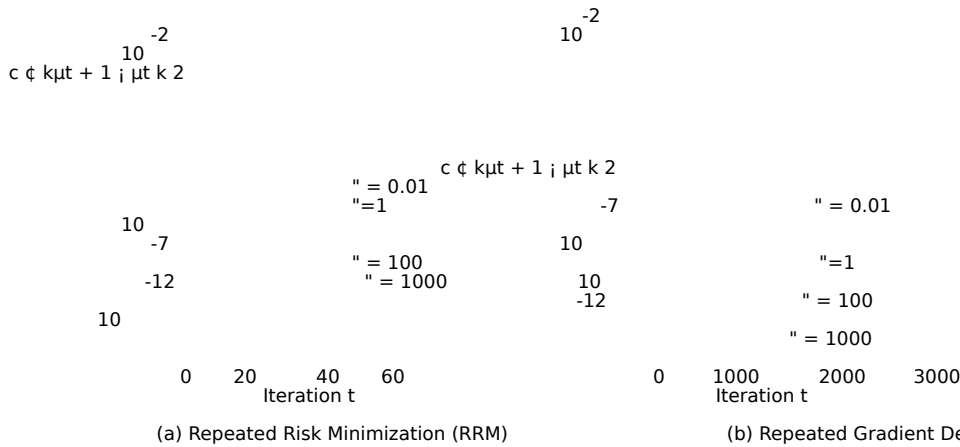
$$\| \theta_{PO} - \theta_{PS} \|_2 \leq \frac{2L_Z \varepsilon}{\gamma}.$$

response to f_θ . In fact, we see that this equilibrium notion exactly matches our definition of performative optimality:

$$f_\theta SE \text{ is a Stackelberg equilibrium} \Leftrightarrow \theta SE \in \arg \min_{\theta} PR(\theta).$$

[form-feed in source extraction]

Performative Prediction



Expanded PaperInterface specification

```

∀ {Parameter : Type u_1} {Data : Type u_2} [inst : NormedAddCommGroup Parameter]
[inst_1 : InnerProductSpace ℝ Parameter] [inst_2 : MeasurableSpace Data] [inst_3 : MetricSpace Data]
{domain : Set Parameter} (model : AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain)
(gradients : Data → ↑ domain → Parameter),
IsClosed domain →
Convex ℝ domain →
∀ {modulus : ℝ},
0 < modulus →
model.IsPointwiseGradientStronglyConvex gradient modulus →
∀ {constant : NNReal},
model.IsDataLossLipschitz constant →
∀ {sensitivity : ℝ},
0 ≤ sensitivity →
model.IsWassersteinSensitive sensitivity →
∀ (optimal stable : ↑ domain),
model.IsPerformativelyOptimal optimal →
model.IsPerformativelyStable stable →
|| ↑ optimal - ↑ stable || ≤ 2 * ↑ constant * sensitivity / modulus

```

Retained governing declarations

- [AppliedModelingLib.MeasurePerformativeModelOn](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsDataLossLipschitz](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsPerformativelyOptimal](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsPerformativelyStable](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsPointwiseGradientStronglyConvex](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsWassersteinSensitive](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: NormedAddCommGroup Parameter
4. parameter: InnerProductSpace $\mathbb{R}$ Parameter
5. parameter: MeasurableSpace Data
6. parameter: MetricSpace Data
7. parameter: Set Parameter
8. parameter: AppliedModelingLib.MeasurePerformativeModelOn Parameter Data domain
9. parameter: Data $\rightarrow \uparrow$ domain $\rightarrow$ Parameter
10. assumption: IsClosed domain
11. assumption: Convex $\mathbb{R}$ domain
12. parameter: $\mathbb{R}$
13. assumption: $0 < \text{modulus}$
14. assumption: model.IsPointwiseGradientStronglyConvex gradient modulus
15. parameter: NNReal
16. assumption: model.IsDataLossLipschitz constant
17. parameter: $\mathbb{R}$
18. assumption: $0 \leq \text{sensitivity}$
19. assumption: model.IsWassersteinSensitive sensitivity
20. parameter: $\uparrow$ domain
21. parameter: $\uparrow$ domain
22. assumption: model.IsPerformativelyOptimal optimal
23. assumption: model.IsPerformativelyStable stable
24. conclusion: Real.le $\uparrow \parallel \uparrow$ optimal - $\uparrow$ stable $\parallel (2 * \uparrow$ constant * sensitivity / modulus)

Lean proof endpoint (built separately)

PZMH20PerformativePrediction.measureWassersteinOptimumStableDistanceProof

Recorded source-to-Spec screening Verdict: matches

Reason: Compared theorem 4 3 optimum stable distance, cited publication:418-423,476-508, directly with the expanded Lean proposition, all 24 elaborated claim roles, and all 11 displayed material prerequisites. The source quantifies over every performatively stable point and every performative optimum and concludes $\text{norm}(\text{thetaPO}-\text{thetaPS}) \leq 2 * L_z * \epsilon / \gamma$. The target has precisely that universal pair quantification and constant; reversing the order of the independent pair binders changes no assertion. The supporting risk definitions use the integral under the deployed law with a separately evaluated parameter; optimality compares the diagonals over all of Theta, while stability fixes its own induced law and compares every feasible evaluated parameter. They do not impose uniqueness or assert existence. The model supplies real integrable expectations for all feasible pairs and no values outside Theta. The positive modulus and the complete A2 prerequisite require, for every datum and feasible center, the loss derivative within Theta together with the quadratic lower bound using that same derivative and $\gamma/2$. Its within-domain zero extension adds no off-domain differentiability requirement. The uniform NNReal data-Lipschitz constant and nonnegative sensitivity include their zero cases. The coupling support displays define a probability law with the prescribed marginals, integrable expected distance costs, and their real infimum; the finite-cost witness therefore supplies the source W1 inequality with the same epsilon times parameter distance. The target retains the closed convex feasible domain and adds neither A1 joint smoothness, a γ/β sensitivity threshold, finite-support restrictions, gradient integrability, nor an attained transport infimum. The concluding claim role is exactly the source distance bound, with no dropped conclusion or additional optimal/stable uniqueness premise. No corrected-target record is invoked.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: cited publication, lines 543-561

Verbatim source input

Corollary 5.1. Let the institution's loss be L_z - and L_θ - Lipschitz in z and θ respectively, β -jointly smooth (A1), and γ -strongly convex (A2). If the induced distribution map is ϵ -sensitive, with $\epsilon < \beta\gamma$, then RRM converges to a stable classifier θ^* that is $2L_z \epsilon(L_\theta + L_z \epsilon)\gamma^{-1}$ close in objective value to the Stackelberg equilibrium.

Input: base distribution D , classifier f_θ , cost function c and utility function u
Sampling procedure for $D(\theta)$:

1. Sample $(x, y) \sim D$
2. Compute $x_{BR} \leftarrow \arg \max_x u(x_0, \theta) - c(x_0, x)$
3. Output sample (x_{BR}, y)

sion classifier. We assume that individuals have linear utilities $u(\theta, x) = -h\theta, x_i$ and quadratic costs $c(x_0, x) = 2\epsilon kx - x^2$, where ϵ is a positive constant that regulates the cost incurred by changing features, and hence the sensitivity of the distribution map. Linear utilities indicate that agents wish to minimize their assigned probability of default. We divide the set of features into strategic features $S \subseteq [m - 1]$, such as the number of open credit lines, and non-strategic features (e.g., age). Solving the optimization problem described in Figure 2, the best response for an individual corresponds to the following update, $x_0^S = x^S - \epsilon\theta^S$, where $x^S, x_0^S, \theta^S \in R^{|S|}$. As per convention in the literature (Brückner et al., 2012; Hardt et al., 2016a; Milli et al., 2019), individual outcomes y are unaffected by manipulation.

Intuitively, this data-generating process is ϵ -sensitive since for a given choice of classifiers, f_θ and f_{θ^*} , an individual feature vector is shifted to $x^S - \epsilon\theta^S$ and to $x^S - \epsilon\theta^{*S}$, re-

Settled source reading The source parameter space Θ is represented by the subtype of its closed convex finite-dimensional Euclidean domain.

Expanded PaperInterface specification

```
∀ {Parameter : Type u_1} {Feature : Type u_2} {Label : Type u_3} {inst : NormedAddCommGroup Parameter}
[inst_1 : InnerProductSpace ℝ Parameter] [CompleteSpace Parameter] [FiniteDimensional ℝ Parameter]
[inst_4 : MeasurableSpace Feature] [inst_5 : MeasurableSpace Label] [inst_6 : MetricSpace (Feature × Label)]
{domain : Set Parameter}
(profile : PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain)
(gradient : Feature × Label → ↑domain → Parameter),
IsClosed domain →
Convex ℝ domain →
  ∀ {smoothness : NNReal},
  ((∀ (datum : Feature × Label) (first second : ↑domain),
    ||gradient datum first - gradient datum second|| ≤ ↑smoothness * dist ↑first ↑second) ∧
    ∀ (first second : Feature × Label) (parameter : ↑domain),
    ||gradient first parameter - gradient second parameter|| ≤ ↑smoothness * dist first second) →
  ∀ {modulus : ℝ},
  0 < modulus →
    profile.performativeModelOn.IsPointwiseGradientStronglyConvex gradient modulus →
      ∀ {dataConstant : parameterConstant : NNReal},
      profile.performativeModelOn.IsDataLossLipschitz dataConstant →
        (∀ (datum : Feature × Label), LipschitzWith parameterConstant (profile.loss datum)) →
          ∀ {sensitivity : ℝ},
          0 ≤ sensitivity →
            profile.performativeModelOn.IsWassersteinSensitive sensitivity →
              ∀ (update : ↑domain → ↑domain),
              (∀ (deployed candidate : ↑domain),
                profile.performativeModelOn.decoupledPerformativeRisk deployed (update deployed) ≤
                  profile.performativeModelOn.decoupledPerformativeRisk deployed candidate) →
                sensitivity * ↑smoothness / modulus < 1 →
                  ∀ (initial optimal : ↑domain),
                  profile.IsStackelbergEquilibrium optimal →
                    ∃ (stable : ↑domain),
                    profile.performativeModelOn.IsPerformativelyStable stable ∧
                    (∀ (other : ↑domain),
                      profile.performativeModelOn.IsPerformativelyStable other →
                        other = stable) ∧
                    Filter.Tendsto (fun (iteration : ℕ) => update^[iteration] initial)
                      Filter.atTop (nhds stable) ∧
                    (∀ (iteration : ℕ),
                      dist (update^[iteration] initial) stable ≤
                        (sensitivity * ↑smoothness / modulus) ^ iteration *
                        dist initial stable) ∧
                    profile.leaderRisk stable - profile.leaderRisk optimal ≤
                      2 * ↑dataConstant * sensitivity *
                        (↑parameterConstant + ↑dataConstant * sensitivity) /
                        modulus
```

Retained governing declarations

- [AppliedModelingLib.MeasurePerformativeModelOn.IsDataLossLipschitz](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsPerformativelyStable](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsPointwiseGradientStronglyConvex](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.IsWassersteinSensitive](#)
- [AppliedModelingLib.MeasurePerformativeModelOn.decoupledPerformativeRisk](#)
- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile](#)

- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.IsStackelbergEquilibrium](#)
- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.leaderRisk](#)
- [PZMH20PerformativePrediction.MeasureStrategicClassificationProfile.performativeModelOn](#)

Lean-elaborated claim roles

```

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: Type u_3
4. parameter: NormedAddCommGroup Parameter
5. parameter: InnerProductSpace ℝ Parameter
6. assumption: CompleteSpace Parameter
7. assumption: FiniteDimensional ℝ Parameter
8. parameter: MeasurableSpace Feature
9. parameter: MeasurableSpace Label
10. parameter: MetricSpace (Feature × Label)
11. parameter: Set Parameter
12. parameter: PZMH20PerformativePrediction.MeasureStrategicClassificationProfile Parameter Feature Label domain
13. parameter: Feature × Label → ↑domain → Parameter
14. assumption: IsClosed domain
15. assumption: Convex ℝ domain
16. parameter: NNReal
17. assumption: (∀ (datum : Feature × Label) (first second : ↑domain),
  ||gradient datum first - gradient datum second|| ≤ ↑smoothness * dist ↑first ↑second) ∧
  (∀ (first second : Feature × Label) (parameter : ↑domain),
  ||gradient first parameter - gradient second parameter|| ≤ ↑smoothness * dist first second)
18. parameter: ℝ
19. assumption: 0 < modulus
20. assumption: profile.performativeModelOn.IsPointwiseGradientStronglyConvex gradient modulus
21. parameter: NNReal
22. parameter: NNReal
23. assumption: profile.performativeModelOn.IsDataLossLipschitz dataConstant
24. assumption: (∀ (datum : Feature × Label), LipschitzWith parameterConstant (profile.loss datum))
25. parameter: ℝ
26. assumption: 0 ≤ sensitivity
27. assumption: profile.performativeModelOn.IsWassersteinSensitive sensitivity
28. parameter: ↑domain → ↑domain
29. assumption: (∀ (deployed candidate : ↑domain),
  profile.performativeModelOn.decoupledPerformativeRisk deployed (update deployed) ≤
  profile.performativeModelOn.decoupledPerformativeRisk deployed candidate)
30. assumption: sensitivity * ↑smoothness / modulus < 1
31. parameter: ↑domain
32. parameter: ↑domain
33. assumption: profile.IsStackelbergEquilibrium optimal
34. conclusion: ∃ (stable : ↑domain),
  profile.performativeModelOn.IsPerformativelyStable stable ∧
  (∀ (other : ↑domain), profile.performativeModelOn.IsPerformativelyStable other → other = stable) ∧
  Filter.Tendsto (fun (iteration : ℕ) => update^[iteration] initial) Filter.atTop (nhds stable) ∧
  (∀ (iteration : ℕ),
    dist (update^[iteration] initial) stable ≤
    (sensitivity * ↑smoothness / modulus) ^ iteration * dist initial stable) ∧
  profile.leaderRisk stable - profile.leaderRisk optimal ≤
  2 * ↑dataConstant * sensitivity * (↑parameterConstant + ↑dataConstant * sensitivity) / modulus

```

Lean proof endpoint (built separately)

PZMH20PerformativePrediction.measureWassersteinStableObjectiveGapProof

Recorded source-to-Spec screening Verdict: matches

Reason: Under the same recorded finite-dimensional source-domain convention, the exact Spec carries the on-domain data and parameter Lipschitz conditions, A1/A2, sensitivity, RRM convergence, stable and Stackelberg comparators, and the exact 2 Lz epsilon (Ltheta + Lz epsilon) over gamma objective-gap bound. This matches the literal Corollary 5.1 result without a whole-space restriction.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 315-323

Verbatim source input

if $\theta > 12$, and a point mass at 1 if $\theta < 21$. Clearly there is no performatively stable point, and RRM will simply result in the alternating sequence 1, 0, 1, 0, The performative optimum in this case is $\theta_{PO} = 12$.

To show convergence of retraining schemes, it is hence necessary to make a regularity assumption on $D(\cdot)$, such as ϵ -sensitivity. We are now ready to state our main result regarding the convergence of repeated risk minimization.

Proposition 3.6. Suppose that the distribution map $D(\cdot)$ is ϵ -sensitive with $\epsilon > 0$. RRM can fail to converge at all in any of the following cases, for any choice of $\beta, \gamma > 0$:

(a) The loss is β -jointly smooth and convex, but not strongly convex.

(b) The loss is γ -strongly convex, but not jointly smooth.

(c) The loss is β -jointly smooth and γ -strongly convex, but $\epsilon > \beta\gamma$.

Expanded PaperInterface specification

```

∀ (epsilon beta gamma : ℝ),
  0 < epsilon →
    0 < beta →
      0 < gamma →
        AppliedModelingLib.IsMeasureRepeatedRiskMinimizationOn
          (AppliedModelingLib.proposition36aMeasureModel epsilon beta) AppliedModelingLib.proposition36aDomain
          AppliedModelingLib.proposition36aRRMUpdate ∧
        (∀ (datum candidate : ℝ),
          HasGradientAt (fun (parameter : ℝ) => AppliedModelingLib.proposition36aMeasureLoss beta datum parameter)
            (AppliedModelingLib.proposition36aMeasureGradient beta datum candidate) candidate) ∧
        (∀ (datum : ℝ),
          ConvexOn ℝ Set.univ fun (candidate : ℝ) =>
            AppliedModelingLib.proposition36aMeasureLoss beta datum candidate) ∧
        AppliedModelingLib.IsJointlySmoothGradient (AppliedModelingLib.proposition36aMeasureGradient beta)
          beta.toNNReal ∧
        PZMH20PerformativePrediction.IsMeasureWassersteinSensitive
          (AppliedModelingLib.proposition36aMeasureModel epsilon beta) epsilon ∧
        (∀ (modulus : ℝ),
          0 < modulus →
            ¬PZMH20PerformativePrediction.IsMeasurePointwiseGradientStronglyConvex
              (AppliedModelingLib.proposition36aMeasureModel epsilon beta)
              (AppliedModelingLib.proposition36aMeasureGradient beta) modulus) ∧
        (∀ (theta : ℝ),
          AppliedModelingLib.IsMeasurePerformativelyStableOn
            (AppliedModelingLib.proposition36aMeasureModel epsilon beta)
            AppliedModelingLib.proposition36aDomain theta ↔
            theta = 0) ∧
        ¬∃ (limit : ℝ),
          Filter.Tendsto
            (fun (iteration : ℕ) => AppliedModelingLib.proposition36aRRMUpdate^[iteration] 1)
            Filter.atTop (nhds limit)

```

Retained governing declarations

- [AppliedModelingLib.IsJointlySmoothGradient](#)
- [AppliedModelingLib.IsMeasurePerformativelyStableOn](#)
- [AppliedModelingLib.IsMeasureRepeatedRiskMinimizationOn](#)
- [AppliedModelingLib.proposition36aDomain](#)
- [AppliedModelingLib.proposition36aMeasureGradient](#)
- [AppliedModelingLib.proposition36aMeasureLoss](#)
- [AppliedModelingLib.proposition36aMeasureModel](#)
- [AppliedModelingLib.proposition36aRRMUpdate](#)
- [PZMH20PerformativePrediction.IsMeasurePointwiseGradientStronglyConvex](#)
- [PZMH20PerformativePrediction.IsMeasureWassersteinSensitive](#)

Lean-elaborated claim roles

1. parameter: $\mathbb{R}$
2. parameter: $\mathbb{R}$
3. parameter: $\mathbb{R}$
4. assumption: $0 < \text{epsilon}$
5. assumption: $0 < \text{beta}$
6. assumption: $0 < \text{gamma}$
7. conclusion: $\text{AppliedModelingLib.IsMeasureRepeatedRiskMinimizationOn } (\text{AppliedModelingLib.proposition36aMeasureModel } \text{epsilon } \text{beta})$
 $\text{AppliedModelingLib.proposition36aDomain } \text{AppliedModelingLib.proposition36aRRMUpdate } \wedge$
 $(\forall (\text{datum candidate} : \mathbb{R}),$
 $\text{HasGradientAt } (\text{fun } (\text{parameter} : \mathbb{R}) \Rightarrow \text{AppliedModelingLib.proposition36aMeasureLoss } \text{beta } \text{datum } \text{parameter})$
 $(\text{AppliedModelingLib.proposition36aMeasureGradient } \text{beta } \text{datum } \text{candidate}) \text{ candidate}) \wedge$
 $(\forall (\text{datum} : \mathbb{R}),$
 $\text{ConvexOn } \mathbb{R} \text{ Set.univ fun } (\text{candidate} : \mathbb{R}) \Rightarrow \text{AppliedModelingLib.proposition36aMeasureLoss } \text{beta } \text{datum } \text{candidate}) \wedge$
 $\text{AppliedModelingLib.IsJointlySmoothGradient } (\text{AppliedModelingLib.proposition36aMeasureGradient } \text{beta}) \text{ beta.toNNReal } \wedge$

```

PZMH20PerformativePrediction.IsMeasureWassersteinSensitive
  (AppliedModelingLib.proposition36aMeasureModel epsilon beta) epsilon  $\wedge$ 
  ( $\forall$  (modulus :  $\mathbb{R}$ ),
    0 < modulus  $\rightarrow$ 
       $\neg$ PZMH20PerformativePrediction.IsMeasurePointwiseGradientStronglyConvex
        (AppliedModelingLib.proposition36aMeasureModel epsilon beta)
        (AppliedModelingLib.proposition36aMeasureGradient beta) modulus)  $\wedge$ 
  ( $\forall$  (theta :  $\mathbb{R}$ ),
    AppliedModelingLib.IsMeasurePerformativelyStableOn
      (AppliedModelingLib.proposition36aMeasureModel epsilon beta) AppliedModelingLib.proposition36aDomain
      theta  $\leftrightarrow$ 
      theta = 0)  $\wedge$ 
 $\neg \exists$  (limit :  $\mathbb{R}$ ),
  Filter.Tendsto (fun (iteration :  $\mathbb{N}$ ) => AppliedModelingLib.proposition36aRRMUpdate^[iteration] 1)
    Filter.atTop (nhds limit)

```

Lean proof endpoint (built separately)

PZMH20PerformativePrediction.proposition36aExactDiracCounterexample

Recorded source-to-Spec screening Verdict: matches
Reason: Independent review: the exact Dirac linear-loss witness gives beta-smooth convex, non-strongly-convex RRM nonconvergence for the positive source constants.
Reviewer check: ☐ Matches source input
 If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 8

Source locator: cited publication, lines 315-323

Verbatim source input

if $\theta > 12$, and a point mass at 1 if $\theta < 21$. Clearly there is no performatively stable point, and RRM will simply result in the alternating sequence 1, 0, 1, 0, The performative optimum in this case is $\theta_{PO} = 12$.

To show convergence of retraining schemes, it is hence necessary to make a regularity assumption on $D(\cdot)$, such as ϵ -sensitivity. We are now ready to state our main result regarding the convergence of repeated risk minimization.

Proposition 3.6. Suppose that the distribution map $D(\cdot)$ is ϵ -sensitive with $\epsilon > 0$. RRM can fail to converge at all in any of the following cases, for any choice of $\beta, \gamma > 0$:

- (a) The loss is β -jointly smooth and convex, but not strongly convex.
- (b) The loss is γ -strongly convex, but not jointly smooth.
- (c) The loss is β -jointly smooth and γ -strongly convex, but $\epsilon > \beta\gamma$.

Expanded PaperInterface specification

$\forall$ (epsilon beta gamma : $\mathbb{R}$),
0 < epsilon $\rightarrow$
0 < beta $\rightarrow$
0 < gamma $\rightarrow$
 $\exists$ (penalty : $\mathbb{R}$),
AppliedModelingLib.IsProposition36bLargePenalty epsilon gamma penalty $\wedge$
PZMH20PerformativePrediction.proposition36bExactDiracEndpointCounterexampleSpec epsilon gamma penalty

Retained governing declarations

- [AppliedModelingLib.IsProposition36bLargePenalty](#)
- [PZMH20PerformativePrediction.proposition36bExactDiracEndpointCounterexampleSpec](#)

Lean-elaborated claim roles

1. parameter: $\mathbb{R}$
2. parameter: $\mathbb{R}$
3. parameter: $\mathbb{R}$
4. assumption: 0 < epsilon
5. assumption: 0 < beta
6. assumption: 0 < gamma
7. conclusion: $\exists$ (penalty : $\mathbb{R}$),
AppliedModelingLib.IsProposition36bLargePenalty epsilon gamma penalty $\wedge$
PZMH20PerformativePrediction.proposition36bExactDiracEndpointCounterexampleSpec epsilon gamma penalty

Lean proof endpoint (built separately)

PZMH20PerformativePrediction.proposition36bSourceUniformCounterexample

Recorded source-to-Spec screening Verdict: matches
Reason: Independent review: the uniform large-penalty witness supplies the positive-constant, strongly-convex nonsmooth RRM nonconvergence alternative.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

Review row 9

Source locator: cited publication, lines 315-323

Verbatim source input

if $\theta > 12$, and a point mass at 1 if $\theta < 21$. Clearly there is no performatively stable point, and RRM will simply result in the alternating sequence 1, 0, 1, 0, ... The performative optimum in this case is $\theta_{PO} = 12$.

To show convergence of retraining schemes, it is hence necessary to make a regularity assumption on $D(\cdot)$, such as ϵ -sensitivity. We are now ready to state our main result regarding the convergence of repeated risk minimization.

Proposition 3.6. Suppose that the distribution map $D(\cdot)$ is ϵ -sensitive with $\epsilon > 0$. RRM can fail to converge at all in any of the following cases, for any choice of $\beta, \gamma > 0$:

- (a) The loss is β -jointly smooth and convex, but not strongly convex.
- (b) The loss is γ -strongly convex, but not jointly smooth.
- (c) The loss is β -jointly smooth and γ -strongly convex, but $\epsilon > \beta\gamma$.

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For every positive epsilon, beta, and gamma with gamma at most beta and epsilon at least gamma divided by beta, there is an epsilon-sensitive $\hookrightarrow$ performative model with beta-jointly-smooth and gamma-strongly-convex loss whose repeated-risk-minimization trajectory diverges.

Archival source anchor: cited publication, lines 315-323

Expanded PaperInterface specification

```

∀ (epsilon beta gamma : ℝ),
  0 < epsilon →
    0 < beta →
      0 < gamma →
        gamma ≤ beta →
          gamma / beta ≤ epsilon →
            AppliedModelingLib.IsMeasureRepeatedRiskMinimization
              (AppliedModelingLib.proposition36cRegularityMeasureModel epsilon beta gamma)
              (AppliedModelingLib.proposition36cRegularityRRMUpdate epsilon beta gamma) ∧
            AppliedModelingLib.IsJointlySmoothGradient
              (AppliedModelingLib.proposition36cRegularityGradient beta gamma) beta.toNNReal ∧
            PZMH20PerformativePrediction.IsMeasurePointwiseGradientStronglyConvex
              (AppliedModelingLib.proposition36cRegularityMeasureModel epsilon beta gamma)
              (AppliedModelingLib.proposition36cRegularityGradient beta gamma) gamma ∧
            (∀ (datum candidate : ℝ),
              HasGradientAt
                (fun (parameter : ℝ) =>
                  AppliedModelingLib.proposition36cRegularityLoss beta gamma datum parameter)
                (AppliedModelingLib.proposition36cRegularityGradient beta gamma datum candidate) candidate) ∧
            PZMH20PerformativePrediction.IsMeasureWassersteinSensitive
              (AppliedModelingLib.proposition36cRegularityMeasureModel epsilon beta gamma) epsilon ∧
            ∃ (initial : ℝ),
              Filter.Tendsto
                (fun (iteration : ℕ) =>
                  (AppliedModelingLib.proposition36cRegularityRRMUpdate epsilon beta gamma)^[iteration]
                  initial)
              Filter.atTop Filter.atTop

```

Retained governing declarations

- [AppliedModelingLib.IsJointlySmoothGradient](#)
- [AppliedModelingLib.IsMeasureRepeatedRiskMinimization](#)
- [AppliedModelingLib.proposition36cRegularityGradient](#)
- [AppliedModelingLib.proposition36cRegularityLoss](#)
- [AppliedModelingLib.proposition36cRegularityMeasureModel](#)
- [AppliedModelingLib.proposition36cRegularityRRMUpdate](#)
- [PZMH20PerformativePrediction.IsMeasurePointwiseGradientStronglyConvex](#)
- [PZMH20PerformativePrediction.IsMeasureWassersteinSensitive](#)

Lean-elaborated claim roles

1. parameter: $\mathbb{R}$
2. parameter: $\mathbb{R}$
3. parameter: $\mathbb{R}$
4. assumption: $0 < \text{epsilon}$
5. assumption: $0 < \text{beta}$
6. assumption: $0 < \text{gamma}$
7. assumption: $\text{gamma} \leq \text{beta}$
8. assumption: $\text{gamma} / \text{beta} \leq \text{epsilon}$
9. conclusion: $\text{AppliedModelingLib.IsMeasureRepeatedRiskMinimization}$
 $(\text{AppliedModelingLib.proposition36cRegularityMeasureModel } \text{epsilon } \text{beta } \text{gamma})$
 $(\text{AppliedModelingLib.proposition36cRegularityRRMUpdate } \text{epsilon } \text{beta } \text{gamma}) \wedge$
 $\text{AppliedModelingLib.IsJointlySmoothGradient } (\text{AppliedModelingLib.proposition36cRegularityGradient } \text{beta } \text{gamma})$
 $\text{beta.toNNReal } \wedge$

```

PZMH20PerformativePrediction.IsMeasurePointwiseGradientStronglyConvex
  (AppliedModelingLib.proposition36cRegularityMeasureModel epsilon beta gamma)
  (AppliedModelingLib.proposition36cRegularityGradient beta gamma) gamma  $\wedge$ 
( $\forall$  (datum candidate :  $\mathbb{R}$ ),
  HasGradientAt
    (fun (parameter :  $\mathbb{R}$ ) => AppliedModelingLib.proposition36cRegularityLoss beta gamma datum parameter)
    (AppliedModelingLib.proposition36cRegularityGradient beta gamma datum candidate) candidate)  $\wedge$ 
PZMH20PerformativePrediction.IsMeasureWassersteinSensitive
  (AppliedModelingLib.proposition36cRegularityMeasureModel epsilon beta gamma) epsilon  $\wedge$ 
 $\exists$  (initial :  $\mathbb{R}$ ),
  Filter.Tendsto
    (fun (iteration :  $\mathbb{N}$ ) =>
      (AppliedModelingLib.proposition36cRegularityRRMUpdate epsilon beta gamma)^[iteration] initial)
    Filter.atTop Filter.atTop

```

Lean proof endpoint (built separately)

PZMH20PerformativePrediction.proposition36cFeasibleUniformCounterexample

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: Compared the selected proposition_3_6_counterexamples_c source item, its identity-bound approved correction for proposition-3.6c-arbitrary-regularity-constants, the complete expanded Spec and claim roles, and all 17 displayed prerequisites. PMLR PDF page 5 visibly states $\epsilon \geq \gamma/\beta$; the extracted text loses the non-strict glyph and fraction layout. The approved correction expressly adds $\gamma \leq \beta$ and disclaims archival equivalence. For every positive ϵ , β , γ satisfying these two inequalities, the displayed real-valued model has $D(\theta) = \delta (1 + \epsilon \theta)$, loss $\gamma x^2/2 - \beta z x$, and actual gradient $\gamma x - \beta z$. Its parameter-gradient and data-gradient Lipschitz constants are respectively $\gamma \leq \beta$ and β , and its γ -strong-convexity inequality holds with the required $\gamma/2$ remainder. Dirac laws make every frozen loss integrable and give $W1 = \epsilon |\theta - \theta'|$ through the displayed finite-cost coupling definition. Frozen risk is minimized over all real candidates by $G(\theta) = (\beta/\gamma)(1 + \epsilon \theta)$, exactly the expanded RRM predicate. Writing $G(\theta) = a\theta + b$ gives $a = \epsilon \beta/\gamma \geq 1$ and $b = \beta/\gamma > 0$; from initial 0, the iterates are at least $n b$ and tend to $+\infty$. Equality $\epsilon = \gamma/\beta$ is therefore covered by positive translation. The complete quantified claim matches the approved corrected target, with no additional restriction on its admissible constants and no archival-equivalence claim.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 373-415

Verbatim source input

Theorem 3.10. Suppose that the loss $\ell(z; \theta)$ is β -jointly smooth (A1) and γ -strongly convex (A2), and that there exist $\alpha > 1, \mu > 0$ such that $\xi_{\alpha, \mu} = \mathbb{E} \int_0^1 \mathbb{E}_{\theta \sim D(\theta)} \ell(\theta) dD(\theta)$ is finite. If the distribution map $D(\cdot)$ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then RGD with step size $\eta \leq \frac{1}{6(\beta + \gamma)}$ satisfies the following:

(a) $kG_d(\theta) - G_d(\theta_0) \leq k^2$ for all $k \leq \frac{1}{\epsilon}$ if the map $D(\cdot)$ is ϵ -sensitive with $\epsilon < 2\beta$, then with probability $1 - p$, RERM satisfies,

(b) The iterates θ_t of RGD converge to a unique performatively stable point θ_{PS} at a linear rate,

(b) If the map $D(\cdot)$ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then with probability $1 - p$, REGD with satisfies,

The conclusion of Theorem 3.8 is a strict generalization of a classical optimization result which considers a static objective, in which case the rate of contraction is $1 - \eta\beta$ for a constant choice of step size $\eta \leq \frac{1}{6(\beta + \gamma)}$.

(see for example Theorem 2.1.15 in Nesterov (2013) or Lemma 3.7 in Hardt et al. (2016b)). Our rate exactly matches this standard result in the case that $\epsilon = 0$. The proof of Theorem 3.8 can be found in Appendix E.3.

Proof sketch. The basic idea behind these results is the following. While $k\theta_t - \theta_{PS} \geq \delta$, the sample size nt is sufficiently large to ensure a behavior similar to that on a

[Next verbatim source excerpt]

Throughout our presentation, we focus on predictive models that are parametrized by a vector $\theta \in \Theta$, where the parameter space $\Theta \subseteq \mathbb{R}^d$ is a closed, convex set. We use capital letters to denote random variables and their lowercase counterparts to denote realizations of these variables. We consider instances $z = (x, y)$ defined as feature, outcome pairs, where $x \in \mathbb{R}^{m-1}$ and $y \in \mathbb{R}$. Whenever we define a

Here, ϵ represents the performative aspect of the model. If $\epsilon = 0$, outcomes are independent of the assigned scores and the problem reduces to standard supervised learning where the optimal predictor is the conditional expectation $f_{\theta SL}(x) = \mathbb{E}[Y | X = x] = \mu + \mu x$, with $\theta SL = \mu$. In the performative setting with $\epsilon \neq 0$, the optimal prediction θPO balances between its predictive accuracy as well

[Next verbatim source excerpt]

performatively optimal classifiers need not be performatively stable and performatively stable classifiers need not be performatively optimal. We illustrate this point in the context

Definition 3.1 (ϵ -sensitivity). We say that a distribution map $D(\cdot)$ is ϵ -sensitive if for all $\theta, \theta_0 \in \Theta$:

of our previous biased coin toss example. Example 2.2 (continued). Consider again our model of a biased coin toss. In order for a classifier f_θ to be performatively optimal, we need

where W_1 denotes the earth mover's distance.

[Next verbatim source excerpt]

In addition to this assumption on the distribution map, we will often make standard assumptions on the loss function $\ell(z; \theta)$ which hold for broad classes of losses. Each technical result to follow will invoke some subset of them. To simplify our presentation, let $Z = \cup_{\theta \in \Theta} \text{supp}(D(\theta))$.

(A1) (joint smoothness) A loss function $\ell(z; \theta)$ is β -jointly smooth if $\forall \theta, \theta_0 \in \Theta$ and $z, z_0 \in Z$:

(A2) (strong convexity) A loss function $\ell(z; \theta)$ is γ -strongly convex if $\forall \theta, \theta_0 \in \Theta$ and $z \in Z$:

The main message of this theorem is that in performative prediction, if the loss function is sufficiently "nice" and the distribution map is sufficiently (in)sensitive, then one need only retrain a classifier a small number of times before it converges to a unique stable point. Here, we provide a proof sketch illustrating the main ideas behind the theorem and defer the full proof to Appendix E.1.

Proof Sketch. Fix $\theta, \theta_0 \in \Theta$. Let $f(\phi) = \mathbb{E}[\ell(Z; \phi)]$ and $f_0(\phi) = \mathbb{E}[\ell(Z; \phi)]$. By applying standard properties of strong convexity and the fact that $G(\theta)$ is the unique

If $\gamma = 0$, this last assumption is equivalent to convexity. We

[Next verbatim source excerpt]

globally optimal. However, in our framework, strong convexity is in fact necessary to guarantee convergence of repeated risk minimization, even for arbitrarily smooth losses and an arbitrarily small sensitivity parameter.

3.3. Repeated Gradient Descent

Theorem 3.5 demonstrates that repeated risk minimization converges to a unique stable point if the sensitivity parameter ϵ is small enough. However, implementing RRM requires access to an exact optimization oracle. We now relax this requirement and demonstrate how a simple gradient descent algorithm also converges to a unique stable point.

Definition 3.7 (RGD). Repeated gradient descent (RGD) is

the procedure where, starting from an initial model f_{θ_0} we perform the following sequence of updates for $t > 0$:

$$\theta_{t+1} = \text{Ggd}(\theta_t) = \Pi_{\Theta}(\theta_t - \eta \nabla \mathbb{E}_{Z \sim D(\theta_t)} r_{\theta}(Z; \theta_t)),$$

$$\theta_{t+1} = \text{Gngdt}(\theta_t) = \Pi_{\Theta}(\theta_t - \eta \nabla \mathbb{E}_{Z \sim D(\theta_t)} r_{\theta}(Z; \theta_t)).$$

where $\eta > 0$ is a fixed step size and Π_{Θ} denotes the Eu-

clidean projection operator onto Θ .

Here, $\eta > 0$ is a step size and Π_{Θ} denotes the Euclidean projection operator onto Θ .

[Next verbatim source excerpt]

Lemma 3.7 in Hardt et al. (2016b)). Our rate exactly matches this standard result in the case that $\epsilon = 0$. The proof of Theorem 3.8 can be found in Appendix E.3.

[form-feed in source extraction]

Performative Prediction

population level: as in Theorems 3.5 and 3.8, the iterates θ_t contract toward θ_{PS} . This implies that θ_t eventually enters a δ -ball around θ_{PS} , for some large enough t . Once this hap-

pens, contrary to population-level results, a contraction is no longer guaranteed due to the noise inherent in observing only finite-sample approximations of $D(\theta_t)$. Nevertheless, the sample size n_t is sufficiently large to ensure that θ_t cannot escape a δ -ball around θ_{PS} either.

Proof sketch. The basic idea behind these results is the following. While $\|\theta_t - \theta_{PS}\|_2 > \delta$, the sample size n_t is sufficiently large to ensure a behavior similar to that on a

$D(\cdot)$ is ϵ -sensitive. Then, for every performatively stable point θ_{PS} and every performative optimum θ_{PO} :

$$2L\epsilon \leq \|\theta_{PO} - \theta_{PS}\|_2^2 \leq \frac{1}{\gamma}.$$

This result shows that in cases where repeated risk minimization converges to a stable point, the resulting classifier

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For every positive data dimension, use the explicit all-dimensional selected-shell count schedule and inverse-square confidence allocation. Suppose the

- input data provide a shared bounded exponential-moment and tail envelope along every adaptive history; measurable adaptive sampling and
- raw-Wasserstein bad events; a convex parameter domain; A1's data-gradient bound and A2's pointwise strong-convexity lower model on that domain;
- empirical-risk integrability; a checked RERM trajectory and population risk-minimizing update with a performatively stable point; and the stated
- sensitivity, numerical W1 budget, absorbing-radius, and entry conditions. Then there is a finite selected-shell gate scale and an entry iteration such
- that, under the adaptive conditionally IID sampling law, with probability at least one minus p , every later RERM iterate lies within the prescribed radius
- of the stable point. The selected-shell construction derives the raw W1 roundwise failure bound and deterministic threshold internally. No
- data-coordinate loss-Lipschitz bridge, gradient-integrability premise, pointwise first-order certificate, or caller-supplied selected-shell certificate is
- assumed. The companion exact count bounds are regime-specific: dimension one has inverse-square times one radius logarithm, the separately
- connected sharp dimension-two trajectory has inverse-square times squared radius logarithm, and the supercritical schedule has inverse-dimension
- power. The printed single all-dimension rate is not claimed.

Archival source anchor: cited publication, lines 373-415

Expanded PaperInterface specification

```

∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(data : PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter),
∃ (gateScale : ℕ),
data.scaledDeviation ≤ AppliedModelingLib.Probability.pOneFournierGuillinHeadGateRadius data.eta gateScale →
(AppliedModelingLib.measurePerformativeHeterogeneousBatchTraceInfiniteLaw data.model data.sampling
(AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule data.dimension
data.cutoff data.eta data.alpha data.gamma data.tailBound data.deviation data.scaledDeviation data.p
data.headTolerance)
data.deployedOfHistory data.hmeasurableDeployed).real
{trace :
(i : ℕ) →
AppliedModelingLib.HeterogeneousBatchTraceCoordinate
(fun (round : ℕ) =>
Fin
(AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule
data.dimension data.cutoff data.eta data.alpha data.gamma data.tailBound data.deviation
data.scaledDeviation data.p data.headTolerance round))
(EuclideanSpace ℝ (Fin data.dimension))) i |
∀ (iteration : ℕ),
data.entryIteration ≤ iteration →
dist
(data.deployedOfHistory iteration

```

```

      (AppliedModelingLib.heterogeneousBatchTraceOfInfiniteTrace iteration trace))
    data.stable ≤
    data.radius} ≥
1 - data.p

```

Retained governing declarations

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.Probability.pOneFournierGuillinHeadGateRadius](#)
- [AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace](#)
- [AppliedModelingLib.heterogeneousBatchTraceOfInfiniteTrace](#)
- [AppliedModelingLib.measurePerformativeHeterogeneousBatchTraceInfiniteLaw](#)
- [AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule](#)
- [PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData](#)

Lean-elaborated claim roles

```

1. parameter: Type u_1
2. parameter: MeasurableSpace Parameter
3. parameter: NormedAddCommGroup Parameter
4. parameter: InnerProductSpace ℝ Parameter
5. assumption: CompleteSpace Parameter
6. parameter: PZMH20PerformativePrediction.Theorem310RERMRawWassersteinAllDimensionalSamplingRecoveryData Parameter
7. conclusion: ∃ (gateScale : ℕ),
  data.scaledDeviation ≤ AppliedModelingLib.Probability.pOneFournierGuillinHeadGateRadius data.eta gateScale →
  (AppliedModelingLib.measurePerformativeHeterogeneousBatchTraceInfiniteLaw data.model data.sampling
   (AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule data.dimension
    data.cutoff data.eta data.alpha data.gamma data.tailBound data.deviation data.scaledDeviation data.p
    data.headTolerance)
   data.deployedOfHistory data.hmeasurableDeployed).real
{trace :
 (i : ℕ) →
  AppliedModelingLib.HeterogeneousBatchTraceCoordinate
  (fun (round : ℕ) =>
    Fin
    (AppliedModelingLib.pOneFournierGuillinTheorem310AllDimensionalConstantHeadCountSchedule
     data.dimension data.cutoff data.eta data.alpha data.gamma data.tailBound data.deviation
     data.scaledDeviation data.p data.headTolerance round))
   (EuclideanSpace ℝ (Fin data.dimension))) i |
∀ (iteration : ℕ),
  data.entryIteration ≤ iteration →
  dist
    (data.deployedOfHistory iteration
     (AppliedModelingLib.heterogeneousBatchTraceOfInfiniteTrace iteration trace))
    data.stable ≤
    data.radius} ≥
1 - data.p

```

Lean proof endpoint (built separately)

PZMH20PerformativePrediction.theorem310RERMRawWassersteinAllDimensionalSamplingRecovery

Recorded source-to-Spec screening Verdict: matches approved corrected target

Reason: The previously reviewed direct raw-Wasserstein RERM Data-plus-Spec surface is unchanged: it fixes the positive-dimensional selected-shell schedule, adaptive sampling/event measurability, uniform history-wise exponential-moment/tail envelope, convex-domain A1/A2 conditions, integrability, RERM/population/stability, sensitivity, numerical budget, and entry inputs. It derives the gate scale and all-later containment with probability at least $1-p$, while deriving rather than assuming the raw-event finite-IID bound and threshold. The approved correction retains no loss-Lipschitz, gradient-integrability, pointwise first-order, or caller-supplied selected-shell certificate premise.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: cited publication, lines 373-415

Verbatim source input

Theorem 3.10. Suppose that the loss $\ell(z; \theta)$ is β -jointly smooth (A1) and γ -strongly convex (A2), and that there exist $\alpha > 1, \mu > 0$ such that $\xi_{\alpha, \mu} = \mathbb{E} \int_0^1 \mathbb{E}_{\theta \sim D(\theta)} \ell(\theta) dD(\theta)$ is finite. If the distribution map $D(\cdot)$ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then RGD with step size $\eta \leq \frac{1}{6(\beta + \gamma)}$ satisfies the following:

(a) $kG_d(\theta) - G_d(\theta_0) \leq k^2$ for all $k \leq \frac{1}{\epsilon}$ and $\theta_0 \in \Theta$. Let $\delta \in (0, 1)$ be a radius of convergence.

(a) If the map $D(\cdot)$ is ϵ -sensitive with $\epsilon < 2\beta$, then with probability $1 - p$, RERM satisfies,

(b) The iterates θ_t of RGD converge to a unique performatively stable point θ_{PS} at a linear rate,

(b) If the map $D(\cdot)$ is ϵ -sensitive with $\epsilon < (\beta + \gamma)(1 + 1.5\eta\beta)$, then with probability $1 - p$, REGD with satisfies,

The conclusion of Theorem 3.8 is a strict generalization of a classical optimization result which considers a static objective, in which case the rate of contraction is $1 - \eta\beta$ for a constant choice of step size $\eta \leq \frac{1}{6(\beta + \gamma)}$.

(see for example Theorem 2.1.15 in Nesterov (2013) or Lemma 3.7 in Hardt et al. (2016b)). Our rate exactly matches this standard result in the case that $\epsilon = 0$. The proof of Theorem 3.8 can be found in Appendix E.3.

Proof sketch. The basic idea behind these results is the following. While $k\theta_t - \theta_{PS} \leq \delta$, the sample size n_t is sufficiently large to ensure a behavior similar to that on a

[Next verbatim source excerpt]

Throughout our presentation, we focus on predictive models that are parametrized by a vector $\theta \in \Theta$, where the parameter space $\Theta \subseteq \mathbb{R}^d$ is a closed, convex set. We use capital letters to denote random variables and their lowercase counterparts to denote realizations of these variables. We consider instances $z = (x, y)$ defined as feature, outcome pairs, where $x \in \mathbb{R}^{m-1}$ and $y \in \mathbb{R}$. Whenever we define a

Here, ϵ represents the performative aspect of the model. If $\epsilon = 0$, outcomes are independent of the assigned scores and the problem reduces to standard supervised learning where the optimal predictor is the conditional expectation $f_{\theta SL}(x) = \mathbb{E}[Y | X = x] = \mu + \mu x$, with $\theta_{SL} = \mu$. In the performative setting with $\epsilon \neq 0$, the optimal prediction θ_{PO} balances between its predictive accuracy as well

[Next verbatim source excerpt]

performatively optimal classifiers need not be performatively stable and performatively stable classifiers need not be performatively optimal. We illustrate this point in the context

of our previous biased coin toss example. Example 2.2 (continued). Consider again our model of a biased coin toss. In order for a classifier f_{θ} to be performatively optimal, we need

[Next verbatim source excerpt]

In addition to this assumption on the distribution map, we will often make standard assumptions on the loss function $\ell(z; \theta)$ which hold for broad classes of losses. Each technical result to follow will invoke some subset of them. To simplify our presentation, let $Z = \cup_{\theta \in \Theta} \text{supp}(D(\theta))$.

(A1) (joint smoothness) A loss function $\ell(z; \theta)$ is β -jointly smooth if $\forall \theta, \theta_0 \in \Theta$ and $z, z_0 \in Z$:

(A2) (strong convexity) A loss function $\ell(z; \theta)$ is γ -strongly convex if $\forall \theta, \theta_0 \in \Theta$ and $z \in Z$:

If $\gamma = 0$, this last assumption is equivalent to convexity. We

[Next verbatim source excerpt]

globally optimal. However, in our framework, strong convexity is in fact necessary to guarantee convergence of repeated risk minimization, even for arbitrarily smooth losses and an arbitrarily small sensitivity parameter.

We now extend our main results regarding the convergence of RRM and RGD to the finite-sample regime. To do so, we leverage the fact that, under mild regularity conditions, the empirical distribution D_n given by n samples drawn i.i.d. from a true distribution D is with high probability close to D in earth mover's distance (Fournier & Guillin, 2015).

3.3. Repeated Gradient Descent

Theorem 3.5 demonstrates that repeated risk minimization converges to a unique stable point if the sensitivity parameter ϵ is small enough. However, implementing RRM requires access to an exact optimization oracle. We now relax this requirement and demonstrate how a simple gradient descent algorithm also converges to a unique stable point.

Definition 3.7 (RGD). Repeated gradient descent (RGD) is

the procedure where, starting from an initial model θ_0 we perform the following sequence of updates for $t > 0$:

$$\theta_{t+1} = \text{Ggd}(\theta_t) = \Pi_{Z \sim D(\theta_t)} \theta_t - \eta \nabla \mathbb{E}_{Z \sim D(\theta_t)} r_\theta(Z; \theta_t),$$

where $\eta > 0$ is a fixed step size and Π_Θ denotes the Euclidean projection operator onto Θ .

Here, $\eta > 0$ is a step size and Π_Θ denotes the Euclidean projection operator onto Θ .

[Next verbatim source excerpt]

Lemma 3.7 in Hardt et al. (2016b)). Our rate exactly matches this standard result in the case that $\epsilon = 0$. The proof of Theorem 3.8 can be found in Appendix E.3.

[form-feed in source extraction]

Performative Prediction

population level: as in Theorems 3.5 and 3.8, the iterates θ_t contract toward θ_{PS} . This implies that θ_t eventually enters a δ -ball around θ_{PS} , for some large enough t . Once this happens,

contrary to population-level results, a contraction is no longer guaranteed due to the noise inherent in observing only finite-sample approximations of $D(\theta_t)$. Nevertheless, the sample size nt is sufficiently large to ensure that θ_t cannot escape a δ -ball around θ_{PS} either.

Proof sketch. The basic idea behind these results is the following. While $\|\theta_t - \theta_{PS}\|_2 > \delta$, the sample size nt is sufficiently large to ensure a behavior similar to that on a

$D(\cdot)$ is ϵ -sensitive. Then, for every performatively stable point θ_{PS} and every performative optimum θ_{PO} :

$$2L_Z \epsilon \|\theta_{PO} - \theta_{PS}\|_2 \leq \epsilon.$$

This result shows that in cases where repeated risk minimization converges to a stable point, the resulting classifier

Formalized review target The archival source above is not asserted equivalent to this formalized target.

In data dimension greater than two, use the explicit supercritical batch-count schedule, evaluated at the canonical capped deviation, together with the inverse-square confidence schedule. Suppose the input data provide the stated joint-gradient smoothness, pointwise strong convexity, Wasserstein sensitivity, step-size, and small-sensitivity conditions; a convex parameter domain with a variational nonexpansive projection; a shared bounded exponential-moment and tail envelope along every adaptive history; measurable adaptive sampling and bad events; population-gradient integrability, local loss measurability, pointwise loss differentiation, and empirical-gradient integrability; a checked REGD trajectory with a performatively stable point; and the stated numerical W_1 budget, outer-contraction, absorbing-radius, and initial-distance conditions. Then there exists an entry iteration such that, under the adaptive conditionally IID sampling law, with probability at least one minus p , every REGD iterate at or after that entry iteration lies within the prescribed radius of the stable point. The model and expected-loss integrability are defined for every ambient parameter. Joint-gradient smoothness and pointwise strong convexity quantify over all ambient parameters and data points; Wasserstein sensitivity, gradient integrability, and loss differentiation quantify over all ambient parameters. These global hypotheses are stronger in scope than the source assumptions on the parameter domain and the union of its induced supports; their necessity for the source conclusion is unresolved.

Archival source anchor: cited publication, lines 373-415

Expanded PaperInterface specification

```

∀ {Parameter : Type u_1} [inst : MeasurableSpace Parameter] [inst_1 : NormedAddCommGroup Parameter]
[inst_2 : InnerProductSpace ℝ Parameter] [inst_3 : CompleteSpace Parameter]
(data : PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter),
∃ (entryIteration : ℕ),
  (AppliedModelingLib.measurePerformativeHeterogeneousBatchTraceInfiniteLaw data.model data.sampling
  (AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule data.dimension
  data.cutoff data.eta data.alpha data.gamma data.momentBound data.tailBound data.scaledDeviation data.p
  data.headTolerance)
  data.deployedOfHistory data.hmeasurableDeployed).real
{trace :
  (i : ℕ) →
    AppliedModelingLib.HeterogeneousBatchTraceCoordinate
    (fun (round : ℕ) =>
      Fin
        (AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule
        data.dimension data.cutoff data.eta data.alpha data.gamma data.momentBound data.tailBound
        data.scaledDeviation data.p data.headTolerance round))
    (EuclideanSpace ℝ (Fin data.dimension))) i |
  ∀ (iteration : ℕ),
    entryIteration ≤ iteration →
      dist
        (data.deployedOfHistory iteration
        (AppliedModelingLib.heterogeneousBatchTraceOfInfiniteTrace iteration trace))

```

```

    data.stable ≤
    data.radius} ≥
1 - data.p

```

Retained governing declarations

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace](#)
- [AppliedModelingLib.heterogeneousBatchTraceOfInfiniteTrace](#)
- [AppliedModelingLib.measurePerformativeHeterogeneousBatchTraceInfiniteLaw](#)
- [AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule](#)
- [PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData](#)

Lean-elaborated claim roles

```

1. parameter: Type u_1
2. parameter: MeasurableSpace Parameter
3. parameter: NormedAddCommGroup Parameter
4. parameter: InnerProductSpace ℝ Parameter
5. assumption: CompleteSpace Parameter
6. parameter: PZMH20PerformativePrediction.Theorem310CorrectedRGDAIIRoundTrajectoryData Parameter
7. conclusion: ∃ (entryIteration : ℕ),
  (AppliedModelingLib.measurePerformativeHeterogeneousBatchTraceInfiniteLaw data.model data.sampling
    (AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule data.dimension
      data.cutoff data.eta data.alpha data.gamma data.momentBound data.tailBound data.scaledDeviation data.p
      data.headTolerance)
    data.deployedOfHistory data.hmeasurableDeployed).real
{trace :
  (i : ℕ) →
    AppliedModelingLib.HeterogeneousBatchTraceCoordinate
    (fun (round : ℕ) =>
      Fin
        (AppliedModelingLib.pOneFournierGuillinTheorem310SupercriticalCappedConcreteCountSchedule data.dimension
          data.cutoff data.eta data.alpha data.gamma data.momentBound data.tailBound data.scaledDeviation data.p
          data.headTolerance round))
    (EuclideanSpace ℝ (Fin data.dimension)) i |
  ∀ (iteration : ℕ),
    entryIteration ≤ iteration →
      dist
        (data.deployedOfHistory iteration
          (AppliedModelingLib.heterogeneousBatchTraceOfInfiniteTrace iteration trace))
        data.stable ≤
        data.radius} ≥
1 - data.p

```

Lean proof endpoint (built separately)

PZMH20PerformativePrediction.theorem310CorrectedRGDAIIRoundTrajectoryReview

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: The selected route displays the archival Theorem 3.10(b) text together with a complete approved corrected target, its two defect identifiers, scope note, and exact approval record, and it expressly disclaims archival equivalence. The carrier exposes dimension > 2, the capped supercritical count and inverse-square confidence schedules, a uniform adaptive moment/tail envelope, global joint smoothness, strong convexity, Wasserstein sensitivity, measurability, loss differentiation and gradient-integrability premises, the variational nonexpansive projection, the source step-size and small-sensitivity inequalities, the exact empirical REGD recurrence, a stable point, and the numerical contraction, absorption, budget, radius, and initial-distance conditions named by that corrected target. The Spec quantifies over every such carrier and chooses one entry iteration outside the event; the displayed trace law samples each round's batch conditionally IID from the data law at the history-dependent deployment, and the event requires every iterate at or after entry to lie within radius with measure at least 1 - p. The broader p > 0 and radius domains retain all source-relevant nontrivial cases and only add cases to the universal result.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation